

Logarithmic-Depth Pseudorandom Functions from Well-Founded Code-Based Assumptions

Yulong Ding*

Aayush Jain[†]

Ilan Komargodski[‡]

Abstract

We give the first NC^1 -computable Pseudorandom Function (PRF) constructions from well-founded (non-ad-hoc) code-based assumptions. Specifically, we give two constructions based on two different classical variants of the learning parity with noise (LPN) assumption:

- An NC^1 -computable PRF from hardness of Sparse-LPN [Alekhnovich, FOCS '03] (with respect to a sublinear-depth explicit expander graph family). This PRF is also key-homomorphic.
- An NC^1 -computable PRF from hardness of Ring-LPN [Heyse et al., FSE '12].

Both of these assumptions have been used and studied for many years. Notably, within the range of parameters that we need, none of these assumptions is known to imply collision-resistant hashing, and the first is not known to imply public-key encryption. Prior constructions of code-based PRFs (let alone key-homomorphic) are either super-logarithmic depth, or rely on ad-hoc and new assumptions.

As a bonus, we give a similar result relying on the *classical* LPN assumption, albeit with quasi-polynomial hardness:

- A key-homomorphic NC^1 -computable PRF from quasi-polynomial hardness of classical LPN [Blum et al., CRYPTO '93].

Technically, all of our results are obtained via a refinement and substantial extension of the recent ideas of [Ding, Jain, and Komargodski, STOC '25].

*The Hebrew University of Jerusalem, Israel. Email: yulong.ding@mail.huji.ac.il.

[†]Carnegie Mellon University, USA. Email: aayushja@andrew.cmu.edu.

[‡]The Hebrew University of Jerusalem, Israel. Email: ilank@cs.huji.ac.il.

Contents

1	Introduction	1
1.1	Our Results	3
2	Technical Overview	5
3	Preliminaries	13
3.1	Pseudorandom Functions	14
3.2	Code-Based Assumptions	16
3.2.1	The LPN Assumption	16
3.2.2	The RingLPN Assumption	16
3.2.3	The SparseLPN Assumption	18
3.3	Bernoulli Sample Function $S\text{-Ber}$ over $\mathbb{F}_2$	20
3.4	Additional Notation for Matrix Operations	21
4	PRFs from Quasi-Polynomial Hardness of LPN	23
4.1	Reduction to LPN with Regular Sparse Coefficients	23
4.2	Functionality	24
4.3	Construction	26
5	PRFs from Polynomial Hardness of RingLPN	34
5.1	Bernoulli Sample Function over $\mathbb{F}_q$	34
5.2	Functionality	34
5.3	Construction	36
6	PRFs from Polynomial Hardness of SparseLPN	40
6.1	Functionality	40
6.2	Walking on Expander Graphs	42
6.3	Construction	45
A	Unrolled PRF Construction (with SPN structure)	49
B	Omitted Proofs	50
B.1	Proof of Lemma 4	50
B.2	Proof of Lemma 8	50
B.3	Proof of Lemma 10	51
B.4	Proof of Lemma 16	52
B.5	Proof of Theorem 5	53
B.6	Proof of Theorem 7	57

1 Introduction

Pseudorandom functions (PRFs) are a family of efficient keyed functions $f_{\mathbf{k}}: \{0, 1\}^n \rightarrow \{0, 1\}^{m(n)}$ such that for a randomly chosen key $\mathbf{k}$ (from a key space), $f_{\mathbf{k}}(\cdot)$ is indistinguishable from a random function $R(\cdot)$ by an efficient adversary who has oracle access to it. Since their introduction (Goldreich, Goldwasser, and Micali [GGM86]), PRFs have emerged as a cornerstone of modern cryptography, underpinning an extraordinary range of constructions across nearly every domain, as well as providing a tool to answer questions in complexity theory, learning, data structures, and more. Given this influence, understanding the inherent computational complexity of PRFs is a long-standing holy grail in foundational cryptography.

In terms of theoretical feasibility, it is well known that PRFs are existentially equivalent to one-way functions [GGM86, HILL99], the minimal cryptographic assumption. However, this sequence of transformations incurs a non-trivial cost in efficiency [HRV13, VZ12, BMM24]. A central measure of efficiency of PRFs is their parallel complexity, or circuit depth. Often, the depth of PRFs directly affects efficiency of applications, beyond computational complexity. For instance, it affects round complexity of secure computation protocols [DI06, IKOS08, BJKL21], enables advanced cryptographic systems [GVW12, Zha25], and gives applications in computational complexity [Val84, KV89, RR97, IL90, BFKL93] and meta-complexity [AN25]. Unfortunately, GGM’s construction [GGM86] is not “depth-optimal:” it transforms a polynomial-stretch pseudorandom generator (PRG) on n input bits, with polynomial stretch, and computable in depth $d(n)$, into a PRF computable in depth $\omega(d(n) + \log n)$.

Constructions of lower-depth PRFs are known from almost any standard (“algebraic”) assumption. Naor, Reingold, and Rosen [NR97, NR04, NRR02] gave constructions in $\text{TC}^0 \subseteq \text{NC}^1$ from DDH and factoring. Lewko and Waters [LW09] gave a construction in TC^0 from the k -Linear assumption (which is possibly weaker than DDH). Banerjee, Peikert, and Rosen [BPR12] gave a construction in TC^0 from the ring variant of learning with errors assumption (Ring-LWE), where the modulus to noise ratio is super-polynomial. Viola [Vio15] gave a PRF in $\text{AC}^0[\oplus]$ assuming Quasi-Polynomial Hardness of factoring.¹ See Table 1 for a list of currently known results. It is worth noting that there is a generic transformation [IKOS08, FLY22], turning any PRF in NC^1 into a PRF with $(2+\epsilon)n$ size and $(1+\epsilon)\log n$ depth. We refer to [BR17] for an elaborate overview of prior work in this area.

Towards Code-Based NC^1 PRFs. Code-based cryptography, dating back to [McE78], relies on the average-case hardness assumptions for decoding noisy linear codes. It is often referred to as Learning Parity of Noise (LPN) in the cryptography literature [BFKL93] and as syndrome decoding in the coding theory community [Pra62]. Given a field $\mathbb{F}$, let $\mathcal{C}$ be a code generation algorithm such that $\mathcal{C}(n, m)$ outputs a matrix in $\mathbb{F}^{n \times m}$. For $\mu \in (0, 1/2)$, let $\text{Ber}_\mu(\mathbb{F})$ denote the distribution which outputs a uniformly random nonzero element in $\mathbb{F} \setminus \{0\}$ with probability μ , and 0 with probability $1-\mu$. An LPN assumption with respect to $\mathcal{C}$ postulates that the following two distributions are computationally indistinguishable:

$$(\mathbf{A}, \mathbf{s} \cdot \mathbf{A} + \mathbf{e}) \approx (\mathbf{A}, \mathbf{u}), \quad \text{where } \mathbf{A} \leftarrow \mathcal{C}(n, m) \in \mathbb{F}^{n \times m}, \mathbf{s} \leftarrow \$ \mathbb{F}^n, \mathbf{e} \leftarrow \$ \text{Ber}_\mu^m(\mathbb{F}), \mathbf{u} \leftarrow \$ \mathbb{F}^m.$$

¹ TC^0 consists of all languages decidable by constant depth and polynomial size Boolean circuits, containing only unbounded fan-in AND gates, OR gates, NOT gates, and majority gates. NC^1 consists of all languages decidable by polynomial size and logarithmic depth circuits, containing constant fan-in AND, OR, and NOT gates. AC^0 consists of all languages decidable by polynomial size and constant depth circuits, containing unbounded fan-in AND, OR, and NOT gates. $\text{AC}^0[m]$ is similar to AC^0 except that the circuits can additionally have unbounded fan-in MOD_m gates. ACC^0 is the union over all m of $\text{AC}^0[m]$.

Informally, it postulates that noisy decoding of $\mathcal{C}$ is hard (viewing $\mathbf{s}$ as a message, and $\mathbf{s} \cdot \mathbf{A} + \mathbf{e}$ as a noisy codeword). A unique feature of LPN-style assumptions is that, as long as the underlying code $\mathcal{C}$ has a large dual distance, they are unconditionally secure against linear tests, a framework that captures many existing attacks on noisy linear-equation assumptions.

The classical LPN [BFKL93] is recovered by letting $\mathbb{F} = \mathbb{F}_2$ and $C(n, m)$ be the uniform distribution over $\mathbb{F}_2^{n \times m}$. Commonly used variants include:

- **SparseLPN** [Ale03, Ale11], where $\mathcal{C}$ outputs a column-sparse matrix $\mathbf{A}$ such that $\mathbf{A}$ is (the adjacency matrix of) a good expander graph. The most common instantiation of **SparseLPN** uses a trivial expander graph, which corresponds to a random column-sparse matrix². Jumping ahead, we will adopt a slightly more structured instantiation.

SparseLPN is closely related to the syndrome decoding of the duals of expander codes. Since its introduction, it has been studied and used for over a couple of decades in cryptography and computational learning theory [AIK08, IKOS08, ABW10, ADI⁺17, JLS21, JLS22, DIJL23, JLS24, RVV24, CHKV25, BBTv25, DJK25, GHJS25, CSZ25]. It is also closely related to Goldreich’s local function (also known as “expander-based cryptography” in the literature) [Gol00, BQ12, OW14, App16, AR16, OST19, OST22], where instead of injecting fresh noise, a fixed, handcrafted predicate f is applied separately on each subset of $\mathbf{s}$ touched by each column of $\mathbf{A}$.

- **RingLPN** [HKL⁺12, BCG⁺20], where $\mathcal{C}$ outputs a matrix $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_{m/n})$, such that each $\mathbf{A}_i \in \mathbb{F}_q^{n \times n}$ is induced by a random element in a degree- n cyclotomic polynomial ring $\mathcal{R}_q$. This assumption is closely related to the syndrome decoding of random quasi-negacyclic codes. It has received growing attention in recent years [BL12, DP12, LP15, GJL15, BCG⁺20, LWYY24] due to its appealing efficiency advantages over the classical version of LPN.

Key-Homomorphism. A PRF $f_{\mathbf{k}}$ is key-homomorphic (KH) if for *any two keys* $\mathbf{k}_1, \mathbf{k}_2$, and any input x , it holds that $f_{\mathbf{k}_1 \oplus \mathbf{k}_2}(x) = f_{\mathbf{k}_1}(x) + f_{\mathbf{k}_2}(x)$ [NPR99, BLMR13]. KH-PRFs have many useful applications in symmetric cryptography and give rise to distributed PRFs, symmetric-key proxy re-encryption, and updatable encryption [BLMR13, EPRS17, LT18, KLR19, BDGJ20, BEKS20], OT extension [Sch18], and single-server secure aggregation [LLPT23]. In the plain model (no random oracles), existing KH-PRFs constructions satisfy a relaxed notion called almost-key-homomorphism [BLMR13], where the homomorphism requirement is “approximately” correct, i.e., $f_{\mathbf{k}_1 \oplus \mathbf{k}_2}(x) \in f_{\mathbf{k}_1}(x) + f_{\mathbf{k}_2}(x) \pm \delta$ for some (typically small) “error” parameter $\delta > 0$.³

A recent work of Ding, Jain, and Komargodski [DJK25] made significant progress towards logarithmic-depth (NC^1) PRFs from LPN-style assumptions, a major open problem in foundational cryptography. They give:

- A *weak* PRF computable in NC^1 from classical LPN. In a *weak* PRF the adversary is only given (input,output)-samples for a random key and uniformly random inputs.
- A (strong) PRF computable in $\text{NC}^{1+\epsilon}$ from **SparseLPN**.

²This relies on the fact that a random column-sparse matrix is a good expander with high probability.

³Approximate key-homomorphism suffices for all of the above mentioned applications.

- A (strong) PRF computable in NC^1 from the *super-polynomial hardness* of a *new, ad hoc* variant of the LPN assumption (called “seeded LPN”)⁴.

Despite this progress, it remains open whether one can obtain an NC^1 -computable *strong* PRF from a *well-founded* code-based assumption. We ask:

Are there NC^1 -computable PRFs based on well-founded code-based assumptions?

It is also worth noting that their constructions are key-homomorphic. To date, the only known constructions of key-homomorphic *strong* PRFs from *well-founded* assumptions are based on LWE (e.g., [BLMR13, BP14, Kim20]) and the aforementioned Sparse-LPN-based construction; however, none of these achieves NC^1 . We further ask:

Are there NC^1 -computable KH-PRFs based on well-founded assumptions?

1.1 Our Results

In this paper, we propose new constructions of PRFs that answer both of the above questions affirmatively. We refer to Table 1 for a summary. We show for the first time the possibility of building NC^1 and key-homomorphic PRFs from well-founded code-based assumptions.

Theorem 1 (Informal; see Corollary 6). *There exist KH-PRFs computable by NC^1 circuits assuming polynomial hardness of SparseLPN (with respect to an explicit expander graph family whose neighbor function is computable in sublinear depth).*

Remark 1. We note that the above construction is the first NC^1 KH-PRF from *any* well-founded assumption to be in NC^1 and key-homomorphic. In addition, in the range of parameters we use, the underlying assumption is not known to imply collision-resistant hashing or public-key encryption.

Remark 2 (Security of SparseLPN). The original SparseLPN assumption [Ale03] posits that it holds with respect to any expander with sufficiently good expansion. We note, however, that this formulation becomes false in the quasi-polynomial sample regime (i.e., $m = n^{\omega(1)}$). In particular, it is violated when the expander is computable by subexponential-size $\text{AC}^0[\oplus]$ circuits [OST19, OST22]. The reason is that the resulting noisy function can then be approximated by a subexponential-size $\text{AC}^0[\oplus]$ circuit, which is PAC-learnable [RR97, CIKK16]. Nevertheless, to date there is still no known counterexample to SparseLPN in the standard polynomial-sample regime. Our results require only that the assumption hold for a slightly super-linear number of samples (i.e., $m = n^{1+\epsilon}$) and for expanders of slightly sublinear depth (see Definition 13).

Remark 3 (How much structure is sufficient for NC^1 PRFs). Note that a random column-sparse matrix can be viewed as a trivial expander graph whose neighbor function is computable in exponential size and linear depth (measured in terms of the input length to the neighbor function), by hardwiring neighbor lists and performing a lookup. For our PRFs to be computable in NC^1 , it suffices to use any expander whose neighbor function is computable in *sublinear depth*; importantly, we impose no non-trivial restriction on its *size*, which may be super-polynomial or even sub-exponential. In this sense, the amount of “structuredness” we require is mild. In addition, one

⁴Specifically, they first build a PRG G with super-polynomial stretch and random access to its output, which is why a super-polynomial assumption is required. The induced PRF is obtained by setting $F_k(i) := G(k)_i$.

should therefore expect that essentially any non-trivial explicit expander construction meets our requirement. Indeed, known explicit families have neighbor functions computable in polynomial size and polylogarithmic depth [TUZ01, TUZ07, GUV09, KT22].

Our second result is NC^1 PRFs from RingLPN, which can be viewed as a highly structured variant of LPN.

Theorem 2 (Informal; see Corollary 4). *There exist PRFs computable by NC^1 circuits assuming polynomial hardness of RingLPN.*

Remark 4. We note that the underlying assumption of the above construction is not known to imply collision-resistant hashing.

Table 1: Known constructions of NC^1 -computable PRFs. Newly conjectured/ad hoc/heuristic assumptions are highlighted with yellow background. **KH** stands for key homomorphism. t is the adversary running time, ϵ is the adversary advantage. For code-based assumptions, n is the dimension of the code, m is the length of the code, μ is the Bernoulli noise rate, d is the degree of the expander code. The constant polynomial power of 1.01 can be made arbitrarily close to 1.

Construction	Assumption	Category	KH
[NR97, NR04]	DDH	Group-Based	✗
[LW09]	k -Linear	Group-Based	✗
[NR97, NR04] [NRR02]	Factoring	Number Theory-Based	✗
[MV12, MV15]	Ad Hoc	SPN-Based	✗
[BPR12]	$[n, m = \text{poly}(n), q = n^{\omega(1)}, t = \text{poly}(n), \epsilon = n^{-\omega(1)}]_{\mathcal{R}_q}$ -Ring-LWE	Lattice-Based	✗
[AR16]	Strong Goldreich's OWF	Expander-Based	✗
[BIP ⁺ 18]	Hash + Alternating Moduli	Alternating Moduli-Based	✗
[DJK25]	$[n, m = n^{\omega(1)}, \mu = n^{-o(1)}, t = n^{\omega(1)}, \epsilon = \text{negl}(\lambda)]_2$ -Seeded-LPN	Code-Based	✓
This work	$[n, m = \text{poly}(n), \mu = \frac{\text{poly}(\log n)}{n}, t = \text{poly}(n), \epsilon = \text{negl}(n)]_{\mathcal{R}_q}$ -RingLPN	Code-Based	✗
This work	$[n, m = n^{1.01}, \mu = n^{-1/\log \log n}, t = \text{poly}(n), \epsilon = \text{negl}(n)]_2$ -SparseLPN	Code-Based	✓

Finally, as a bonus, we construct an NC^1 KH-PRF, albeit under the *quasi-polynomial hardness* of classical LPN assumption. The non-trivial aspect of the above result is the combination of having logarithmic depth and being key-homomorphic (while relying on the classical LPN assumption). Indeed, one can either construct key-homomorphic PRFs with *super-logarithmic* depth using SparseLPN (namely, [DJK25]), or construct a *non-key-homomorphic* log-depth PRF from quasi-polynomial hardness of LPN by converting the complexity-leveraged weak PRF in [DJK25] to a strong PRF via [NR95]. Achieving both properties under quasi-polynomial hardness of standard LPN turns out to be non-trivial.

Theorem 3 (Informal; see Corollary 2). *There exist KH-PRFs computable by NC^1 circuits assuming quasi-polynomial hardness of (classical) LPN.*

Remark 5 (SPN-like structure). *As in [DJK25], our PRFs also nicely fit into the substitution-permutation network (SPN) design framework used in modern block ciphers (e.g. AES). Indeed, our constructions can be viewed as iterative procedures, where each iteration “confuses” the input by dividing it into bundles and applying a substitution function (S-box) to each bundle, “diffuses” the bundles by applying a matrix with certain “branching” properties, and then performs “key mixing” by XORing with the private round key. See Appendix A for details.*

Remark 6. *As in [DJK25], our PRFs support a slightly super-polynomial input domain $\mathcal{X} = \{0, 1\}^{\omega(\log \lambda)}$. Using generic domain-extension techniques, one can obtain an exponentially large input domain [Lev85, BHK13, DS15] while preserving the logarithmic depth; however, these techniques do not preserve key-homomorphism.*

Remark 7 (Polynomial Security). *Our PRF security notion is the standard notion of polynomial security (i.e., any polynomial time adversary’s advantage is bounded by a negligible function). We are not aware of any quasi-polynomial time attack and conjecture our PRFs achieve sub-exponential security (as long as the adversary can make at most a polynomial number of queries).*

Conceptual Implications: Code-based vs. Lattice-based. We find the conceptual implications of this work intriguing. Recall that, historically, the development of code-based cryptography has arguably lagged behind that of lattice-based cryptography. In the context of PRFs, lattice-based constructions achieved:

1. NC^1 PRFs from Ring-LWE (i.e., [BPR12]), and
2. (Super-log-depth) KH-PRFs from LWE (i.e., [BLMR13]).

By obtaining NC^1 PRF from RingLPN, and NC^1 KH-PRF from SparseLPN, code-based cryptography appears to match, and in some sense even surpass, lattice-based cryptography in the context of PRFs.

2 Technical Overview

Before we describe the main ideas behind our constructions we set up some minimal notations for the overview.

Notation: We consider inputs $x \in [\lambda]^k$, where λ is the security parameter and $k = \omega(1)$ is a small super-constant (e.g., $k = \log \log \log \lambda$). Let $n = \text{poly}(\lambda)$ denote the dimension of the LPN assumption, and also the PRF output length. Let μ be the noise rate, with $\mu \gg 1/n$. We write $\ell := \log(1/\mu)$ for the number of random bits needed to sample a Bernoulli noise Ber_μ by taking a product (i.e., for $\mathbf{r} \in \{0, 1\}^\ell$, $\text{S-Ber}(\mathbf{r}) := \prod_{i \in \ell} r_i \in \{0, 1\}$). S-Ber is extended by applying it on each ℓ -bit block separately, therefore shrinking input length by a factor of ℓ . Finally, our constructions are defined recursively with a parameter $\tau = \omega(1)$ (e.g., $\tau = \log \log \log \lambda$) that upper bounds the number of recursion layers.

Since our constructions of low-depth PRFs extend and generalize ideas from [DJK25], it will be beneficial to start by first recalling their ideas.

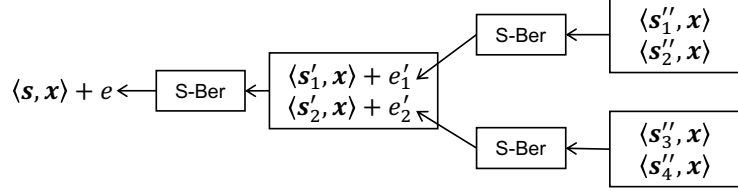


Figure 1: An illustration of weak PRFs from LPN via recursive derandomization, in a toy case of recursion depth $\tau = 2$, and $\ell = 2$, so $\text{S-Ber}: \mathbb{F}_2^2 \mapsto \mathbb{F}_2$. The secret key is 7 vectors $\mathbf{s}, \mathbf{s}'_{1/2}, \mathbf{s}''_{1/2/3/4} \in \mathbb{F}_2^n$. On input $\mathbf{x} \in \mathbb{F}_2^n$, the output of the weak PRF F is $\langle \mathbf{s}, \mathbf{x} \rangle + e \in \mathbb{F}_2$, where $e := \text{S-Ber}(\mathbf{r}') \in \mathbb{F}_2$ is obtained by applying S-Ber to the output of the preceding layer $\mathbf{r}' := (\langle \mathbf{s}'_i, \mathbf{x} \rangle + e'_i)_{i \in [2]} \in \mathbb{F}_2^2$, where $\{e'_i\}_{i \in [2]} := \text{S-Ber}(\mathbf{r}'') \in \mathbb{F}_2^2$ is again obtained by applying S-Ber to the output of its preceding layer $\mathbf{r}'' := (\langle \mathbf{s}''_i, \mathbf{x} \rangle)_{i \in [4]} \in \mathbb{F}_2^4$. This is the base layer and no noise is added.

Recursive Derandomization. A main idea of [DJK25] is that the LPN assumption almost directly gives us a weak PRF in NC^1 . If one could materialize a function $F_{\mathbf{s}}(\mathbf{x}) = \langle \mathbf{s}, \mathbf{x} \rangle + e$ where $\mathbf{s} \in \mathbb{F}_2^n$ and e is a Bernoulli error chosen with probability μ , then this would form a weak PRF by the LPN assumption with noise rate μ . While the linear term $\langle \mathbf{s}, \mathbf{x} \rangle$ is NC^1 computable, it is less clear from which one derives e . Ding et al. [DJK25] suggest to generate the error recursively (illustrated in Figure 1), by calling functions $F_{\mathbf{s}'}(\mathbf{x})$ ℓ times to obtain ℓ bits, and deriving the noise by applying a Bernoulli sampling function S-Ber_μ .

We now describe the main ideas behind proving its security and NC^1 computability.

Efficiency:⁵ The key observation for analyzing the required circuit depth is that each inner product of the form $\langle \mathbf{s}, \mathbf{x} \rangle$ can be computed in $\log n$ depth in parallel. Furthermore, each S-Ber requires $\log \ell$ depth to multiply ℓ bits. So the total evaluation depth is

$$O(\log n + \tau \cdot \log \ell).$$

As long as $\ell = o(n)$, we can evaluate the function in $O(\log \lambda)$ depth for sufficiently small super constant τ .

Security: The security proof relies on a certain resiliency property of the constructed function, such that any changes to the base layer (the right most layer in the picture 1) will not change the output with high probability. The intuition is that each S-Ber is insensitive to its input and outputs the value 0 with probability $1 - 2^{-\ell}$. Therefore, as we sequentially apply S-Ber for τ times, the value at the final layer will hardly change. The idea in the proof is that one could start with a random string at the base (rightmost layer) and can replace $\{\langle \mathbf{x}, \mathbf{s}''_i \rangle\}$ by a fresh random vector. This gives us almost the same function (in the sense that the output is unchanged), but for this function we can prove the (weak) pseudorandomness under LPN. Indeed, random vectors help us derive truly Bernoulli distributed error and appeal to the security of LPN inductively layer by layer to maintain this invariant.

Extending to PRFs via a GGM-like approach. The above approach produces a weak PRF in NC^1 from LPN. To achieve a strong PRF, [DJK25] suggested to extend the above technique to a GGM-like construction. The observation is that we can view LPN samples as a “PRG” stretching a

⁵Note that it suffices to ensure logarithmic depth, since with bounded fan-in gates (in particular, fan-in 2), this automatically implies polynomial size.

PRF, recursively defined

Global Parameters: Max recursion layer τ . Let $T = O((k\ell)^\tau)$,

$$\mathbf{S}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad \mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_\lambda), \text{ where } \mathbf{A}_i \in \mathbb{F}_2^{n \times n} \text{ for } i \in [\lambda].$$

Notation: Denote $\mathbf{M}_{(i,j)} = \prod_{l=i}^j \mathbf{A}_{x_l}$.

For each input length $k' = 1, \dots, k$, we define $F_{(k')}$ as:

Function $F_{(k')}(x \in \{0, 1\}^{k'}, \text{ layer})$:

1. If $\text{layer} = \tau$: return $\mathbf{0} \in \mathbb{F}_2^n$.
2. For each input prefix $i = 1, \dots, k'$, call $F_{(i)}$ ℓ times (denoted by $F_{(i)}^\ell$) and derive the noise vector:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_\mu(\mathbf{R}_i) \in \mathbb{F}_2^n, \quad \text{where } \mathbf{R}_i \leftarrow F_{(i)}^\ell(x_{1,\dots,i}, \text{layer} + 1) \in \mathbb{F}_2^{\ell \times n},$$

3. Let $\mathbf{s} \in \mathbb{F}_2^n$ be an unused row of $\mathbf{S}_{(k')}$:

$$\begin{aligned} \mathbf{y} &\leftarrow (\cdots (\mathbf{s} \cdot \mathbf{A}_{x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{x_2} + \mathbf{e}_2 \cdots) \cdot \mathbf{A}_{x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{s} \cdot \mathbf{M}_{(1,k')} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \mathbf{M}_{(i+1,k')} \in \mathbb{F}_2^n, \end{aligned} \tag{1}$$

4. Return $\mathbf{y} \in \mathbb{F}_2^n$.

Figure 2: We define $\text{PRF}(x) = F_{(k)}(x, 0)$. Observe that a single evaluation of PRF makes a total of $T = O((k\ell)^\tau)$ recursive calls. Each call uses a vector $\mathbf{s}$ corresponding to a previously unused row of $\mathbf{S}_{(k')}$. Consequently, the ℓ invocations of $F_{(i)}$ are performed with different $\mathbf{s}$, and hence return different values.

seed $\mathbf{s} \in \mathbb{F}_2^n$ to $\mathbf{s} \cdot \mathbf{A} + \mathbf{e} \in \mathbb{F}_2^m$. Then, we can repeatedly apply a PRG guided by an input-dependent selection. The resulting construction is the recursive derandomization of the following:

$$\begin{aligned} \mathbf{y} &\leftarrow (\cdots (\mathbf{s} \cdot \mathbf{A}_{x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{x_2} + \mathbf{e}_2 \cdots) \cdot \mathbf{A}_{x_k} + \mathbf{e}_k \\ &= \mathbf{s} \prod_{i=1}^{k'} \mathbf{A}_{x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{A}_{x_j} \end{aligned} \tag{2}$$

More formally, on input $x \in [\lambda]^k$, the PRF outputs

$$\text{PRF}(x; \mathbf{S}, \mathbf{A}) := F_{(k)}(x, 0), \text{ where } F_{(k)} \text{ is recursively defined in Figure 2.}$$

A few remarks on how exactly the recursion is done. For a GGM-style proof to work, the error $\mathbf{e}_1$ should depend only on x_1 , $\mathbf{e}_2$ should depend on x_1 and x_2 , and any $\mathbf{e}_i$ should depend on the

prefix of the input of length i . Therefore, in Figure 2, the function is defined for any input length $k' \leq k$. For $i \in [k]$, it derives $\mathbf{e}_i$ by calling $F_{(i)}$ ℓ times. The total depth of recursion is τ at which point the function returns $\mathbf{0}$ (thereby initiating the base layer with $\mathbf{0}$).

If each column of $\mathbf{A}$ is sparse, then

- For **security**, similar to weak PRFs, one can prove a resiliency property that the output is not sensitive to *any* changes in the base layer. Because the product of sparse matrices remain sparse. The matrix product $\prod \mathbf{A}$ is itself sufficiently sparse so that the computing $\sum \mathbf{e} \cdot \prod \mathbf{A}$ exhibits only local dependency on $\mathbf{e}$.
- For **key homomorphism**, since $\mathbf{e}$ and $\mathbf{A}$ are sparse, $\sum \mathbf{e} \prod \mathbf{A} \approx \mathbf{0}$. Therefore, the constructed PRF is close to a linear function on $\mathbf{s}$.

The largest challenge is NC^1 -computability. In general, computing the product of $k = \omega(1)$ (sparse) matrices is unlikely to be NC^1 . Therefore, [DJK25] only constructed an $\text{NC}^{1+\epsilon}$ PRF from SparseLPN and had to settle the weaker notion of an encoded-input PRF (wherein the non- NC^1 computation is offloaded to a preprocessing phase). They then resort to a different approach to construct NC^1 PRFs, which necessitates a super-polynomial (ad hoc) assumption (called “Seeded-LPN”).

An equivalent recursive definition

Let global parameters and notations be as in Figure 2.

For each input length $k' = 1, \dots, k$, we define $F_{(k')}$ as:

Function $F'_{(k')}(x \in \{0, 1\}^{k'}, \mathcal{I} \subseteq [n], \text{layer})$:

1. If $\text{layer} = \tau$: return $\mathbf{0} \in \mathbb{F}_2^n$.
2. For each input prefix $i = 1, \dots, k'$:

$$\mathcal{I}_i \leftarrow \text{RowSupp}(\overline{\mathbf{M}}_{(i+1, k')}), \text{ where } \overline{\mathbf{M}}_{(i+1, k')} \leftarrow \mathbf{M}_{(i, k')}|_{\mathcal{I}}$$

$$\bar{\mathbf{e}}_i \leftarrow \text{S-Ber}_{\mu}(\mathbf{R}_i|_{\mathcal{I}_i}) \in \mathbb{F}_2^{|\mathcal{I}_i|}, \text{ where } \mathbf{R}_i|_{\mathcal{I}_i} \leftarrow (F'_{(i)})^{\ell}(x_{1\dots i}, \mathcal{I}_i, \text{layer} + 1) \in \mathbb{F}_2^{\ell \times |\mathcal{I}_i|},$$

3. Let $\mathbf{s} \in \mathbb{F}_2^n$ be an unused row of $\mathbf{S}_{(k')}$: (letting $\bar{\mathbf{s}} \leftarrow \text{ColSelect}(\mathbf{s}, \mathcal{I}_1)$)

$$\mathbf{y} = \bar{\mathbf{s}} \cdot \overline{\mathbf{M}}_{(1, k')} + \sum_{i=1}^{k'} \bar{\mathbf{e}}_i \cdot \overline{\mathbf{M}}_{(i+1, k')} \in \mathbb{F}_2^{|\mathcal{I}|}, \quad (3)$$

4. Return $\mathbf{y} \in \mathbb{F}_2^{|\mathcal{I}|}$.

Figure 3: An equivalent recursive definition.

In this work, we provide three new constructions of standard PRFs (two homomorphic, and one without homomorphism) from well-studied assumptions, circumventing the challenges faced

by [DJK25]. Specifically, we manage to compute the above recursively defined function in NC^1 via three instantiations of the choice of the distributions of $\mathbf{S}$ and $\mathbf{A}$ (while admitting reductions to well-studied assumptions). All of them are analyzed in a unified framework. Below we set the stage that is common to our constructions.

A Unified Framework. Our PRF function is defined recursively, essentially as in Figure 2 with respect to parameters $\mathbf{S}, \mathbf{A}$. (For now, we do not specify the exact distributions of these matrices, but only assume that $\mathbf{A}$ is a d -(column) sparse matrix, which is crucial to prove the resiliency property.) One technical but important modification we make is that before returning $\mathbf{y}$ after each execution of $F_{(k')}$, we add a random padding $\mathbf{p}$ (treated as part of the key) to $\mathbf{y}$. This does not change the depth of the computation asymptotically, or, the key homomorphism properties of the output. However, this makes the analysis of the resiliency property *independent of* the distributions of the sparse matrices $\mathbf{A}$. Though simple in hindsight, this is important because it allows us to freely choose the distributions of $\mathbf{A}$ in order to squeeze the depth of PRF. For simplicity, from now on we ignore this additional padding in the remainder of this section.

We consider an equivalent formulation, which we call PRF' that is simply an output restricted version of PRF defined before. Namely, this PRF' on input: $x \in [\lambda]^k$, and a support $\mathcal{I} \subseteq [n]$, outputs $\mathbf{y} = \text{PRF}(x)|_{\mathcal{I}} \in \mathbb{F}_2^{|\mathcal{I}|}$, corresponding to indices in $\mathcal{I}$ as opposed to the entire output.

The reason we work with this augmented notation is that it helps us unravel the structure of the computation via a recursive framework that we can hope to analyze better. We claim that

$$\text{PRF}(x) = \text{PRF}'(x, \{1, \dots, n\}) = F'_{(k)}(x, \{1, \dots, n\}, 0)$$

where F' is defined in Figure 3. Note that the first equality holds by definition of PRF' . The second equality holds by closer inspection of Figure 3:

1. Since the input $\mathcal{I} \subseteq [n]$ tells us we only need to return $\mathbf{y}|_{\mathcal{I}}$, we do not need to compute $\mathbf{y}|_{[n] \setminus \mathcal{I}}$.
2. Observe that $\mathbf{y}|_{\mathcal{I}} = \mathbf{s} \cdot \mathbf{M}|_{\mathcal{I}} + \sum \mathbf{e} \cdot \mathbf{M}|_{\mathcal{I}}$, where $\mathbf{M}|_{\mathcal{I}}$ is the column restriction of $\mathbf{M}$ to $\mathcal{I}$. Let $\mathcal{I}' \subseteq [n]$ be the row support of $\mathbf{M}|_{\mathcal{I}}$ (i.e., the set of row indices that have a nonzero entry). We only need to know $\mathbf{e}|_{\mathcal{I}'} = \text{S-Ber}(\mathbf{R})|_{\mathcal{I}'}$, which is actually equal to $\text{S-Ber}(\mathbf{R}|_{\mathcal{I}'})$ since S-Ber operates on each column of $\mathbf{R}$ independently.
3. Each row $\mathbf{r}$ of $\mathbf{R}$ is returned by a recursive call. Since now we are only interested in entries in $\mathcal{I}'$, we make the recursive call with $\mathcal{I}'$. This yields a recursive closure.

Now that F' provides the desired functionality for us, we can compute each output bit $j \in [n]$ separately by calling $F'_{(k)}(x, \{j\}, 0)$:

$$\text{PRF}(x) = (F'_{(k)}(x, \{j\}, 0))_{j \in [n]}.$$

Fix any $j \in [n]$ from now on. It suffices to analyze the circuit depth of $F'_{(k)}(x, \{j\}, 0)$. Let the input to the PRF be $x \in [\lambda]^k$. Observe that throughout the evaluation of $\text{PRF}(x)$, whenever a subroutine $F'_{(k')}(\cdot, \cdot, \cdot)$ is invoked, its first argument is always $x_{1 \dots k'}$. Can we also ‘predict’ the second argument of $F'_{(k')}(\cdot, \cdot, \cdot)$? We hope that it only depends on k' (as hinted by our notation in Figure 3, $F'_{(i)}$ is invoked with second argument $\mathcal{I}_i$ with a subscript that only depends on i), and independent of its actual caller $F'_{(k'')}$ for some $k'' \geq k'$?

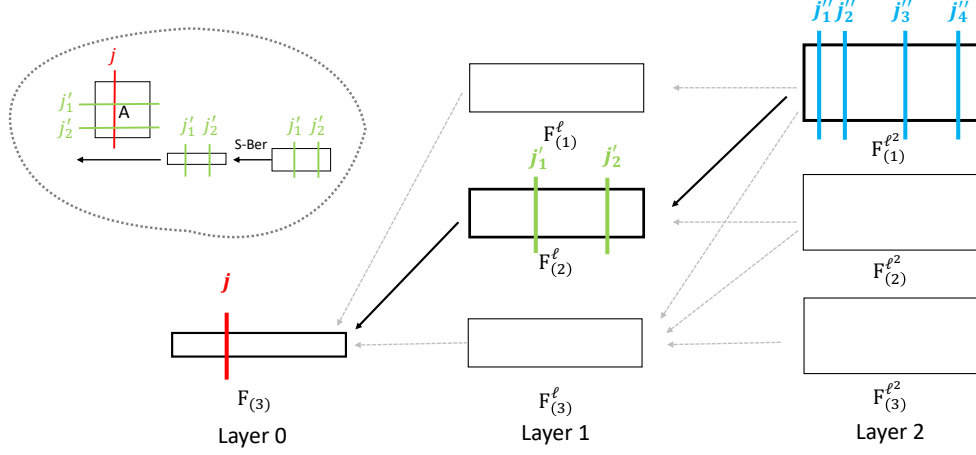


Figure 4: An illustration with $k = 3$. The dependency sequence $j \rightarrow j' \rightarrow j''$ can be viewed as a graph traversal on the layered graph induced by $\mathbf{A}_2, \mathbf{A}_3$.

We consider a slightly different interpretation of this, where these dependency sets $\mathcal{I}_1, \dots, \mathcal{I}_k$ can be viewed as follows. We can start with $\mathcal{I}_k = \mathcal{I} = \{j\}$, and then construct $\mathcal{I}_i$ inductively from $\mathcal{I}_{i+1}$: $\mathcal{I}_i$ is defined as the set of all the nodes that are reachable by taking a 1-step walk from the starting set $\mathcal{I}_{i+1}$ on a graph whose adjacency matrix is given by $\mathbf{A}_{x_{i+1}}$.

In other words, the dependency set $\mathcal{I}_{k'}$ defined above can be viewed as the *reachability set* of a $(k - k')$ -step walk on the layered graph induced by matrices $\mathbf{A}_{x_k}, \mathbf{A}_{x_{k-1}}, \dots, \mathbf{A}_{x_1}$, where the starting node is j .

Note that the $\mathcal{I}_i$ constructed this way might be slightly different from the sets that are derived according to the row-support approach described previously. This is because multiplication of $\prod \mathbf{A}$ to form $\mathbf{M}_{(i,j)}$ can introduce accidental zero terms mod 2. Nevertheless, the sets formed by taking the walks as described above contain the sets formed using the row-support approach.

This graph-traversal-based interpretation gives us a mathematical closure guarantee for these sets that will be useful for analysis. Namely, one can show that throughout the evaluation of $\text{PRF}(x)$, all of the recursive calls can always be performed by calling $F'_{(k')}(\mathcal{I}_{k'})$ for $k' \leq k$ on these sets $\mathcal{I}_1, \dots, \mathcal{I}_k$. In other words, there are fixed sets of dependency that are relevant across all recursive calls, and we do not need to worry about every coordinate.

With this understanding, the depth analysis considerably simplifies by exploiting the following facts:

- The maximum size of the sets $\mathcal{I}_1, \dots, \mathcal{I}_k$ is small. For a starting set $\mathcal{I} = \{j\}$, each walk can introduce a multiplicative factor of at most d (the column sparsity of $\mathbf{A}$), and thus, the maximum size of these sets is bounded by $w = d^k$. We set it to be a subpolynomial by choosing d to be poly-logarithmic and k to be suitable super-constant.
- If one can precompute these sets, each computation involved in recursive calls to $F'_{(k')}$ becomes a local computation that acts only on these sets $\mathcal{I}_{k'}$. The size of the set is bounded by $w = d^k$. We also note that each computation is of the form $\mathbf{e} \cdot \mathbf{M}$, corresponding to a linear function acting on at most w variables of $\mathbf{e}$. This can be computed in depth $O(\log w + \log \log q)$ if the linear function that needs to be computed can be preprocessed. Furthermore, the Bernoulli

noise derivation acts column-wise and is very local. Therefore this also does not pose a problem.

In a nutshell, we can compute the PRF function in $O(\log \lambda)$ depth if

1. The graph-traversal reachability set $\mathcal{I}_1, \dots, \mathcal{I}_k$ can be computed in $O(\log \lambda)$ depth.
2. The iterated matrix multiplication $\prod \mathbf{A}$ can be computed in $O(\log \lambda)$ depth.

With that big picture in mind, now we are ready to present how to instantiate $\mathbf{A}$ and $\mathbf{S}$, yielding three different constructions. We note that we will rely crucially on the connection between iterated matrix multiplication and graph traversal. Roughly speaking, if we can compute one of the above tasks in $O(\log \lambda)$ depth, then we can also compute the other task in $O(\log \lambda)$ depth.

Result 1: From (polynomial hardness of) SparseLPN with respect to a sublinear-depth expander graph. Consider an equivalence between a d -sparse matrix in $\mathbb{F}_2^{n \times m}$ and a d -regular expander graph. Namely, we consider d -left regular bipartite graphs where the left set of m vertices represent the columns of this matrix and the right set of n vertices represent the rows. Each left vertex in $[m]$ is connected to d vertices on the right corresponding to the d non-zero entries in the column. We now consider $\Gamma: [m] \rightarrow [n]^d$ as the neighbor function for this graph.

Random d -sparse matrices correspond to d -regular random expander graphs, whose neighbor function Γ can be computed in depth $O(\log m)$ and size $O(md \log n)$, which is in linear depth and exponential size (in its input length $\log m$). One way to implement this is to hardwire its truth table (a total of md values in $[n]$) and then perform a lookup based on input.

Our main observation is that if we consider expander graphs whose neighbor function is computable in sublinear depth, i.e., $o(\log m)$, barely smaller than random expanders, one can do a k -step graph traversal in depth $O(\log n)$ for some suitably chosen $k = \omega(1)$. Roughly speaking, one can start with a set $\mathcal{I} = \{j\}$ and apply $\Gamma(\mathcal{I}) = \{u_1, \dots, u_d\}$ to obtain $\mathcal{I}_{k-1}$. Then, one can apply Γ to the elements in this set to obtain (upon renaming and possibly with duplicates) $\mathcal{I}_{k-2} = \{u_1, \dots, u_{d^2}\}$. Continuing this way for all the k steps, we will eventually obtain a set of size d^k . So far this computation takes depth $k \cdot \text{depth}(\Gamma)$. But the resulting vector has many duplicates and is not sorted. We need to accumulate it into the desired sparse vector. This can be done in $O(\log(d^k)) = O(\log \lambda)$ depth via a bin-style sorting algorithm. We refer to the technical section for the full details.

To compute the iterated matrix product, we rely on the connection between iterated matrix multiplication and graph traversal. Namely, $\prod_i \mathbf{A}_i$ over $\mathbb{F}_2$ is equivalent to counting the number of paths (modulo 2) between each pair of nodes, which then reduces to the graph traversal problem. In more detail, we again take a k -step walk starting from each node j and obtain a reachability set (that contains many duplicates). This time we count (modulo 2) the occurrence of every node $i \in [n]$ in the reachability set.

Thus, using SparseLPN with respect to sublinear-depth expander graphs, we can construct a key-homomorphic PRF in NC^1 . In our setting, we need to work with $m = n^{1.01}$, $d = \text{poly}(\log n)$, such explicit constructions are known [TUZ01, TUZ07, GUV09, KT22]. Indeed, their constructions are NC-computable, which translates into a depth of $\text{poly}(\log \log n)$ here.

Finally, we remark that in the literature of explicit expanders, it is required that the neighbor function of the expander can be computed in polynomial time. However, for NC^1 PRFs, an

exponential time expander will also work for us, as long as it has a sublinear depth, which is barely smaller than a random expander. In this sense, we believe the required amount of “structuredness” is very mild.

Result 2: From (polynomial hardness of) RingLPN. Our second idea is to resort to a distribution of matrices $\mathbf{A}$ for which iterated product of matrices is in logarithmic depth. For this, we let the coefficient matrix $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_\lambda)$ be a random quasi-negacyclic matrix. More concretely, each $\mathbf{A}_i \in \mathbb{F}_q^{n \times n}$ corresponds to a random n -degree cyclotomic polynomial ring element $a_i \in \mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$ ⁶ (where n is a power of two and q is an appropriately chosen prime with $O(\log n)$ bits). Then, iterated product of $\prod \mathbf{A}_{x_i}$ can be done in logarithmic depth, a property also used to construct logarithmic-depth PRFs from Ring-LWE [BPR12]. In more detail, through Chinese remainder representation, iterated product of ring elements $a_x = \prod a_{x_i}$ can be reduced to the problem of iterated integer multiplication, which is in $\text{TC}^0 \subseteq \text{NC}^1$ [BCH86, HAB02]. We then convert the result from the evaluation representation back to the coefficient representation. Recall that the conversion between them is a matrix-vector multiplication over $\mathbb{Z}_q$, which is in NC^1 because both integer multiplication and iterated integer addition are in NC^1 .

Recall that $\mathbf{A}$ is required to be sparse. One can show that if a_i is a sparse polynomial, then the induced matrix $\mathbf{A}_i$ is also column-sparse. However, RingLPN with *sparse coefficients* is not known to be as hard as RingLPN. To fix this problem, we forgo key homomorphism completely and use a dual version of the template above. All we have to do is a renaming adjustment. We treat $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_\lambda)$ as λ *secret* polynomials (in matrix representation) chosen from random sparse polynomials, and treat $\mathbf{S} \in \mathbb{F}_q^{T \times n}$ as the concatenation of T *public* random polynomials (in coefficient representation). This allows us to reduce the PRF security to RingLPN with *sparse secrets chosen from the noise distribution*. This variant can be proven to be as hard as RingLPN.

Now we rely on the connection between iterated matrix multiplication and graph traversal in the other direction. Namely, we rely on iterated matrix multiplication over $\mathbb{F}_q$ to do graph traversal. Observe that if the multiplication is over $\mathbb{Z}$, then the reachability set can be simply derived from the non-zero entries in the output of iterated matrix product. Working over $\mathbb{F}_q$ may introduce accidental zero terms, but as long as $q = \lambda^{o(1)}$, the cancellation probability is negligible. In fact, a polynomial modulus $q = \text{poly}(n)$ is sufficient, as we can amplify the correctness probability by repeating.

Result 3: Key Homomorphic PRF from quasi-polynomial hardness of LPN. We now explore the impossibility of constructing NC^1 PRFs relying on LPN, without any additional structure, but assuming its quasi-polynomial hardness.

Recall that in our first result, if we are forced to use a random expander, then the only way to achieve in $O(\log \lambda)$ is to set $n = \lambda^{o(1)}$, e.g., $n(\lambda) = \lambda^{1/\log \log \lambda}$. In this case, $\log n = o(\log \lambda)$. The main obstacle is if this construction can be proven to be secure assuming standard LPN? Super-polynomial sparse LPN can be used to prove its security, but what about super-polynomial time standard LPN?

Our main idea is to carefully leverage recent reductions [BBTV25] that show hardness of sparse LPN under standard LPN. Roughly speaking, the reductions say that SparseLPN with d -sparse random coefficient, and noise rate μ is as hard as LPN with dimension $\approx d$, and the same noise rate μ . The reductions are also sample preserving.

⁶Namely, the j -th row of $\mathbf{A}_i$ is $\text{Vec}(X^{j-1} \cdot a_i)$, where $\text{Vec}(\cdot)$ denotes the coefficient vector of a ring element. It then holds that $\text{Vec}(s \cdot a_i) = \text{Vec}(s) \cdot \mathbf{A}_i$.

However, we note that it poses an implicit restriction on the sparsity d . That is, the reduction is only meaningful for $d \gg 1/\mu$, otherwise the resulting assumption LPN is clearly false. However, on the other hand, our resilience analysis suggests that we should pick μ small enough such that $1/\mu \gg d$ or else we lose the resilience property. At this point, it would appear that we are stuck.

Our idea is that instead of using the same sparsity d and noise rate μ for each of the k components of the input, we carefully craft a sequence of sparsity and error probabilities:

$$1/\mu_1 \gg d_2 \gg 1/\mu_2 \gg d_3 \gg 1/\mu_3 \gg \dots \gg 1/\mu_{k-1} \gg d_k,$$

such that it simultaneously allows a meaningful reduction and enables the resilience property.

Namely, the coefficient matrix is now $\mathbf{A} = (\mathbf{A}_{1,1}, \dots, \mathbf{A}_{k,\lambda})$. The PRF construction is the recursive derandomization of

$$\begin{aligned} \text{PRF}_{(k)}(x; \mathbf{s}) &\leftarrow (\dots (\mathbf{s} \cdot \mathbf{A}_{1,x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{2,x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{A}_{k,x_k} + \mathbf{e}_k \\ &= \mathbf{s} \prod_{i=1}^k \mathbf{A}_{i,x_i} + \sum_{i=1}^k \mathbf{e}_i \cdot \prod_{j=i+1}^k \mathbf{A}_{j,x_j} \in \mathbb{F}_2^n, \end{aligned} \tag{4}$$

where

$$\mathbf{e}_i \leftarrow \text{S-Ber}_{\mu_i}(\mathbf{R}_i) \in \mathbb{F}_2^n, \quad \text{and } \mathbf{A}_{i,\star} \in \mathbb{F}_2^{n \times n} \text{ has column sparsity } d_i.$$

Finally, we remark that the previous work of [DJK25] constructed weak PRFs in logarithmic depth from LPN. Assuming quasi-polynomial hardness of LPN, and choosing $n = \lambda^{o(1)}$, it can be converted into a synthesizer computable in $o(\log n)$ depth, which yields a PRF in $O(\log n)$ depth [NR95, NR99], but without key homomorphism. Our construction, on the other hand, preserves homomorphism. Curiously, this result gives us the first application of the reduction between LPN and sparse LPN to derive a cryptosystem based on standard LPN.

3 Preliminaries

Sets and distributions. We use $[n]$ to denote the set $\{1, \dots, n\}$. We use $\text{Funcs}[\mathcal{X}, \mathcal{Y}]$ to denote the set of all functions mapping $\mathcal{X}$ to $\mathcal{Y}$. For a probability distribution X over a domain D , let X^n denote its n -fold product distribution over D^n . The uniform distribution over a finite domain D is denoted by $U(D)$. We use Ber_μ to denote the Bernoulli distribution with parameter μ , i.e., $\Pr[\text{Ber}_\mu = 1] = \mu$, $\Pr[\text{Ber}_\mu = 0] = 1 - \mu$. Extending it over larger fields, we use $\text{Ber}_\mu(\mathbb{F})$ to denote the Bernoulli distribution that returns 0 with probability $1 - \mu$, and a uniformly random non-zero element of $\mathbb{F} \setminus \{0\}$ otherwise. By $X \sim D$ we denote that random variable X follows distribution D . For a distribution D , we use $s \leftarrow \$ D$ to denote sampling an element s according to distribution D . For a finite set S , we use $s \leftarrow \$ S$ to denote sampling s uniformly from finite set S . For a value x , we use $y \leftarrow x$ to denote assigning the value of x to y .

Vectors and matrices. We adopt the following vector convention: when we left-multiply a vector $\mathbf{v}$, it is treated as a row vector; when we right multiply a vector $\mathbf{v}$, it is treated as a column vector. For a vector $\mathbf{v}$, we use $|\mathbf{v}|$ or $\text{wt}(\mathbf{v})$ to denote the Hamming weight of $\mathbf{v}$. We denote the Hamming distance between two vectors $\mathbf{x}$ and $\mathbf{y}$ by $\Delta(\mathbf{x}, \mathbf{y})$.

For a matrix $\mathbf{M} \in \mathbb{F}^{n \times m}$, for $i \in [n]$, we use $\mathbf{M}[i, :] \in \mathbb{F}^{1 \times m}$ to denote its i -th row in $\mathbb{F}^m$. If $i = 0$, $\mathbf{M}[i, :]$ is defined to be a zero vector in $\mathbb{F}^m$. Similarly, we use $\mathbf{M}[:, j] \in \mathbb{F}^{n \times 1}$ to denote its j -th column in $\mathbb{F}^n$. If $j = 0$, $\mathbf{M}[:, j]$ is defined to be a zero vector in $\mathbb{F}^n$. For matrices $\mathbf{A} \in \mathbb{F}^{n \times m_1}$ and $\mathbf{B} \in \mathbb{F}^{n \times m_2}$, write $\mathbf{A} \parallel \mathbf{B} \in \mathbb{F}^{n \times (m_1 + m_2)}$ for the *column-wise* concatenation (appending the columns of $\mathbf{B}$ to $\mathbf{A}$). Similarly, for $\mathbf{A} \in \mathbb{F}^{n_1 \times m}$ and $\mathbf{B} \in \mathbb{F}^{n_2 \times m}$, write $\mathbf{A} \parallel \mathbf{B} \in \mathbb{F}^{(n_1 + n_2) \times m}$ for the *row-wise* concatenation (stacking the rows of $\mathbf{B}$ beneath $\mathbf{A}$). We defer more notations to Section 3.4.

Circuit model. We work in the standard Boolean-circuit model with $\{\text{AND}, \text{OR}, \text{NOT}\}$ gates of bounded fan-in. We define NC circuits as follows.

Definition 1 (NC circuits). For an integer $i > 0$, an NC^i circuit is a Boolean circuit $C_\lambda: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{m(\lambda)}$ with $\{\text{AND}, \text{OR}, \text{NOT}\}$ gates of bounded fan-in, depth $O(\log^i \lambda)$, size $\text{poly}(n)$. An NC circuit is a Boolean circuit $C_\lambda: \{0, 1\}^\lambda \rightarrow \{0, 1\}^{m(\lambda)}$ with $\{\text{AND}, \text{OR}, \text{NOT}\}$ gates of bounded fan-in, depth $\text{poly}(\log \lambda)$, size $\text{poly}(\lambda)$.

Computational indistinguishability. We use the following simplified notation to ease presentation. The PRF security parameter through this paper is λ , and all algorithms (including the adversary) are implicitly given the security parameter λ in unary, and most other parameters are functions of λ . We omit λ when it is clear from the context. We say that a function $\epsilon = \epsilon(\lambda)$ is negligible if for every $c > 0$, it is smaller than $\frac{1}{\lambda^c}$ for a sufficiently large $\lambda > N_c$.

We consider adversaries interacting as part of probabilistic experiments called games. For an adversary $\mathcal{A}$ and two games $H_0(n), H_1(n)$ with which it can interact, $\mathcal{A}$'s distinguishing advantage (as a function of n) is $\text{Adv}_{H_0, H_1}(\mathcal{A}) := |\Pr[\mathcal{A} \text{ accepts in } H_0] - \Pr[\mathcal{A} \text{ accepts in } H_1]|$. When we say that an algorithm is *efficient*, we mean that it is a probabilistic polynomial-time algorithm.

Definition 2 (Computational Indistinguishability). We say that games (or distributions) H_0, H_1 are computationally indistinguishable, and denote $H_0 \approx_{t, \epsilon} H_1$, if for any probabilistic distinguisher $\mathcal{A}$ running in time t , $\text{Adv}_{H_0, H_1}(\mathcal{A}) \leq \epsilon$.

3.1 Pseudorandom Functions

In this section, we formally define pseudorandom functions [GGM86].

Definition 3 (PRFs [GGM86]). Let $q, t, \epsilon > 0$. A keyed function $F: \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is called (q, t, ϵ) -pseudorandom if for any t -sized oracle-aided circuit $D^{(\cdot)}$ with at most q oracle gates, the distinguishing advantage

$$\left| \Pr_{k \leftarrow \mathcal{K}} [D^{F_k(\cdot)} = 1] - \Pr_{R \leftarrow \text{Funcs}[\mathcal{X}, \mathcal{Y}]} [D^{R(\cdot)} = 1] \right| \leq \epsilon.$$

Now, let $q(\cdot), t(\cdot), \epsilon(\cdot)$ all be functions of λ . A (q, t, ϵ) -PRF family is a sequence of keyed functions $\mathcal{F} = \{F_\lambda: \mathcal{K}_\lambda \times \mathcal{X}_\lambda \rightarrow \mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a polynomial time key sampling algorithm and a polynomial time evaluation algorithm where each F_λ is $(q(\lambda), t(\lambda), \epsilon(\lambda))$ -pseudorandom. We say $\mathcal{F}$ is a secure PRFs family if for any polynomial $q(\cdot), t(\cdot)$, there exists a negligible function $\epsilon(\cdot)$, such that $\mathcal{F}$ is a (q, t, ϵ) -PRFs family. We say $\mathcal{F}$ is in NC^1 if each f_λ can be evaluated by an NC^1 circuit.

It will suffice to construct NC^1 PRFs for input domain $\mathcal{X} = \{0, 1\}^{\omega(\log \lambda)}$ of slightly super-polynomial size. One can transform them to support input domain $\mathcal{X} = \{0, 1\}^\lambda$ via domain extension techniques [Lev85, Lev87, BHKN13, BHKN19, DS15], and then obtain linear size PRFs [IKOS08, FLY22], while preserving logarithmic depth. We note that these transformations will not preserve key homomorphism.

Lemma 1 (Domain Extension for NC^1 PRFs [DS15]). *Let λ be a security parameter. Let q, t, ϵ, d all be functions of λ . Let $\mathcal{X} = \{0, 1\}^{\omega(\log \lambda)}$ be an input domain. If there exists an (q, t, ϵ) -pseudorandom function $F: \{0, 1\}^{k(\lambda)} \times \mathcal{X} \rightarrow \mathcal{Y}$ computable in depth $O(\log \lambda)$, then there exists an NC^1 $(\min(q, |\mathcal{X}|/2), t - \text{poly}(\lambda), \epsilon + q \cdot 2^{-\Omega(\lambda)})$ -pseudorandom function $F': \mathcal{K}' \times \{0, 1\}^\lambda \rightarrow \mathcal{Y}$.*

Lemma 2 (Size Reduction for NC^1 PRFs [FLY22]). *Let λ be a security parameter. Let q, t, ϵ, d all be functions of λ . If there exists an NC^1 (q, t, ϵ) -pseudorandom function $F: \mathcal{K} \times \{0, 1\}^\lambda \rightarrow \mathcal{Y}$, then there exists an NC^1 $(q, t - \text{poly}(\lambda), \epsilon + q^2 \cdot 2^{-\Omega(\lambda^c)})$ -pseudorandom function $F': \mathcal{K}' \times \{0, 1\}^\lambda \rightarrow \mathcal{Y}$ of size $2\lambda + o(\lambda)$, and depth $1.01 \log \lambda$, where $c \in (0, 1)$ is some constant.*

Key-homomorphic PRFs are a special family of PRFs that satisfy an additional algebraic property. Specifically, for a key-homomorphic PRF, the key space $\mathcal{K}$ and the range $\mathcal{Y}$ of the PRF exhibit certain group structures such that its evaluation on any fixed input $x \in \mathcal{X}$ is homomorphic with respect to these group structures. Key-homomorphic PRFs have many useful applications in symmetric cryptography and give rise to distributed PRFs, symmetric-key proxy re-encryption, and updatable encryption [BLMR13, EPRS17, LT18, KLR19, BDGJ20, BEKS20], OT extension [Sch18], and secure aggregation [LLPT23]. To date, KH-PRFs (Definition 4) are only known to exist in the random oracle model. In the plain model, we only know how to achieve a relaxation of key-homomorphism called almost-key-homomorphism [BLMR13, DJK25].

Definition 4 (Approximate Key-Homomorphic PRFs [NPR99, BLMR13, DJK25]). Let $n \in \mathbb{N}$. Let $\gamma \in (0, 1)$. Let $(\mathbb{G}, +)$ be a group. We say $T: \mathbb{G} \rightarrow \mathbb{F}^n$ is a γ -linear operator if for any $x_1, x_2 \in \mathbb{G}$, and any $\lambda_1, \lambda_2 \in \mathbb{Z}$, it holds that

$$T(\lambda_1 \cdot x_1 + \lambda_2 \cdot x_2) = \lambda_1 \cdot T(x_1) + \lambda_2 \cdot T(x_2) + \epsilon,$$

where $\epsilon \in \mathbb{F}^n$ is a sparse vector such that $\text{wt}(\epsilon) \leq \gamma n$. Let $F: \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a PRF (Definition 3). We say F is a γ -KH if for every $x \in \mathcal{X}$, $T_x := F(\cdot, x): \mathcal{K} \rightarrow \mathcal{Y}$ is a γ -linear operator.

We note that the KH-PRF requires that the linear property holds for every key in the key space. We now introduce KH-structure, which only requires a related property holds with high probability over random keys.

Definition 5 (KH-structure [DJK25]). Let $\gamma \in (0, 1/2)$. Let $(\mathcal{K}, \oplus)$ be a group. Let $F: \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{F}$ be a PRF. We say F has γ -KH structure if there exists a tuple (f, h, z) such that for any $k \in \mathcal{K}, x \in \mathcal{X}$, F can be represented as

$$F(k, x) = \langle f(x), h(k) \rangle + z(k, x), \quad (5)$$

where

- $f: \mathcal{X} \rightarrow \mathbb{F}_2^m$ can be any function.
- $h: \mathcal{K} \rightarrow \mathbb{F}_2^m$ is a group homomorphism.
- $z: \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{F}_2$ is a function satisfying:

1. **(Efficiency)** For any $x \in \mathcal{X}, k \in \mathcal{K}$, $z(k, x)$ can be computed as efficiently as $F(k, x)$.
2. **(Vanishing)** For any $x \in \mathcal{X}$, $\Pr_{k \leftarrow \mathcal{K}}[z(k, x) = 1] \leq \gamma$.

PRFs with KH property can be transformed into KH-PRFs.

Lemma 3 (KH structure implies KH-PRFs [DJK25]). *Let $n, m \in \mathbb{N}$. If there exists a PRF $F: \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{F}_2$ with γ -AKH structure, then there exists a 6γ -AKH-PRF $F': \mathcal{K}^n \times \mathcal{X} \rightarrow \mathbb{F}_2^n$ such that for any PRF adversary $\mathcal{A}$, it holds that*

$$\text{Adv}_{F'}(\mathcal{A}, n, m) \leq m \cdot \text{Adv}_F(\mathcal{A}) + \exp\left(\log |\mathcal{X}| - \frac{m\gamma}{3}\right).$$

Moreover, if F can be computed in depth d , then F' can be computed in depth $2d + O(\log m)$.

3.2 Code-Based Assumptions

3.2.1 The LPN Assumption

In this section, we recall the LPN assumption.

Definition 6 (LPN [BFKL93]). Let $n, m, t \in \mathbb{N}$, $\mu \in (0, 1/2)$, $\epsilon \in (0, 1)$. The $[n, m, \mu, t, \epsilon]_2$ -LPN assumption states that

$$(\mathbf{A}, \mathbf{s} \cdot \mathbf{A} + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{A}, \mathbf{u}),$$

where $\mathbf{s} \leftarrow \$ \mathbb{F}_2^n$, $\mathbf{A} \leftarrow \$ \mathbb{F}_2^{n \times m}$, $\mathbf{e} \leftarrow \$ \text{Ber}_\mu^m$, and $\mathbf{u} \leftarrow \$ \mathbb{F}_2^m$.

Definition 7 (Quasi-polynomial hardness of LPN). Let $n \in \mathbb{N}$. Let $m = m(n), \mu = \mu(n), t = t(n), \epsilon = \epsilon(n)$ all be functions of n . We say LPN is quasi-polynomially hard if for any noise rate

$$\mu(n) = \Omega(n^{-1/\log \log n}),^7$$

there exists constant $c_1, c_2, c_3 > 0$, such that for

$$m(n) = 2^{c_1 \log^{1.1} n}, \quad t(n) = 2^{c_2 \log^{1.1} n}, \quad \epsilon(n) = 2^{-c_3 \log^{1.1} n},$$

the $[n, m, \mu, t, \epsilon]_2$ -LPN assumption (Definition 6) holds for all sufficiently large n .

Under a constant noise rate (i.e., $\mu = \Theta(1)$), the best known LPN solvers [BKW03, LF06] require time and query complexity both $2^{\Omega(n/\log n)}$. The time complexity goes up to $2^{\Omega(n/\log \log n)}$ when $m = \text{poly}(n)$ [Lyu05], or even $2^{\Omega(n)}$ given only $m = O(n)$ samples [MMT11]. Under our slightly sub-polynomial noise rate $\mu = n^{-1/\log \log n}$, for super-polynomial m , the best known attacks [Ste89, CC98, BLP11, Kir11, BJMM12] are not asymptotically better, i.e., still at the order of $2^{\tilde{\Omega}(n)}$. The quasi-polynomial (or subexponential) LPN was used in several previous works [YZ16, YZW⁺19, JLS21, CM21, JLS22, RVV24].

3.2.2 The RingLPN Assumption

In this section, we recall the RingLPN assumption, which was first introduced (over $\mathbb{F}_2$) in [HKL⁺12]. Since then, it has received growing attention from the cryptography community [BL12, DP12, LP15,

⁷We note that this noise level is only slightly sub-constant, and larger than any inverse polynomial

[GJL15](#), [BCG⁺20](#), [LWYY24](#)], due to its appealing efficiency advantages over LPN. We will be mainly interested in RingLPN over cyclotomic rings, introduced in [\[BCG⁺20\]](#), which have previously been used for Ring-LWE [\[LPR10, LPR13\]](#). For discussions regarding existing attacks on RingLPN, we refer the reader to [\[BCG⁺20\]](#).

Definition 8 (RingLPN [\[HKL⁺12, BCG⁺20\]](#)). Let $n, m, t \in \mathbb{N}$, n be a power of two, $\mu \in (0, 1/2)$, $\epsilon \in (0, 1)$. Let $\mathcal{R}$ be cyclotomic polynomial ring $\mathcal{R} = \mathbb{Z}[X]/(X^n + 1)$. Let q be a prime such that $q = 1 \pmod{2n}$. Define the quotient ring $R_q = R/qR$. Let χ be the noise distribution over $\mathcal{R}_q$ whose coefficient is distributed as $\text{Ber}_\mu(\mathbb{F}_q)$. The $[n, m, \mu, t, \epsilon]$ -RingLPN assumption states that

$$(\mathbf{a}, s \cdot \mathbf{a} + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{a}, \mathbf{u}),$$

where $s \leftarrow \mathcal{R}_q$, $\mathbf{a} \leftarrow \mathcal{R}_q^m$, $\mathbf{e} \leftarrow \chi^m$, and $\mathbf{u} \leftarrow \mathcal{R}_q^m$. Or equivalently,

$$(\mathbf{A}, \mathbf{s} \cdot \mathbf{A} + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{A}, \mathbf{u}),$$

where $\mathbf{A} \leftarrow \text{Mat}(a_1) \parallel \dots \parallel \text{Mat}(a_m) \in \mathbb{F}_q^{n \times nm}$ (Definition 9) for $a_i \leftarrow \mathcal{R}_q$, $\mathbf{s} \leftarrow \mathbb{F}_q^n$, $\mathbf{e} \leftarrow \mathbb{F}_q^{nm}$, $\mathbf{u} \leftarrow \mathbb{F}_q^{nm}$,

Definition 9 (Vec and Mat). Let all parameters be as in Definition 8. Write $a \in \mathcal{R}_q$ in the monomial basis as $a = \sum_{i=0}^{n-1} a_i X^i$, $a_i \in \mathbb{F}_q$. Define the coefficient vector embedding

$$\text{Vec}: \mathcal{R}_q \rightarrow \mathbb{F}_q^n, \quad \text{Vec}(a) := (a_0, \dots, a_{n-1}).$$

For $b \in \mathcal{R}_q$, define the coefficient matrix embedding

$$\text{Mat}: \mathcal{R}_q \rightarrow \mathbb{F}_q^{n \times n}, \quad (\text{Mat}(b))_{i,*} := \text{Vec}(X^{i-1} \cdot b), \text{ for } i \in [n].$$

Then for all $a, b \in \mathcal{R}_q$ it holds that

$$\text{Vec}(a \cdot b) = \text{Vec}(a) \cdot \text{Mat}(b).$$

An equivalent view of RingLPN is that it postulates $(\mathbf{A}, \mathbf{s} \cdot \mathbf{A} + \mathbf{e})$ is pseudorandom, where $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_m)$ with each block $\mathbf{A}_i$ generated as the following procedure: its first row is generated uniformly at random in $\mathbb{F}_q^n$, then the remaining rows of $\mathbf{A}_i$ are obtained by a cyclic shift of the previous row with a sign flip on the wrapped-around entry (i.e., a negacyclic shift). Since each block of $\mathbf{A}$ is negacyclic, RingLPN can be viewed as the problem of decoding random quasi-negacyclic codes.

We now introduce RingLPN ^{$s \sim \chi$} , where the secret is also chosen from the noise distribution.

Definition 10 (RingLPN ^{$s \sim \chi$}). Let all parameters be as in Definition 8. The $[n, m, \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN ^{$s \sim \chi$} assumption states that

$$(\mathbf{a}, \mathbf{a} \cdot s + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{a}, \mathbf{u}), \tag{6}$$

where $s \leftarrow \chi$, $\mathbf{a} \leftarrow \mathcal{R}_q^m$, $\mathbf{e} \leftarrow \chi^m$, and $\mathbf{u} \leftarrow \mathcal{R}_q^m$.

We now formally define the polynomial hardness of RingLPN (or RingLPN ^{$s \sim \chi$}), which says that for noise rate of $\mu = \Omega(\frac{\log^c n}{n})$, RingLPN (or RingLPN ^{$s \sim \chi$}) holds against any polynomial time distinguisher.

Definition 11 (Polynomial Hardness of RingLPN and RingLPN^{s~χ}). Let $m = m(n), \mu = \mu(n), q = q(n), t = t(n), \epsilon = \epsilon(n)$ all be functions of $n \in \mathbb{N}$. Let c be an arbitrarily large constant. We say (low-noise) RingLPN (or RingLPN^{s~χ}) is polynomially hard (or hard, for short) if for any noise rate

$$\mu(n) = \Omega\left(\frac{\log^c n}{n}\right),$$

and any

$$m(n) = \text{poly}(n), \quad q(n) = \text{poly}(n), \quad t(n) = \text{poly}(n),$$

there exists a negligible $\epsilon(n) = \text{negl}(n)$, such that the $[n, m, \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN (or $[n, m, \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN^{s~χ}) assumption holds for all sufficiently large n .

Different to the previous noise rate $\mu = \Omega(n^{-1/\log \log n})$ used in LPN, for RingLPN we will use an extremely low-noise rate $\mu = \Omega(\frac{\log^c n}{n})$, but only needs to assume the resulting RingLPN is polynomially hard. In this noise rate regime, the assumption is still believed to hold for any polynomial $m = \text{poly}(n)$. However, it can only achieve security against adversaries that run in quasi-polynomial time $2^{o(\log^2 n)}$ due to Gaussian elimination [EKM17]. This low noise rate was used in several previous works [BLVW19, AMR25].

We will rely on a reduction from RingLPN to RingLPN^{s~χ}.

Lemma 4. *If RingLPN is polynomially hard, then RingLPN^{s~χ} is also polynomially hard.*

Proof. See Appendix B.1. □

3.2.3 The SparseLPN Assumption

In this section, we first recall expanders, then recall the SparseLPN assumption [Ale03, Ale11], also known as the “Alekhovich’s assumption.”

Unbalanced expanders. We introduce the notion of *unbalanced (bipartite) expanders*. From now on, we will simply refer to them as expanders.

Definition 12 (Unbalanced Expander). Let $n, m, d, m' \in \mathbb{N}$, $d < n < m$, $m' < m$. Define a neighbor function

$$\Gamma: [m] \times [d] \rightarrow [n],$$

which induces a right-regular bipartite graph of right-degree d . For a vertex $u \in [m]$, let

$$\Gamma(u) := \{\Gamma(u, i) : i \in [d]\} \subseteq [n],$$

and for a set $S \subseteq [m]$ define $\Gamma(S) := \bigcup_{u \in S} \Gamma(u)$.

We say that Γ is a (n, m, d, m') -expander if for every $S \subseteq [m]$ with $|S| \leq m'$,

$$|\Gamma(S)| > \frac{1}{2} \cdot d |S|. \text{⁸}$$

⁸This is an expansion regime where a certain property called “unique neighbors” is guaranteed, which will then guarantee that the minimal distance of the expander codes induced by Γ will be $m' + 1$.

The bipartite expander graph induced by Γ is $G_\Gamma = (L \cup R, E)$ with

$$E = \{(u, \Gamma(u, i)) : u \in [m], i \in [d]\}.$$

The expander matrix $A_\Gamma \in \{0, 1\}^{n \times m}$ induced by Γ is defined by

$$(\mathbf{A}_\Gamma)_{v,u} = 1 \iff v \in \Gamma(u).$$

We will use $\Gamma_\S$ to denote random expanders, i.e., a uniformly random map $[L] \times [d] \rightarrow [R]$. Otherwise, Γ is assumed to be an explicit expander.

A key observation we will rely on is that many (if not all) existing constructions of explicit expanders can be computed by sublinear-depth circuits. This is in contrast to random expanders. Recall that random expanders requires linear depth to evaluate (“linear” is measured with respect to its input length of $\log m + \log d$).

Lemma 5 (Unbalanced Expanders in Sublinear Depth [TUZ01, TUZ07, GUV09, KT22]). *Let $m = m(n), d = d(n), m' = m'(n)$ for $n \in \mathbb{N}$. There exists constant $c_1 > 1, c_2 \in (0, 1)$, such that for all sufficiently large n , for*

$$m = n^{1.01}, \quad d = \log^{c_1} n, \quad m' = n^{c_2},$$

there exists (n, m, d, m') -expander $\Gamma : [m] \times [d] \rightarrow [n]$ that is computable by circuits of sublinear depth

$$o(\log m + \log d) = o(\log n).$$

We now define the **SparseLPN** assumption, which was introduced in [Ale03, Ale11], although its lineage traces back much earlier to expander codes [SS94, SS96]. Recall that expander codes are defined by taking the expander matrix as a *parity-check* matrix of a linear code, whereas in **SparseLPN** the expander matrix is used as a *generator* matrix for a linear code. Thus, **SparseLPN** can be viewed as the noisy decoding problem for the duals of expander codes. Since its introduction in 2003, **SparseLPN** has been studied and used for over a couple of decades in cryptography, and computational learning theory [AIK08, IKOS08, ABW10, ADI⁺17, JLS21, JLS22, DIJL23, JLS24, RVV24, CHKV25, BBTV25, DJK25, GHJS25, CSZ25].

The most common instantiation of **SparseLPN** uses a random expander (i.e., the matrix $\mathbf{A}$ is a random column-sparse matrix). We will adopt a more careful and structured instantiation. Namely, $\mathbf{A}$ will be the adjacency matrix of an *explicit* expander graph whose neighbor function is computable in sublinear depth (see Lemma 5).

Definition 13 (**SparseLPN** [Ale03, Ale11] (with respect to a sublinear-depth explicit expander)). Let $n, m, d, t \in \mathbb{N}, d < n, \mu \in (0, 1/2), \epsilon \in (0, 1)$. Let $\Gamma : [m] \times [d] \rightarrow [n]$ be the (n, m, d, m') -expander in Lemma 5. The $[n, m, \mu, t, \epsilon]_2$ -**SparseLPN** assumption with respect to Γ states that

$$(\mathbf{A}_\Gamma, \mathbf{s} \cdot \mathbf{A}_\Gamma + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{A}_\Gamma, \mathbf{u}), \tag{7}$$

where $\mathbf{s} \leftarrow \mathbb{F}_2^n$, $\mathbf{A}_\Gamma \in \mathbb{F}_2^{n \times m}$ is the expander matrix induced by Γ , $\mathbf{e} \leftarrow \text{Ber}_\mu^m$, and $\mathbf{u} \leftarrow \mathbb{F}_2^m$.

We now formally define the polynomial hardness of **SparseLPN**, which says that for noise rate $\mu = \Omega(n^{-1/\log \log n})$, and degree of the expander $d = \text{poly}(\log n)$, **SparseLPN** holds against any polynomial time distinguisher.

Definition 14 (Polynomial hardness of SparseLPN). Let Γ be in Lemma 5. Let $m = m(n), \mu = \mu(n), d = d(n), t = t(n), \epsilon = \epsilon(n)$ all be functions of $n \in \mathbb{N}$. We say SparseLPN is polynomially hard if for any noise rate

$$\mu(n) = \Omega(n^{-1/\log \log n}),^9$$

and any

$$m(n) = \text{poly}(n), \quad d(n) = \text{poly}(\log n), \quad t(n) = \text{poly}(n),$$

there exists a negligible $\epsilon(n) = \text{negl}(n)$, such that $[n, m, \mu, t, \epsilon]_2$ -SparseLPN assumption (with respect to Γ) holds for all sufficiently large n .

Remark 8. The original SparseLPN assumption [Ale03] posits that it holds with respect any good expander. We note, however, that this formulation becomes false in the quasi-polynomial sample regime (i.e., $m = n^{\omega(1)}$). In particular, it is violated when the expander is computable by subexponential-size $\text{AC}^0[\oplus]$ circuits [OST19, OST22]. The reason is that the resulting noisy function can then be approximated by a subexponential-size $\text{AC}^0[\oplus]$ circuit, which is PAC-learnable [RR97, CIKK16]. Nevertheless, to date there is still no known counterexample to SparseLPN in the standard polynomial-sample regime. Our results require only that the assumption hold for a slightly super-linear number of samples (i.e., $m = n^{1+\epsilon}$) and for expanders of slightly sublinear depth.

We conclude this section with a fact that SparseLPN unconditionally holds against a family of (potentially unbounded) distinguishers called “linear test,” which capture almost all existing attacks against (variants of) LPN assumptions. This follows because the dual distance of the underlying codes, namely, the duals of the expander codes induced by Γ , equals the minimum distance of the expander codes themselves, which is $m' = n^c$ for some constant c . Then the distinguishing advantage of linear test is $2^{-\Omega(n^{c'})}$ for some constant c' .

3.3 Bernoulli Sample Function S-Ber over $\mathbb{F}_2$

In this section, we introduce a central gadget in our constructions, the Bernoulli sample function S-Ber over $\mathbb{F}_2$.

Definition 15 (Bernoulli Sample Function over $\mathbb{F}_2$). Let $\ell \in \mathbb{N}$, $\mu = 2^{-\ell}$. On input $\mathbf{x} \in \mathbb{F}_2^\ell$, define $\text{S-Ber}_\mu: \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2$ as,

$$\text{S-Ber}(\mathbf{x}; \mu) := \prod_{i=1}^{\ell=\log(1/\mu)} x_i. \quad (8)$$

We abuse notation and extend S-Ber_μ to handle input $\mathbf{x} \in \mathbb{F}_2^{n\ell}$ by applying S-Ber_μ separately on each ℓ -bit block $\mathbf{x}$, yielding an output in $\mathbb{F}_2^n$. More formally, on input $\mathbf{x} \in \mathbb{F}_2^{n\ell}$, define

$$\text{S-Ber}(\mathbf{x}; \mu) := (\text{S-Ber}_\mu(\mathbf{x}_{1\dots\ell}), \dots, \text{S-Ber}_\mu(\mathbf{x}_{(n-1)\ell+1\dots n\ell})) \in \mathbb{F}_2^n.$$

We abuse notation and extend S-Ber_μ to handle inputs $\mathbf{X} \in \mathbb{F}_2^{n\ell \times m}$ by applying S-Ber_μ separately on each column of $\mathbf{X}$, yielding an output in $\mathbb{F}_2^{n \times m}$. More formally, on input $\mathbf{X} \in \mathbb{F}_2^{n\ell \times m}$, define

$$\text{S-Ber}(\mathbf{X}; \mu) := \text{S-Ber}_\mu(\mathbf{X}[:, 1]) \parallel \dots \parallel \text{S-Ber}_\mu(\mathbf{X}[:, m]) \in \mathbb{F}_2^{n \times m}.$$

⁹We note that this quantity is only slightly sub-constant, and larger than any inverse polynomial

Lemma 6 (Low-Depth of S-Ber_μ). *Let $\ell \in \mathbb{N}$, $\mu = 2^{-\ell}$. S-Ber_μ can be computed by a circuit of depth $O(\log \log(1/\mu))$.*

Proof. S-Ber_μ operates each column independently, and for each column, it simply AND all ℓ bits of each chunk. Therefore, the depth is $O(\log \ell) = O(\log \log(1/\mu))$. $\square$

If the input to S-Ber is uniform, then the output will be distributed as a Bernoulli random variable, as indicated by the name of S-Ber .

Claim 1 (Bernoulli Output under a Uniform Input). *It holds that $\text{S-Ber}_\mu(\mathbf{U}) \sim \text{Ber}_\mu^{n \times m}$ for $\mathbf{U} \leftarrow_{\$} \mathbb{F}_2^{n \times m}$.*

We now introduce a key property of S-Ber , called *high collision*, that plays a central role in our security reduction.

Lemma 7 (High Collision of S-Ber). *Let $n, m, \ell \in \mathbb{N}$, $\mu = 2^{-\ell}$. Let $\mathbf{P}$ be a uniformly random variable over $\mathbb{F}_2^{n \times m}$. Let $\mathbf{M}, \mathbf{M}'$ be (arbitrary) random variables over $\mathbb{F}_2^{n \times m}$, each independent of $\mathbf{P}$. It holds that*

$$\Pr_{\mathbf{P}, \mathbf{M}, \mathbf{M}'}[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \text{S-Ber}(\mathbf{P} + \mathbf{M}')] \leq 2\mu nm. \quad (9)$$

Proof. Because $\mathbf{P}$ is uniform and $\mathbf{M}$ is independent of $\mathbf{P}$, the sum $\mathbf{P} + \mathbf{M}$ is uniform over $\mathbb{F}_2^{n \times m}$. By Claim 1, $\text{S-Ber}(\mathbf{X} + \mathbf{M}) \sim \text{Ber}_\mu^{n \times m}$, hence

$$\Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) = \mathbf{0}] = (1 - \mu)^{nm} \geq 1 - nm\mu,$$

and so

$$\Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \mathbf{0}] \leq nm\mu.$$

The same bound holds with $\mathbf{M}'$ in place of $\mathbf{M}$. By a union bound,

$$\Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \mathbf{0} \vee \text{S-Ber}(\mathbf{P} + \mathbf{M}') \neq \mathbf{0}] \leq 2nm\mu.$$

Finally, we conclude that

$$\begin{aligned} \Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \text{S-Ber}(\mathbf{P} + \mathbf{M}')] &= 1 - \Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) = \text{S-Ber}(\mathbf{P} + \mathbf{M}')] \\ &\leq 1 - \Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) = \text{S-Ber}(\mathbf{P} + \mathbf{M}') = \mathbf{0}] \\ &= \Pr[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \mathbf{0} \vee \text{S-Ber}(\mathbf{P} + \mathbf{M}') \neq \mathbf{0}] \\ &\leq 2nm\mu. \end{aligned}$$

$\square$

3.4 Additional Notation for Matrix Operations

In this section, we introduce some matrix operations that will be used.

Lists. We use an n -dimensional vector $\mathbf{v}$ to denote a list of n elements.

Iterators. An iterator `Iter` is a data structure supporting two operations, `Init` and `Next`. Given a matrix $\mathbf{A} \in \mathbb{F}^{T \times n}$, `Iter.Init(A)` initializes the iterator on $\mathbf{A}$ (with an internal cursor at the first row), and each call to `Iter.Next()` returns the next unused row of $\mathbf{A}$ (in increasing row index). Hence, once initialized, `Iter` can be invoked at most T times before it is exhausted.

Column selection. We define the column selection operator `ColSelect`, which outputs a matrix formed by selecting columns of an input matrix, according to the column indices given by a list.

Definition 16 (`ColSelect`). Let $n, m, m' \in \mathbb{N}$. On input a matrix $\mathbf{A} \in \mathbb{F}^{n \times m}$ and a list $\mathbf{c} \in (0 \cup [m])^{m'}$, define

$$\text{ColSelect}(\mathbf{A}, \mathbf{c}) := \mathbf{A}[:, \mathbf{c}_1] \parallel \cdots \parallel \mathbf{A}[:, \mathbf{c}_{m'}] \in \mathbb{F}^{n \times m'}.$$

Recall that $\mathbf{A}[:, 0]$ is defined to be a zero vector in $\mathbb{F}^n$.

Row selection. Similarly, we define row selection operator `RowSelect`, which outputs a matrix formed by selecting rows of an input matrix, according to the row indices given by a list.

Definition 17 (`RowSelect`). Let $n, n', m \in \mathbb{N}$. On input a matrix $\mathbf{A} \in \mathbb{F}^{n \times m}$, and a list $\mathbf{r} \in (0 \cup [n])^{n'}$, define

$$\text{RowSelect}(\mathbf{A}, \mathbf{r}) := \mathbf{A}[\mathbf{r}_1, :] \parallel \cdots \parallel \mathbf{A}[\mathbf{r}_{n'}, :] \in \mathbb{F}^{n' \times m}$$

Recall that $\mathbf{A}[0, :]$ is defined to be a zero vector in $\mathbb{F}^m$.

Support of a vector with padding. The operator `Supp` returns the list of (strictly increasing) indices of the nonzero positions of a vector and then pads with zeros at the end to a fixed output length.

Definition 18 (`Supp`). Let $n, m \in \mathbb{N}$. For a padding parameter $w \in \mathbb{N}$, define $\text{Supp}_w : \mathbb{F}^n \rightarrow (0 \cup [n])^w$. For a vector $\mathbf{v} \in \mathbb{F}^n$ with $t \leq w$ number of nonzero entries, let $(i_1, \dots, i_t)$ be the strictly increasing list of indices of the nonzero entries of $\mathbf{v}$. We define

$$\text{Supp}(\mathbf{v}; w) := (i_1, \dots, i_t, \underbrace{0, \dots, 0}_{w-t \text{ entries}}) \in (0 \cup [n])^w,$$

When the padding parameter w is omitted, we define

$$\text{Supp}(\mathbf{v}) := (i_1, \dots, i_t) \in [n]^t,$$

i.e., no padding is applied.

Row-support with padding. The operator `RowSupp` returns the list of (strictly increasing) indices of the nonzero row indices of a matrix and then pads with zeros at the end to a fixed output length.

Definition 19 (`RowSupp`). Let $n, m \in \mathbb{N}$. For a padding parameter $w \in \mathbb{N}$, define $\text{RowSupp}_w : \mathbb{F}^{n \times m} \rightarrow (0 \cup [n])^w$. For a matrix $\mathbf{A} \in \mathbb{F}^{n \times m}$ with $t < w$ number of nonzero rows, let $(i_1, \dots, i_t)$ be the strictly increasing list of indices of the nonzero rows of $\mathbf{A}$. We define

$$\text{RowSupp}(\mathbf{A}; w) := (i_1, \dots, i_t, \underbrace{0, \dots, 0}_{w-t \text{ entries}}) \in (0 \cup [n])^w,$$

When the padding parameter w is omitted, we define

$$\text{RowSupp}(\mathbf{A}) := (i_1, \dots, i_t) \in [n]^t,$$

i.e., no padding is applied.

Row pruning with padding. We then define row-pruning operator RowPrune , which first removes all zero rows of the input matrix, then pads additional zero rows at the end to output a matrix with a fixed number of rows.

Definition 20 (RowPrune). Let $n, m, w \in \mathbb{N}$. For a padding parameter $w \in \mathbb{N}$, define $\text{RowPrune}_w: \mathbb{F}^{n \times m} \rightarrow \mathbb{F}^{w \times m}$ as follows. On input a matrix $\mathbf{A} \in \mathbb{F}^{n \times m}$ with $t < w$ number of nonzero rows, define

$$\text{RowPrune}(\mathbf{A}; w) := \text{RowSelect}(\mathbf{A}, \text{RowSupp}(\mathbf{A}; w))$$

Lemma 8. *Let all parameters be as in Definition 16-20. Let $\mathbb{F} = \mathbb{F}_q$. Every operation in Definition 16-20 can be done in $O(\log(n) + \log \log q)$ depth.*

Proof. See Appendix B.2. □

4 PRFs from Quasi-Polynomial Hardness of LPN

In this section, we provide our PRF construction from the quasi-polynomial hardness of classical LPN.

4.1 Reduction to LPN with Regular Sparse Coefficients

We introduce a variant of LPN, called rsLPN , where each column of $\mathbf{A}$ is a regular d -sparse vector, i.e., concatenation of d 1-sparse vectors in $\mathbb{F}_2^{n/d}$.

Definition 21 (rsLPN). Let $n, m, d, t \in \mathbb{N}, d < n, \mu \in (0, 1/2), \epsilon \in (0, 1)$. The $[n, m, \mu, d, t, \epsilon]_2\text{-rsLPN}$ assumption states that

$$(\mathbf{A}, \mathbf{s} \cdot \mathbf{A} + \mathbf{e}) \approx_{t, \epsilon} (\mathbf{A}, \mathbf{u}),$$

where $\mathbf{s} \leftarrow \$ \mathbb{F}_2^n$, $\mathbf{A} \leftarrow \$ \mathbb{F}_2^{n \times m}$ such that each column of $\mathbf{A}$ is a random regular d -sparse vector, $\mathbf{e} \leftarrow \$ \text{Ber}_\mu^m$, and $\mathbf{u} \leftarrow \$ \mathbb{F}_2^m$.

Lemma 9 (**Reduction from LPN to rsLPN** [BBTV25]). *Let $n, m, d, t \in \mathbb{N}, d < n, \mu \in (0, 1/2), \epsilon \in (0, 1)$. $[n, m, \mu, d, t, \epsilon]_2\text{-rsLPN}$ holds if $[d \log(n/d), m, \mu, t + 5mnd \log(n/d), \epsilon]_2\text{-LPN}$ holds.*

We next show how to choose a sequence of parameters that reduces the quasi-polynomial hardness of a sequence of rsLPN instances to the quasi-polynomial hardness of LPN.

Definition 22 (Quasi-polynomial hardness of (a sequence of) rsLPN). Let $n \in \mathbb{N}$. Let $m = m(n), \mu = \mu(n), d = d(n), t = t(n), \epsilon = \epsilon(n)$ all be functions of $n \in \mathbb{N}$. We say that rsLPN is

quasi-polynomially hard if for a sequence of integers $i = 1, \dots, \log \log \log n$, for any

$$\mu_i(n) = \Theta(n^{-1/(\log \log n)^{2i+3}}), \quad d_i(n) = \Theta(n^{1/(\log \log n)^{2i+1}}),^{10}$$

there exists constant $c_1, c_2, c_3 > 0$, such that for

$$m_i(n) = 2^{c_1 \log^{1.01} n}, \quad t_i(n) = 2^{c_2 \log^{1.01} n}, \quad \epsilon_i(n) = 2^{-c_3 \log^{1.01} n},$$

the $[n, m_i, \mu_i, d_i, t_i, \epsilon_i]_{2\text{-rsLPN}}$ assumption holds for all sufficiently large n .

The following lemma follows by simply plugging in the said parameters in Lemma 9.

Lemma 10. *If LPN is quasi-polynomially hard (Definition 7), then rsLPN is also quasi-polynomially hard (Definition 22).*

Proof. See Appendix B.3. □

4.2 Functionality

In this section, we specify the input–output behavior of our PRF family. While the definition itself is constructive, evaluating it verbatim does not yield low circuit depth. The next section presents our construction, which gives a depth-efficient evaluation of this functionality.

Definition 23 (Functionality of PRFs). We define a keyed function

$$\text{PRF}_{\mathbf{k}}: [\lambda]^k \rightarrow \mathbb{F}_2^n.$$

Parameters.

1. Security parameter $\lambda \in \mathbb{N}$.
2. rsLPN parameter:
 - (a) Dimension n , a sequence of noise rates $(\mu_1, \dots, \mu_k)$, and a sequence of sparsity rates $(d_1, \dots, d_k)$ as defined in Definition 22, chosen so that for $i \in [k]$, $\mu_i = 2^{-\ell_i}$ for $\ell_i \in \mathbb{N}$.
 - (b) $\mathbf{A} = (\mathbf{A}_{1,1}, \dots, \mathbf{A}_{k,\lambda})$, where $\mathbf{A}_{i,x} \in \mathbb{F}_2^{n \times n}$ for $i \in [k]$, $x \in [\lambda]$ is a random regular d_i -sparse matrix.
3. Number of input chunks $k \in \mathbb{N}$. Each chunk has length $\log \lambda$.
4. Maximum recursion depth $\tau \in \mathbb{N}$.

¹⁰It follows that $\mu_i \cdot d_i \gg 1$.

Recursive Definition of $\mathbf{F}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{S}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad (\mathbf{A}_{1,1}, \dots, \mathbf{A}_{k,\lambda}), \text{ where } \mathbf{A}_{i,x} \in \mathbb{F}_2^{n \times n} \text{ for } i, x \in [k] \times [\lambda].$$

For every $k' = 1, \dots, k$, define $\mathbf{F}_{(k')}$ as follows.

Function $\mathbf{F}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_2^n$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\mathbf{S}_{(k')}, \mathbf{P}_{(k')})$.
3. $(\mathbf{s}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_2^n \times \mathbb{F}_2^n$.
4. For each input prefix $i \in [k']$:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_{\mu_i}(\mathbf{R}_i) \in \mathbb{F}_2^n, \quad \text{where } \mathbf{R}_i \leftarrow \mathbf{F}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times n}.$$

5. Compute:

$$\begin{aligned} \mathbf{y} &\leftarrow (\dots (\mathbf{s} \cdot \mathbf{A}_{1,x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{2,x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{A}_{k',x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{s} \prod_{i=1}^{k'} \mathbf{A}_{i,x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{A}_{j,x_j} \in \mathbb{F}_2^n, \end{aligned}$$

6. Output $\mathbf{y} + \mathbf{p} \in \mathbb{F}_2^n$.

Figure 5: Functionality of PRFs from quasi-polynomial LPN, recursively defined.

Parameter Setting.

- Set $n = \lambda^{1/\log \log \lambda}$, $\tau = (\log \log \lambda)^{1/2}$, $k = \log \log \log \lambda$, and as in Definition 22, for $i \in [k]$, set

$$\mu_i = n^{-1/(\log \log n)^{2i+3}}, \quad d_i(n) = n^{1/(\log \log n)^{2i+1}}$$

Shorthand Notation.

- $\ell := \max(\ell_1, \dots, \ell_k) = \ell_k$ denotes the maximum shrinking ratio due to applying **S-Ber**.
- $T := (k \ell)^\tau$ denotes the (maximum) total number of recursive calls throughout the evaluation of PRF.

Function Key. $\mathbf{k} = (\mathbf{S}, \mathbf{P})$, where

- (LPN Secret Matrices) $\mathbf{S} = (\mathbf{S}_{(1)}, \dots, \mathbf{S}_{(k)})$, with $\mathbf{S}_{(k')} \in \mathbb{F}_2^{T \times n}$, for $k' \in [k]$.
- (Padding Matrices) $\mathbf{P} = (\mathbf{P}_{(1)}, \dots, \mathbf{P}_{(k)})$, with $\mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}$, for $k' \in [k]$.

Functionality. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.
2. Invoke the recursive function $\mathbf{F}$ (Figure 5)

$$\mathbf{y} \leftarrow \mathbf{F}_{(k)}(x, 0) \in \mathbb{F}_2^n.$$

3. Output $\mathbf{y} \in \mathbb{F}_2^n$.

4.3 Construction

In this section, we present a depth-efficient evaluation algorithm of the PRF functionality defined in Definition 23, which allows us to compute it in depth $O(\log \lambda)$. In addition, our security analysis (in particular, the resilience property) will crucially rely on this depth-efficient algorithm.

We begin by introducing a basic primitive from graph theory that outputs the reachability set of a walk starting from a given node¹¹. This functionality enables us to compute all terms of the form $\mathbf{e} \cdot \prod \mathbf{A}$ in $o(\log n)$ depth via projecting both $\mathbf{e} \in \mathbb{F}_2^n$ and $\mathbf{A} \in \mathbb{F}^{n \times n}$ into a dimension (denoted by w) equal to the size of the reachability set (which is only $o(n)$),

Reachability over a layered graph. Consider a $(k+1)$ -layer¹² directed graph with n ¹³ nodes per layer. Each node at layer i has an out-degree of d_i ¹⁴. We denote the max degree among them by d . For each $i = 0, \dots, k-1$, the edges from layer i to layer $i+1$ are specified by a neighbor function $\phi_i: [n] \rightarrow [n]^{d_i}$, such that

$$\text{there is an edge } (i, u) \rightsquigarrow (i+1, v) \quad \text{iff} \quad v \in \phi_{i+1}(u)$$

We define a function **Reach** that takes as input a starting node $u \in [n]$ and adjacency matrices $\mathbf{M}_1, \dots, \mathbf{M}_k$, where each $\mathbf{M}_i$ induces a neighbor function $\phi_i(j) := \text{Supp}(\text{ColSelect}(\mathbf{M}_i, j))$. The output is a list of reachability sets $\{\mathbf{c}^{(i)}\}_{i=0}^k$, where each $\mathbf{c}^{(i)} \in [n]^{w_i}$ (with $w_i = \prod_{j=1}^i d_j$ as an upper bound of the reachability set) contains all nodes at layer i that are reachable from u . Each $\mathbf{c}^{(i)}$ is sorted in increasing order and padded with zeros at the end; e.g., $\mathbf{c}^{(i)} = (1, 4, 5, 0, \dots, 0)$.

Reach is formally specified in Figure 6. Here we provide an informal overview. Procedure (2) can be viewed as a breadth-first search (BFS) starting at u for k steps, which can be implemented in depth $O(k \log n)$. After procedure (2), the list $\mathbf{c}^{(t)}$ collects all nodes reachable by walks of exactly t steps¹⁵; it is a multiset (unordered and possibly with duplicates). Procedure (3) then sorts, deduplicates, and pads $\mathbf{c}^{(t)}$ to length w using a bucket/bin-style routine. We exploit that all values lie in $[n]$, enabling parallel bucketing and compaction (via prefix sums), which yields depth

¹¹We note that this graph should not be confused with the recursion tree induced by Figure 5, but they are related to some extent.

¹² k corresponds to PRF input length

¹³ n corresponds to LPN dimension

¹⁴ d_i corresponds to column sparsity of $\mathbf{A}_{i,*}$

¹⁵Since it is a layered graph, all nodes reachable by t steps are at layer t

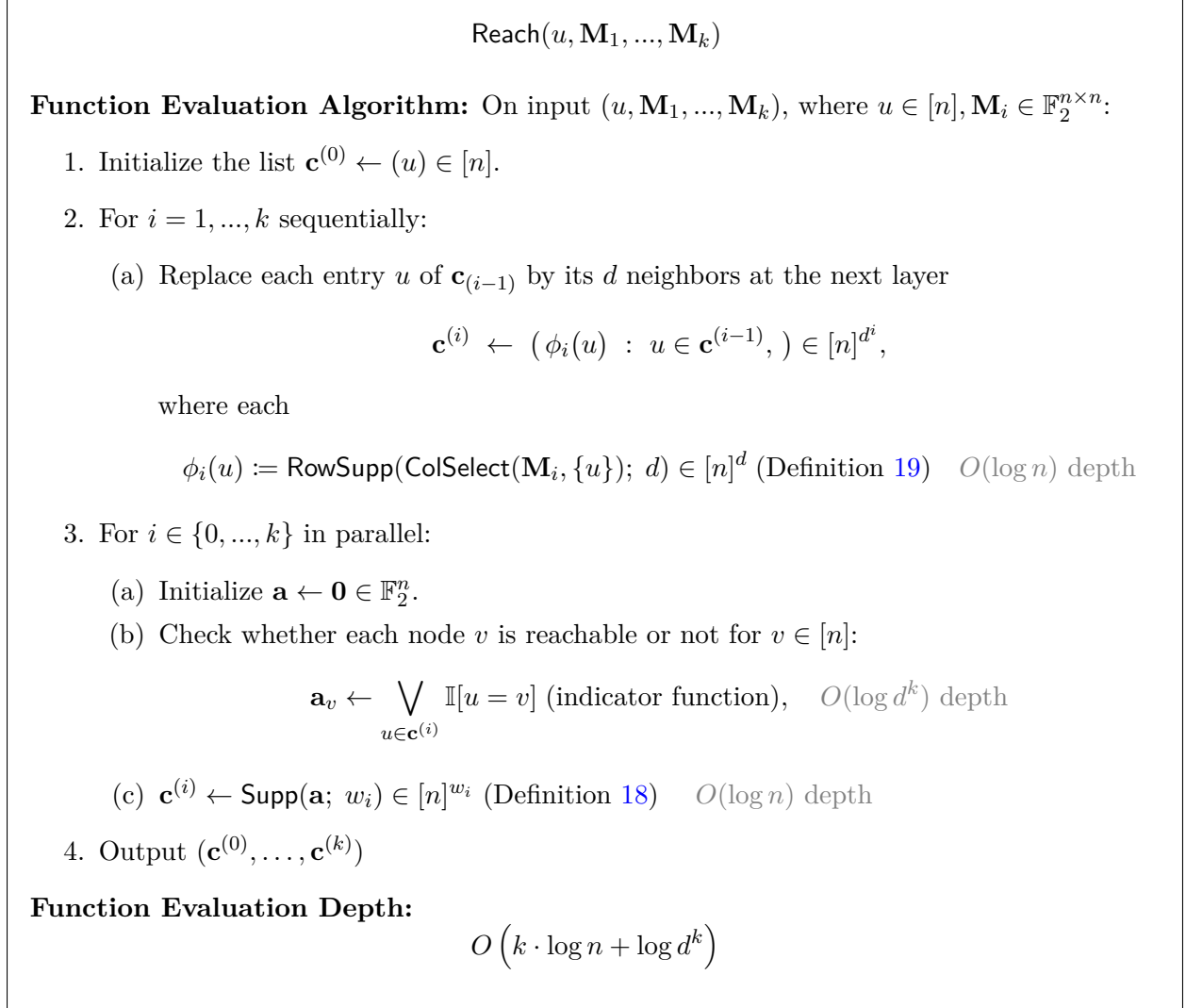


Figure 6: Reachability over $(k + 1)$ -Layered Graph

$O(\log n + \log w)$. Instantiating $w = d^k$, this becomes $O(\log n + \log d^k)$. Combining both phases, the overall circuit depth of **Reach** is

$$O(k \log n) + O(\log d^k + \log n) = O(k \log n + \log d^k).$$

We are now ready to present our PRF construction, which is a depth-efficient algorithm for evaluating the functionality defined in Definition 23. The evaluation is given recursively, mirroring Definition 23, but with two crucial differences: (i) we precompute all products $\prod \mathbf{A}$ in advance; and (ii) at each recursive call we compute $\bar{\mathbf{e}} \cdot \bar{\mathbf{A}}$ (rather than $\mathbf{e} \cdot \mathbf{A}$), leveraging the precomputed reachability sets so that the computation and output live in dimension w instead of n .

Construction 1 (PRFs from Quasi-Polynomial Hardness of LPN). Let all parameters as defined in Definition 23.

$$\text{PRF}_k: [\lambda]^k \rightarrow \mathbb{F}_2^n.$$

Recursive Definition of Core $\mathbf{G}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')} \in \mathbb{F}_2^{T \times w_0} \times \mathbb{F}_2^{T \times w_{k'}}, \text{ for } k' \in [k], \quad \overline{\mathbf{A}}_{(l,r)} \in \mathbb{F}_2^{w_{l-1} \times w_r}, \text{ for } l \in [k], r \in [l, k].$$

For every $k' = 1, \dots, k$, define $\mathbf{G}_{(k')}$ as follows.

Function $\mathbf{G}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_2^{w_{k'}}$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}.Init(\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')})$.
3. $(\overline{\mathbf{s}}, \overline{\mathbf{p}}) \leftarrow \text{Iter}_{(k')}.Next() \in \mathbb{F}_2^{w_0} \times \mathbb{F}_2^{w_{k'}}$.
4. For each input prefix $i \in [k']$:

$$\begin{aligned} \overline{\mathbf{e}}_i &\leftarrow \text{S-Ber}_{\mu_i}(\overline{\mathbf{R}}_i) \in \mathbb{F}_2^{w_i}, \quad O(\log \ell) \text{ depth} \\ \text{where } \overline{\mathbf{R}}_i &\leftarrow \mathbf{G}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times w_i}. \end{aligned}$$

5. Compute

$$\overline{\mathbf{y}} \leftarrow \overline{\mathbf{s}} \cdot \overline{\mathbf{A}}_{(1,k')} + \sum_{i=1}^{k'} \overline{\mathbf{e}}_i \cdot \overline{\mathbf{A}}_{(i+1,k')} \in \mathbb{F}_2^{w_{k'}}. \quad O(\log w) \text{ depth}$$

6. Output $\overline{\mathbf{y}} + \overline{\mathbf{p}} \in \mathbb{F}_2^{w_{k'}}$.

Remark. All overlined objects denote submatrices/subvectors of dimension $w_\star$.

Figure 7: Evaluation of PRFs from quasi-polynomial LPN, recursively defined.

Shorthand Notation.

- $\ell := \max(\ell_1, \dots, \ell_k) = \ell_k$ denotes the maximum shrinking ratio due to applying S-Ber.
- $d := \max(d_1, \dots, d_k) = d_1$ denotes the maximum column sparsity of matrix $\mathbf{A}_{\star,\star}$.
- $T := (k\ell)^\tau$ denotes an upper bound on the total number of recursive calls throughout one evaluation of PRF.
- $w_i := \prod_{j=i+1}^k d_j$ denotes the maximum column sparsity of the product $\prod_{j=i+1}^T \mathbf{A}_{j,\star}$. Define $w_k := 1$.
- $w := d^k$ denotes the maximum column sparsity of the product of the form $\prod_{i=1}^T \mathbf{A}_{i,\star}$.

Evaluation Algorithm. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.
2. (In parallel) pre-compute the iterated products

$$\mathbf{A}_{(l,r)} \leftarrow \prod_{i=l}^r \mathbf{A}_{x_i} \in \mathbb{F}_2^{n \times n}, \quad l \in [k], r \in [l, k].^{16} \quad O(\log \lambda + \log k \cdot \log n) \text{ depth}$$

3. For each output index $j \in [n]$ in parallel:

- (a) Compute reachability set from graph walking starting at node j :

$$(\mathcal{I}_k, \dots, \mathcal{I}_0) \leftarrow \text{Reach}(j, \mathbf{A}_{x_k}, \mathbf{A}_{x_{k-1}}, \dots, \mathbf{A}_{x_1})^{17} \quad (\text{Figure 6}) \quad O(k \cdot \log n + \log d^k) \text{ depth}$$

- (b) Column selection and pruning for $l \in [k], r \in [l, k]$ (all in parallel):

$$\bar{\mathbf{A}}_{(l,r)} \leftarrow \text{RowPrune} \left(\text{ColSelect} \left(\mathbf{A}_{(l,r)}, \mathcal{I}_r \right); w_{l-1} \right) \in \mathbb{F}_2^{w_{l-1} \times w_r}, \quad O(\log n) \text{ depth}$$

- (c) Column selection (all in parallel):

$$\bar{\mathbf{S}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{S}_{(k')}, \mathcal{I}_0; w_0 \right) \in \mathbb{F}_2^{T \times w_0}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

$$\bar{\mathbf{P}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{P}_{(k')}, \mathcal{I}_{k'}; w_{k'} \right) \in \mathbb{F}_2^{T \times w_{k'}}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

- (d) Invoke the recursive core $\mathbf{G}$ (Figure 7) with parameter $\bar{\mathbf{S}}_{(k')}, \bar{\mathbf{P}}_{(k')}, \bar{\mathbf{A}}_{(l,r)}$ ¹⁸:

$$\mathbf{y}_j \leftarrow \mathbf{G}_{(k)}(x, 0) \in \mathbb{F}_2. \quad O(\tau \cdot (\log \ell + \log w)) \text{ depth}$$

4. Output $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_n) \in \mathbb{F}_2^n$.

Depth analysis.

1. The iterated matrix multiplication in procedure (2) is done by first selecting all $\mathbf{A}_{x_i}$ based on x_i , which can be done in depth $O(\log \lambda)$. Then pairwise compute matrix multiplication for $\log k$ times, each takes depth $O(\log n)$.
2. By Lemma 8, ColSelect, RowPrune operations can be computed in depth $O(\log n)$.
3. It remains to analyze the circuit depth needed to evaluate the recursion F. Observe that the *recursion topology* is fixed and independent of the PRF input x and key: it is a rooted tree with depth τ and fanout $k\ell_i$ at tree-depth i . Tree root produces the PRF output. Thus the circuit evaluation amounts to unrolling this tree and computing bottom-up from the leaves to the root. At each internal node at tree-depth i , given the output of its $k\ell_i$ number of children, it first apply S-Ber, which takes depth $O(\log \ell_i)$, then compute the vector matrix multiplication along the dimension of w , which takes depth $\log w$. Consequently, the recursion contributes a total depth of $\tau \cdot (\log \ell + \log w)$. Figure 17 reflects a rolling out version of the construction.

¹⁶We assume $\mathbf{A}_{(k+1,k)} := \mathbf{I}_n$ to simplify subsequent notation.

¹⁷Note that the subscripts are written in reverse order to simplify subsequent notation.

¹⁸Note that these matrices depend on j ; we omit the subscript j for simplicity.

Therefore, the overall circuit depth of PRF is

$$O(\log \lambda + \log k \cdot \log n + \tau \cdot (\log \ell + \log w)) = O(\log \lambda).$$

A Variant, $\mathbf{F}'$ (to facilitate analysis)

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{S}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad \mathbf{A}_{(l,r)} \in \mathbb{F}_2^{n \times n}, \text{ for } l \in [k], r \in [l, k].$$

For every $k' = 1, \dots, k$, define $\mathbf{F}'_{(k')}$ as follows.

Function $\mathbf{F}'_{(k')}(x \in [\lambda]^{k'}, \mathcal{I} \subseteq [n], \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_2^{|\mathcal{I}|}$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\mathbf{S}_{(k')}, \mathbf{P}_{(k')})$.
3. $(\mathbf{s}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_2^n \times \mathbb{F}_2^n$.
4. For each $i = 0, \dots, k'$,

$$\mathcal{I}_i \leftarrow \text{RowSupp}(\text{ColSelect}(\mathbf{A}_{(i+1,k')}, \mathcal{I})),$$

5. For each input prefix $i \in [k']$:

$$\bar{\mathbf{e}}_i \leftarrow \text{S-Ber}_{\mu_i}(\bar{\mathbf{R}}_i) \in \mathbb{F}_2^{|\mathcal{I}_i|}, \quad \text{where } \bar{\mathbf{R}}_i \leftarrow \mathbf{F}'_{(i)}^{\ell_i}(x_{1,\dots,i}, \mathcal{I}_i, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times |\mathcal{I}_i|}.$$

6. Compute

$$\bar{\mathbf{y}} \leftarrow \bar{\mathbf{s}} \cdot \bar{\mathbf{A}}_{(1,k')} + \sum_{i=1}^{k'} \bar{\mathbf{e}}_i \cdot \bar{\mathbf{A}}_{(i+1,k')} \in \mathbb{F}_2^{|\mathcal{I}|}.$$

where

$$\begin{aligned} \bar{\mathbf{s}} &\leftarrow \text{ColSelect}(\mathbf{s}, \mathcal{I}_0) \in \mathbb{F}_2^{|\mathcal{I}_0|} \\ \bar{\mathbf{A}}_{(i+1,k')} &\leftarrow \text{RowPrune}(\text{ColSelect}(\mathbf{A}_{(i+1,k')}, \mathcal{I}) \in \mathbb{F}_2^{|\mathcal{I}_i| \times |\mathcal{I}|} \end{aligned}$$

7. Output $\bar{\mathbf{y}} + \bar{\mathbf{p}} \in \mathbb{F}_2^{|\mathcal{I}|}$, where $\bar{\mathbf{p}} \leftarrow \text{ColSelect}(\mathbf{p}, \mathcal{I}) \in \mathbb{F}_2^{|\mathcal{I}|}$.

Remark. All overlined objects denote submatrices/subvectors of dimension $|\mathcal{I}_\star|$.

Figure 8: A Variant, $\mathbf{F}'$ (to facilitate analysis)

Correctness analysis.

Theorem 4. *Our evaluation algorithm (Construction 1) correctly computes PRF defined in Definition 23.*

Proof. Fix a PRF secret key. We introduce a sequence of hybrids as follows.

- PRF (output restriction): This overloads the PRF defined in Definition 23. In addition to x , it takes an index set $\mathcal{I} \subseteq [n]$ and returns the restriction $\text{PRF}(x)|_{\mathcal{I}} \in \mathbb{F}_2^{|\mathcal{I}|}$, instead of the full vector $\text{PRF}(x) \in \mathbb{F}_2^n$. In particular, $\text{PRF}(x) = \text{PRF}(x, \{1, \dots, n\})$.¹⁹
- PRF': This is the same as the above PRF except that instead of calling $\mathbf{F}$ and selects the output based on $\mathcal{I}$ via $\text{PRF}(x, \mathcal{I}) := \text{ColSelect}(\mathbf{F}(x, 0), \mathcal{I})$, it directly calls $\mathbf{F}'$ (Figure 8) to output $\text{PRF}'(x, \mathcal{I}) := \mathbf{F}'(x, \mathcal{I}, 0)$.
- Eval: This is the evaluation algorithm of our PRF construction (Construction 1).

Lemma 11. $\text{PRF} \equiv \text{PRF}'$.

Proof. This follows from the linearity of vector-matrix multiplication and the way we extend $\mathbf{S}\text{-Ber}$ to handle matrix input (i.e., operating on each column separately and independently). In more detail, take a closer look at the execution of $\mathbf{F}$ underlying PRF:

1. Since we only need to return $\mathbf{y}|_{\mathcal{I}}$, we do not need to compute $\mathbf{y}|_{[n] \setminus \mathcal{I}}$ at all.
2. Observe that $\mathbf{y}|_{\mathcal{I}} = \mathbf{s} \cdot \mathbf{A}|_{\mathcal{I}} + \sum \mathbf{e} \cdot \mathbf{A}|_{\mathcal{I}}$, and because of the matrices $\mathbf{A}$ arising in the computation are sparse, each term only depends on $\mathbf{e}|_{\mathcal{I}'}$, where $\mathcal{I}'$ is the row support of $\mathbf{A}|_{\mathcal{I}}$. Now that we only need to know $\mathbf{e}|_{\mathcal{I}'}$, which is actually equal to $\mathbf{S}\text{-Ber}(\mathbf{R})|_{\mathcal{I}'}$ where $\mathbf{R}$ was the output returned by its invoked recursive call. Since $\mathbf{S}\text{-Ber}$ operates on each column of $\mathbf{R}$ independently, we only need $\mathbf{R}|_{\mathcal{I}'}$. In more detail, it holds that for any $\mathcal{I}$, we have

$$\text{ColSelect}(\mathbf{S}\text{-Ber}(\mathbf{R}), \mathcal{I}) = \mathbf{S}\text{-Ber}(\text{ColSelect}(\mathbf{R}, \mathcal{I})),$$

by definition of how we extend $\mathbf{S}\text{-Ber}$ to handle matrix input (i.e., operating on each column separately and independently).

3. $\mathbf{r}$ (row of $\mathbf{R}$) is returned by a recursive call, and we are only interested in entries in $\mathcal{I}'$. This leads to our recursive definition of $\mathbf{F}'$ underlying PRF'.

□

Lemma 12. $\text{PRF}'(x, \{1, \dots, n\}) \equiv \text{Eval}(x)$, for any $x \in [\lambda]^n$.

Proof. It follows from $\mathbf{F}'(x, \{j\}, 0) = \mathbf{G}(x, \{j\}, 0)$ ²⁰ for each $j \in [n]$, which we will prove.

Fix j . We compare the computation transcript of between them. For any node at recursion tree, which corresponds to an execution of $\mathbf{F}'_{(i)}(x_{1..i}, \mathcal{I}'_i, \text{layer})$ (or $\mathbf{G}_{(i)}(x_{1..i}, \mathcal{I}_i, \text{layer})$). We first prove the following lemma.

¹⁹We abuse notation and write PRF for both interfaces; the intended meaning is clear from whether a second input is present.

²⁰Note that the execution of $\mathbf{G}$ is parameterized by a precomputed $\mathcal{I}_i$ that depends on j . We explicitly include the $\mathcal{I}_i$ as part of the input of $\mathbf{G}$ for reference.

Lemma 13. *For any node at recursion tree, which induces a set $\mathcal{I}'_i$ (corresponds to the input to call of $F'_{(i)}$ at the node) and a set $\mathcal{I}_i$ (corresponds to $G_{(i)}$). It holds that $\mathcal{I}'_i \subseteq \mathcal{I}_i$.*

Proof. We prove by induction on recursion depth $\text{layer} = 0, 1, \dots, \tau$.

- Base case: $\text{layer} = 0$ ²¹. It holds that $\mathcal{I} = \mathcal{I}' = \{j\}$. The base case holds.
- Inductive step: for $\text{layer} \in \{1, \dots, \tau\}$. Assume the lemma holds for $\text{layer} - 1$. Fix any node at depth layer of the recursion tree. Denote the corresponding execution by $F'_{(i)}(x_{1..i}, \mathcal{I}'_i, \text{layer})$ and $G_{(i)}(x_{1..i}, \mathcal{I}_i, \text{layer})$. We need to prove that $\mathcal{I}'_i \subseteq \mathcal{I}_i$. Denote their caller's by the execution of $F'_j(x_{1..k'}, \mathcal{I}'_{k'}, \text{layer} - 1)$ and $G_{k'}(x_{1..k'}, \mathcal{I}_{k'}, \text{layer} - 1)$, respectively. Note that $j \geq i$. The inductive assumption implies that $\mathcal{I}'_{k'} \subseteq \mathcal{I}_{k'}$.

Now take a closer look at how we derive $\mathcal{I}_i$ from $\mathcal{I}_{k'}$ at the execution of the caller $G_{(k')}$. Recall that they are derived from walking on the $(k + 1)$ -layered graph induced by $\mathbf{A}_{x_k}, \dots, \mathbf{A}_{x_1}$. Specifically, $u \in \mathcal{I}_i$ if and only if node u at $i + 1$ -th layer of the layered graph is reachable if we start walking at nodes set $\mathcal{I}_{k'}$ at the k' -layer of the layered graph.

Now take a closer look at how we derive $\mathcal{I}'_i$ from $\mathcal{I}'_{k'}$ at the execution of the caller $F'_{(k')}$. It holds that

$$\mathcal{I}_i \leftarrow \text{RowSupp}(\text{ColSelect}(\mathbf{A}_{(i+1,k')}, \mathcal{I}_{k'}); w_i) \in [n]^{w_i},$$

Since $\mathbf{A}_{(i+1,k')} = \prod_{j=i+1}^{k'} \mathbf{A}_{x_j}$, observe that $v \in \mathcal{I}_i$ *only if* v is reachable if we walk from nodes in $\mathcal{I}_{k'}$ over the layered graph induced by $\mathbf{A}_{x_k}, \dots, \mathbf{A}_{x_{i+1}}$. Since $\mathcal{I}'_{k'} \subseteq \mathcal{I}_{k'}$ and reachability is a monotone function, therefore $\mathcal{I}'_i \subseteq \mathcal{I}_i$.

□

We are now ready to prove the lemma by induction on recursion depth $\text{layer} = \tau, \tau - 1, \dots, 0$.

Lemma 14. *For any node at recursion tree, denote the corresponding execution instance by $F'_{(i)}(x_{1..i}, \mathcal{I}'_i, \text{layer})$ and $G_{(i)}(x_{1..i}, \mathcal{I}_i, \text{layer})$, respectively. Denote their return value by $\mathbf{y}' \in \mathbb{F}_2^{|\mathcal{I}'_i|}$, and $\mathbf{y} \in \mathbb{F}_2^{|\mathcal{I}_i|}$ respectively. It holds that $\mathbf{y}|_{\mathcal{I}'_i} = \mathbf{y}'$.*

Proof. We prove by induction on recursion depth $\text{layer} = \tau, \tau - 1, \dots, 0$.

- Base case: $\text{layer} = \tau$. The base layer always outputs a zero vector. Therefore $\mathbf{y}' = F'_{(i)}(x_{1..i}, \mathcal{I}'_i, \tau) = \mathbf{0}$, and $\mathbf{y} = G_{(i)}(x_{1..i}, \tau) = \mathbf{0}$. The base case holds.
- Inductive step: for $\text{layer} \in \{\tau - 1, \dots, 0\}$, assume the lemma holds for $\text{layer} + 1$. Fix any node at depth layer of the recursion tree. Denote the corresponding execution by $F'_{(i)}(x_{1..i}, \mathcal{I}'_i, \text{layer})$ and $G_{(i)}(x_{1..i}, \mathcal{I}_i, \text{layer})$. By Lemma 13, it holds that $\mathcal{I}'_i \subseteq \mathcal{I}_i$. Therefore, in the execution of $G_{(i)}(x_{1..i})$, we only need to care about its output in a smaller subset $\mathcal{I}'_i$. Then by the same argument as in Lemma 11 (namely, by the linearity of vector matrix multiplication, and by the fact the S-Ber operates on each column independently), in order to compute that, we only rely on the $\mathbf{y}_{(j)}|_{\mathcal{I}_j}$ returned by its recursive call $G_{(j)}$ for some $j \leq i$. By our inductive assumption, $\mathbf{y}_{(j)}|_{\mathcal{I}_j}$ agrees with $\mathbf{y}'_{(j)}|_{\mathcal{I}_j}$, which concludes the proof of inductive step.

²¹Note that this base case of the induction should not be confused with the base cases of our recursion.

□

Note that when $\text{layer} = 0$, we have $\mathcal{I}_k = \mathcal{I}'_k = \{j\}$, Lemma 14 implies that $F'_{(k)}(x, \{j\}, 0) = G_{(k)}(x, \{j\}, 0)$, which concludes the proof of Lemma 12. □

This concludes the correctness proof. □

Security analysis. We now prove the reduction to rsLPN.

Theorem 5 (Reduction from rsLPN). *Let all parameters be as in Construction 1. If $[n, m = n\lambda, \mu, d, t, \epsilon]_{2\text{-rsLPN}}$ (Definition 21) assumption holds, then $F_{\mathbf{k}}: \{0, 1\}^{k \log \lambda} \rightarrow \{0, 1\}^n$ defined in Construction 1 is a $(q \leq \lambda^k, t - \text{poly}(\lambda), q\tau k^2 \cdot \epsilon + \lambda^k \cdot \epsilon')$ -pseudorandom function (Definition 3), where*

$$\epsilon' \leq \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^{\tau} \cdot \prod_{j=i+1}^k d_j \right)$$

Proof. See Appendix B.5. □

Corollary 1 (PRFs Security). *Let λ be a security parameter. Assuming quasi-polynomial hardness of LPN (Definition 6), then there exists a secure key-homomorphic pseudorandom function $F_{\mathbf{k}}: \{0, 1\}^{\omega(\log \lambda)} \rightarrow \{0, 1\}^{\lambda}$ computable in depth $O(\log \lambda)$.*

Proof. Set the parameters in Theorem 5 as $n = \lambda^{1/\log \log \lambda}$, $\tau = (\log \log \lambda)^{1/2}$, $k = \log \log \log \lambda$, and as in Definition 22, for $i \in [k]$, set

$$\mu_i = n^{-1/(\log \log n)^{2i+3}}, \quad d_i(n) = n^{1/(\log \log n)^{2i+1}},$$

Then the corollary follows from $q = \lambda^{\omega(1)}$, $\epsilon + \epsilon' = \epsilon + \lambda^{-\omega(1)} = \text{negl}(\lambda)$. □

Key homomorphism. The constructed PRFs have KH structure, which then implies KH-PRF by Lemma 3.

Theorem 6 (KH structure). *Under the parameter setting in Corollary 1, Construction 1 has γ -KH structure (Definition 5) where $\gamma = n^{-c+\delta}$ for any $\delta > 0$.*

Proof. Observe that the first bit of output of the PRF takes the form $\langle \mathbf{s}, \mathbf{a}_x \rangle + \mathbf{p}_1 + \sum_{i=1}^k \mathbf{e}_i \cdot \mathbf{a}'_{x_i}$, which can be written as $\langle \mathbf{s} \parallel \mathbf{p}_1, \mathbf{a}_x \parallel 1 \rangle + \mathbf{e}$, where $e = \sum_{i=1}^k \mathbf{e}_i \cdot \mathbf{a}'_{x_i}$. It then follows that e is zero except with probability $n^{-c+\delta}$ over the randomness of secret key by plugging into our parameter in Corollary 1. □

Combining Corollary 1 (PRF security), Theorem 6 (KH structure), Lemma 1 (domain extension), Lemma 2 (size reduction), we have the following corollary.

Corollary 2 (NC^1 KH-PRFs from quasi-polynomial LPN). *Let λ be a security parameter. Assuming the quasi-polynomial hardness of LPN (Definition 6), there exists a secure KH-PRF family $\mathcal{F} = \{F_{\lambda}: \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{\lambda}\}_{\lambda \in \mathbb{N}}$ in NC^1 for $n(\lambda) \in \omega(\log \lambda)$, and a secure PRF family $\mathcal{F} = \{F_{\lambda}: \{0, 1\}^{\lambda} \rightarrow \{0, 1\}^{\lambda}\}_{\lambda \in \mathbb{N}}$ in linear-size NC^1 .*

5 PRFs from Polynomial Hardness of RingLPN

5.1 Bernoulli Sample Function over $\mathbb{F}_q$

In this section, we introduce a central gadget in our constructions, the Bernoulli sample function S-Ber. We first give its definition and then prove its key properties.

Definition 24 (Bernoulli Sample Function over $\mathbb{F}_q$). Let $\ell \in \mathbb{N}$. For an input $\mathbf{x} = (x_1, x_2) \in \mathbb{F}_q^2$, the Bernoulli sample function $\text{S-Ber} = \text{S-Ber}_\mu : \mathbb{F}_q^2 \rightarrow \mathbb{F}_q$ is defined as follows.

$$\text{S-Ber}(\mathbf{x}) = \mathbb{I}[(x_1)_{1,\dots,\ell} = \mathbf{0}^\ell] \cdot x_2, \quad (10)$$

where $(x_1)_{1,\dots,\ell} \in \{0, 1\}^\ell$ is the least ℓ significant bits of x_1 .

We abuse notation and extend S-Ber to handle inputs $x \in \mathbb{F}_q^{2m \times n}$ by applying S-Ber separately on each 2-entry block of each column, yielding an output in $\mathbb{F}_q^{m \times n}$.

Lemma 15. Fix $\ell, m \in \mathbb{N}$ and $\mu = 2^{-\ell}$. The function $\text{S-Ber}_\mu(\cdot)$ operating on inputs from $\mathbb{F}_q^{2m}$ can be computed by a circuit of depth $O(\log \ell)$. In particular, for $\mu = 1/\text{poly}(n)$, $\text{S-Ber}_\mu(\cdot)$ is computable in depth $O(\log \log n)$.

Lemma 16. It holds that

$$\text{SD}(\text{S-Ber}(\mathbf{u}), e) \leq 2/q, \quad (11)$$

where $\mathbf{u} \leftarrow \mathbb{F}_q^2$, $e \leftarrow \text{Ber}_\mu(\mathbb{F}_q)$. In particular, if we set $q = \lambda^{\omega(1)}$, then

$$\text{SD}(\text{S-Ber}(\mathbf{u}), \text{Ber}_\mu) \leq \text{negl}(\lambda) \quad (12)$$

Proof. See Lemma 22. □

Remark 9. We note that we do not need to use $q = n^{\omega(1)}$. Our idea is to use $q = \text{poly}(n)$, and we set $\mu' := \Pr_{\mathbf{x} \leftarrow \mathbb{F}_q^2}[\text{S-Ber}(\mathbf{x}) \neq 0]$. By Lemma 16, we have $\mu' = \Theta(\mu)$, given that $\mu > 1/n$.

Claim 2 (Bernoulli Output under a Uniform Input). It holds that $\text{S-Ber}_\mu(\mathbf{U}) \sim \text{Ber}_\mu^{n \times m}$ for $\mathbf{U} \leftarrow \mathbb{F}_q^{n \times 2m}$.

Lemma 17 (High Collision of S-Ber). Let $n, m, \ell \in \mathbb{N}$, $\mu = 2^{-\ell}$. Let $\mathbf{P}$ be a uniformly random variable over $\mathbb{F}_2^{n \times m}$. Let $\mathbf{M}, \mathbf{M}'$ be (arbitrary) random variables over $\mathbb{F}_2^{n \times m}$, each independent of $\mathbf{P}$. It holds that

$$\Pr_{\mathbf{P}, \mathbf{M}, \mathbf{M}'}[\text{S-Ber}(\mathbf{P} + \mathbf{M}) \neq \text{S-Ber}(\mathbf{P} + \mathbf{M}')] \leq 2\mu nm. \quad (13)$$

Proof. The proof is exactly the same as in Lemma 7. □

5.2 Functionality

In this section, we specify the input–output behavior of our PRF family. While the definition itself is constructive, evaluating it verbatim does not yield low circuit depth. The next section presents our construction, which gives a depth-efficient evaluation of this functionality.

Recursive Definition of $\mathbf{F}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{A}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_q^{T \times n}, \text{ for } k' \in [k], \quad (\mathbf{S}_1, \dots, \mathbf{S}_\lambda), \text{ where } \mathbf{S}_x \in \mathbb{F}_q^{n \times n} \text{ for } i, x \in [k] \times [\lambda].$$

For every $k' = 1, \dots, k$, define $\mathbf{F}_{(k')}$ as follows.

Function $\mathbf{F}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_q^n$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\mathbf{A}_{(k')}, \mathbf{P}_{(k')})$.
3. $(\mathbf{a}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_q^n \times \mathbb{F}_q^n$.
4. For each input prefix $i \in [k']$:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_\mu(\mathbf{R}_i) \in \mathbb{F}_q^n, \quad \text{where } \mathbf{R}_i \leftarrow \mathbf{F}_{(i)}^2(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_q^{\ell \times n}.$$

5. Compute

$$\begin{aligned} \mathbf{y} &\leftarrow (\dots (\mathbf{a} \cdot \mathbf{S}_{x_1} + \mathbf{e}_1) \cdot \mathbf{S}_{x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{S}_{x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{a} \prod_{i=1}^{k'} \mathbf{S}_{x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{S}_{x_j} \in \mathbb{F}_q^n, \end{aligned}$$

6. Output $\mathbf{y} + \mathbf{p} \in \mathbb{F}_q^n$.

Figure 9: Functionality of PRFs from polynomial RingLPN, recursively defined.

Definition 25 (Functionality of PRFs). We define a keyed function

$$\text{PRF}_{\mathbf{k}}: [\lambda]^k \rightarrow \mathbb{F}_q^n.$$

Parameters.

1. Security parameter $\lambda \in \mathbb{N}$.
2. RingLPN $^{s \sim \chi}$ parameter:
 - (a) Cyclotomic polynomial ring $\mathcal{R}_q$, dimension n , noise rate $\mu = \Pr_{\mathbf{x} \leftarrow \mathbb{S}\mathbb{F}_q^2}[\text{S-Ber}(\mathbf{x}) \neq 0]$.
3. Number of input chunks $k \in \mathbb{N}$. Each chunk has length $\log \lambda$.
4. Maximum recursion depth $\tau \in \mathbb{N}$.

Parameters Setting.

- $n = \lambda$, $q = \text{poly}(\lambda)$, $\mu = \frac{\log^c n}{n}$ for any $c > 2$, $\tau = (\log \log \log \lambda)^2$, $k = \log \log \log \lambda$.

Shorthand Notation.

- $T := (2k)^\tau$ denotes the (maximum) total number of recursive calls throughout the evaluation of PRF.

Function Key. $\mathbf{k} = (\mathbf{S}, \mathbf{P})$, where

- (RingLPN Secret Matrices) $\mathbf{S} = (\mathbf{S}_1, \dots, \mathbf{S}_\lambda)$, where $\mathbf{S}_x = \text{Mat}(s_x) \in \mathbb{F}_q^{n \times n}$, $s_x \leftarrow \text{Ber}_\mu(\mathcal{R}_q)$, for $x \in [\lambda]$. These matrices are shared among all recursions.
- (RingLPN Coefficient Matrices) $\mathbf{A} = (\mathbf{A}_{(1)}, \dots, \mathbf{A}_{(k)})$, with $\mathbf{A}_{(k')} \in \mathbb{F}_q^{T \times n}$ for $k' \in [k]$. For $i \in [T]$, the i -th row of $\mathbf{A}_{(k')}$, i.e., $\mathbf{a}_{k',i} \leftarrow \text{Vec}(a_{k',i}) \in \mathbb{F}_q^n$ for $a_{k',i} \leftarrow \mathcal{R}_q$.
- (Padding Matrices) $\mathbf{P} = (\mathbf{P}_{(1)}, \dots, \mathbf{P}_{(k)})$, with $\mathbf{P}_{(k')} \leftarrow \mathbb{F}_q^{T \times n}$ for $k' \in [k]$.

Functionality. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.
2. Invoke the recursive function $\mathbf{F}$ (Figure 9)

$$\mathbf{y} \leftarrow \mathbf{F}_{(k)}(x, 0) \in \mathbb{F}_2^n.$$

3. Output $\mathbf{y} \in \mathbb{F}_2^n$.

5.3 Construction

Lemma 18. *Let $k, t \in \mathbb{N}$. Let q be a prime. Let $s_i \in \mathcal{R}_q$ such that $\text{wt}(s_i) \leq t$, for $i \in [k]$. Let $y = \prod_{i=1}^k s_i$. It holds that $\text{wt}(y) \leq t^k$.*

Proof. We prove the lemma by induction on k .

- Base case ($k = 1$): We have $y = s_1$, so $\text{wt}(y) = \text{wt}(s_1) \leq t_1 \leq t_1 \cdot t_2 = t$, and hence $\text{wt}(y) \leq t^1$.
- Inductive step ($k > 1$): Assume the statement holds for $k - 1$, i.e., for $y' = \prod_{i=1}^{k-1} s_i$, we have $\text{wt}(y') \leq t^{k-1}$. Now consider $y = y' \cdot s_k \in \mathcal{R}_q$. Before reduction modulo $f(x)$, the product $y' \cdot s_k$ has at most $\text{wt}(y') \cdot \text{wt}(s_k) \leq t^{k-1} \cdot t$ nonzero terms. Reducing modulo $X^n + 1$ will not increase the weight. Thus, after reduction, the total weight is bounded by $\text{wt}(y) \leq t^{k-1} \cdot t = t^k$.

This completes the induction and proves the lemma. $\square$

In this section, we present a depth-efficient evaluation algorithm of the PRF functionality defined in Definition 25, which allows us to compute it in depth $O(\log \lambda)$. In addition, our security analysis (in particular, the resilience property) will crucially rely on this depth-efficient algorithm.

We begin by introducing a basic primitive from graph theory that outputs the reachability set of a walk starting from a given node, over a layered graph induced by an unbalanced bipartite expander Γ ²².

Reachability over a layered graph induced by polynomial ring elements. Consider a $(k+1)$ -layer directed graph with n nodes per layer. Each node has an out-degree of d ²³. For each $i \in [k]$, the edges from layer i to layer $i+1$ are specified by a neighbor function $\phi_i: [n] \rightarrow [n]^d$, such that

$$\text{there is an edge } (i, u) \rightsquigarrow (i+1, v) \quad \text{iff} \quad v \in \phi_i(u).$$

We define a function **Reach** that takes as input a starting node $u \in [n]$ and polynomial rings $(m_1, \dots, m_k)$, where each $m_i \in \mathcal{R}_q$ induces a matrix via $\mathbf{M}_i := \text{Mat}(m_i) \in \mathbb{F}_q^{n \times n}$ (Definition 9). Each $\mathbf{M}_i$ corresponds to a neighbor function $\phi_i(j) := \text{Supp}(\text{ColSelect}(\mathbf{M}_i, j))$. The output is a list of reachability sets $\{\mathbf{c}^{(i)}\}_{i=0}^k$, where each $\mathbf{c}^{(i)} \in [n]^{w_i}$ (with $w_i = \prod_{j=1}^i d_j$ as an upper bound of the reachability set) contains all nodes at layer i that are reachable from u . Each $\mathbf{c}^{(i)}$ is sorted in increasing order and padded with zeros at the end; e.g., $\mathbf{c}^{(i)} = (1, 4, 5, 0, \dots, 0)$.

Reachability $\text{Reach}(u, m_1, \dots, m_k)$

Input: $u \in [n], m_1, \dots, m_k \in \mathcal{R}_q$.

1. $\mathbf{c}_{(0)} \leftarrow (u)$.
2. For $i \in [k]$ in parallel:
 - (a) $m \leftarrow \prod_{j=1}^i m_j \in \mathcal{R}_q \quad O(\log n) \text{ depth}$
 - (b) $\mathbf{A} \leftarrow \text{Mat}(m) \in \mathbb{F}_q^{n \times n}$ (Definition 9)
 - (c) $\mathbf{c}_{(i)} \leftarrow \text{RowSupp}(\text{ColSelect}(\mathbf{A}, u); w_i) \in [n]^{w_i} \quad O(\log n) \text{ depth.}$
3. Output $(\mathbf{c}_{(0)}, \dots, \mathbf{c}_{(k)})$.

Depth: $O(\log n)$

Figure 10: Reachability via Iterated Multiplication

Reach is formally specified in Figure 10 and relies on the equivalence between matrix multiplication and reachability in layered graphs. To compute the iterated ring product, we first map the ring elements $s_1, \dots, s_k$ to their Chinese-remainder representation modulo q (i.e., evaluate the defining polynomials at the n roots of $X^n + 1$ modulo q). In this representation, iterated multiplication becomes coordinate-wise multiplication of k vectors, which is in NC^1 since iterated integer multiplication is in NC^1 [BCH86, HAB02]. Finally, we convert the result back to coefficient

²²We note that this graph should not be confused with the recursion tree induced by Figure 9, but they are related to some extent.

²³ k corresponds to PRF input length, n corresponds to LPN dimension, d corresponds to sparsity of $\mathbf{A}$.

representation. The conversion between coefficient representation to evaluation representation is simply a matrix-vector multiplication over $\mathbb{Z}_q$, which is also in NC^1 since both integer multiplication and iterated integer addition is in NC^1 . Hence the overall depth is $O(\log n)$.

Amplifying correctness probability. Because all operations are modulo q , cancellations may introduce spurious zeros; however, if we take $q = n^{\omega(1)}$, the probability of such collisions is negligible when inputs are uniformly random on their support, as in our RingLPN setting. In fact, it suffices to choose $q = \text{poly}(n)$: by repeating Reach on independent random inputs (as part of the random secret key) supported on the same set, we can amplify the correctness probability to $1 - n^{-\omega(1)}$.

We are now ready to present our PRF construction, which is a depth-efficient algorithm for evaluating the functionality defined in Definition 25. The evaluation is given recursively, mirroring Definition 25, but with two crucial differences: (i) we precompute all products $\prod \mathbf{S}$ in advance via iterated ring product; and (ii) at each recursive call we compute $\bar{\mathbf{e}} \cdot \bar{\mathbf{S}}$ (rather than $\mathbf{e} \cdot \mathbf{S}$), leveraging the precomputed reachability sets so that the computation and output live in dimension w instead of n .

Construction 2 (PRFs from Polynomial Hardness of RingLPN). Let all parameters as defined in Definition 25. Define

$$\text{PRF}_{\mathbf{k}}: [\lambda]^k \rightarrow \mathbb{F}_q^n.$$

Shorthand Notation.

- $T := (k\ell)^\tau$ denotes an upper bound on the total number of recursive calls throughout one evaluation of PRF.
- $d := 2n \cdot \mu$, such that $\mathbf{S}_x$ has column sparsity of d with overwhelming probability.
- $w_i := \prod_{j=i+1}^k d$ denotes the maximum column sparsity of the product $\prod_{j=i+1}^r \mathbf{S}_{\star}$. Define $w_k := 1$.
- $w := d^k$ denotes the maximum column sparsity of the product of the form $\prod_{i=l}^r \mathbf{S}_{i,\star}$.

Evaluation Algorithm. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.
2. (In parallel) pre-compute the iterated products for $l \in [k]$, $r \in [l, k]$:

$$\mathbf{S}_{(l,r)} \leftarrow \text{Mat} \left(\prod_{i=l}^r s_i \right) \in \mathbb{F}_q^{n \times n},^{24} \quad O(\log \lambda + \log n) \text{ depth}$$

3. For each output index $j \in [n]$ in parallel:

- (a) Compute reachability set from graph walking starting at node j :

$$(\mathcal{I}_k, \dots, \mathcal{I}_1) \leftarrow \text{Reach}(j, s_k, s_{k-1}, \dots, s_1)^{25} \quad (\text{Figure 10}) \quad O(\log n) \text{ depth}$$

²⁴We will simply assume $\mathbf{A}_{(k+1,k)} := \mathbf{I}_n$ later.

²⁵Note that the subscript is denoted in a reversed order to ease the later notation

Recursive Definition of Core $\mathbf{G}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\overline{\mathbf{A}}_{(k')}, \overline{\mathbf{P}}_{(k')} \in \mathbb{F}_q^{T \times w_0} \times \mathbb{F}_q^{T \times w_{k'}}, \text{ for } k' \in [k], \quad \overline{\mathbf{S}}_{(l,r)} \in \mathbb{F}_q^{w_{l-1} \times w_r}, \text{ for } l \in [k], r \in [l, k].$$

For every $k' = 1, \dots, k$, define $\mathbf{G}_{(k')}$ as follows.

Function $\mathbf{G}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_q^{w_{k'}}$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}.Init(\overline{\mathbf{A}}_{(k')}, \overline{\mathbf{P}}_{(k')})$.
3. $(\overline{\mathbf{a}}, \overline{\mathbf{p}}) \leftarrow \text{Iter}_{(k')}.Next() \in \mathbb{F}_q^{w_0} \times \mathbb{F}_q^{w_{k'}}$.
4. For each input prefix $i \in [k']$:

$$\begin{aligned} \overline{\mathbf{e}}_i &\leftarrow \text{S-Ber}(\overline{\mathbf{R}}_i) \in \mathbb{F}_q^{w_i}, \quad O(\log \ell) \text{ depth} \\ \text{where } \overline{\mathbf{R}}_i &\leftarrow \mathbf{G}_{(i)}^2(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_q^{2 \times w_i}. \end{aligned}$$

5. Compute

$$\overline{\mathbf{y}} \leftarrow \overline{\mathbf{a}} \cdot \overline{\mathbf{S}}_{(1,k')} + \sum_{i=1}^{k'} \overline{\mathbf{e}}_i \cdot \overline{\mathbf{S}}_{(i+1,k')} \in \mathbb{F}_q^{w_{k'}}. \quad O(\log w + \log \log q) \text{ depth}$$

6. Output $\overline{\mathbf{y}} + \overline{\mathbf{p}} \in \mathbb{F}_2^{w_{k'}}$.

Remark. All overlined objects denote submatrices/subvectors of dimension w .

Figure 11: Evaluation of PRFs from RingLPN, recursively defined.

- (b) Column selection and pruning for $l \in [k], r \in [l, k]$ (all in parallel):

$$\overline{\mathbf{S}}_{(l,r)} \leftarrow \text{RowPrune} \left(\text{ColSelect} \left(\mathbf{S}_{(l,r)}, \mathcal{I}_r \right), w_{l-1} \right) \in \mathbb{F}_q^{w_{l-1} \times w_r}, \quad O(\log n) \text{ depth}$$

- (c) Column selection (all in parallel):

$$\overline{\mathbf{A}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{A}_{(k')}, \mathcal{I}_{k'}; w_0 \right) \in \mathbb{F}_q^{T \times w_0}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

$$\overline{\mathbf{P}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{P}_{(k')}, \mathcal{I}_{k'}; w_{k'} \right) \in \mathbb{F}_q^{T \times w_{k'}}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

- (d) Invoke the recursive core $\mathbf{G}$ (Figure 11) with parameter $\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')}, \overline{\mathbf{A}}_{(l,r)}$ ²⁶:

$$\mathbf{y}_j \leftarrow \mathbf{G}_{(k)}(x, 0) \in \mathbb{F}_q. \quad O(\tau \cdot (\log \ell + \log w + \log \log q)) \text{ depth}$$

²⁶Note that these matrices depend on j ; we omit the subscript j for simplicity.

4. Output $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_n) \in \mathbb{F}_q^n$.

Depth analysis. The depth analysis is exactly the same as Construction 1, except that use ring product to perform iterated matrix multiplication in low depth. The overall circuit depth of PRF is

$$O(\log \lambda + \log n + \tau \cdot (\log \ell + \log w + \log \log q)) = O(\log \lambda).$$

Correctness analysis. The correctness analysis is exactly the same as Theorem 4 for Construction 1 except that we rename $\mathbf{A}$ with $\mathbf{S}$ and rename $\mathbf{S}$ with $\mathbf{A}$.

Security analysis.

Theorem 7 (Reduction from RingLPN $^{s \sim \chi}$). *Let all parameters be as in Definition 25. If $[n, m = \text{poly}(n), \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN $^{s \sim \chi}$ (Definition 10) assumption holds, then $F_{\mathbf{k}}: \{0, 1\}^{k \log \lambda} \rightarrow \{0, 1\}^n$ defined in Construction 2 is a $(q = \lambda^k, t - \text{poly}(\lambda), \tau k^2 \cdot \text{poly}(n) \cdot \epsilon + \lambda^k \cdot (\mu \cdot k d^k (2k)^\tau)^{\tau-1})$ -pseudorandom function (Definition 3).*

Proof. See Appendix B.6. □

Corollary 3 (PRFs security). *Let λ be a security parameter. Assuming polynomial hardness of RingLPN (Definition 8), then there exists a secure pseudorandom function $F_{\mathbf{k}}: \{0, 1\}^{\omega(\log \lambda)} \rightarrow \{0, 1\}^\lambda$ computable in depth $O(\log \lambda)$.*

Combining Corollary 3 (PRF security), Lemma 1 (domain extension), Lemma 2 (size reduction), we have the following corollary.

Corollary 4 (NC¹ PRFs from polynomial hardness of RingLPN). *Let λ be a security parameter. Let $q = q(\lambda)$ be any polynomial. Assuming the polynomial hardness of RingLPN (Definition 11), then there exists a secure pseudorandom function family $\mathcal{F} = \{F_\lambda: \mathbb{F}_q^\lambda \rightarrow \mathbb{F}_q^\lambda\}_{\lambda \in \mathbb{N}}$ in linear-size NC¹.*

6 PRFs from Polynomial Hardness of SparseLPN

6.1 Functionality

In this section, we specify the input–output behavior of our PRF family. While the definition itself is constructive, evaluating it verbatim does not yield low circuit depth. The next section presents our construction, which gives a depth-efficient evaluation of this functionality.

Definition 26 (Functionality of PRFs). We define a keyed function

$$\text{PRF}_{\mathbf{k}}: [\lambda]^k \rightarrow \mathbb{F}_2^n.$$

Parameters.

1. Security parameter $\lambda \in \mathbb{N}$.

Recursive Definition of $\mathbf{F}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{S}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad (\mathbf{A}_1, \dots, \mathbf{A}_\lambda), \text{ where } \mathbf{A}_x \in \mathbb{F}_2^{n \times n} \text{ for } i, x \in [k] \times [\lambda].$$

For every $k' = 1, \dots, k$, define $\mathbf{F}_{(k')}$ as follows.

Function $\mathbf{F}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_2^n$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\mathbf{S}_{(k')}, \mathbf{P}_{(k')})$.
3. $(\mathbf{s}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_2^n \times \mathbb{F}_2^n$.
4. For each input prefix $i \in [k']$:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_\mu(\mathbf{R}_i) \in \mathbb{F}_2^n, \quad \text{where } \mathbf{R}_i \leftarrow \mathbf{F}_{(i)}^\ell(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell \times n}.$$

5. Compute:

$$\begin{aligned} \mathbf{y} &\leftarrow (\dots (\mathbf{s} \cdot \mathbf{A}_{x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{A}_{x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{s} \prod_{i=1}^{k'} \mathbf{A}_{x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{A}_{x_j} \in \mathbb{F}_2^n, \end{aligned}$$

6. Output $\mathbf{y} + \mathbf{p} \in \mathbb{F}_2^n$.

Figure 12: Functionality of PRFs from polynomial SparseLPN, recursively defined.

2. SparseLPN parameter:

- (a) Dimension n , number of samples m , noise rate μ , chosen so that $\mu = 2^{-\ell}$ for $\ell \in \mathbb{N}$. Degree of expander d .
- (b) Expander $\Gamma: [n\lambda] \times [d] \rightarrow [n]$ computable in sublinear depth, i.e., $o(\log n\lambda)$ depth.
- (c) Γ induces an adjacency matrix $\mathbf{A} = (\mathbf{A}_1, \dots, \mathbf{A}_\lambda) \in \mathbb{F}_2^{n \times n\lambda}$. In other words, for $x \in [\lambda]$, $i \in [n\lambda]$, the support of i -th column of $\mathbf{A}_x$ is given by $\Gamma(j + (i-1) \cdot n) \in [n]^d$.

3. Number of input chunks $k \in \mathbb{N}$. Each chunk has length $\log \lambda$.

4. Maximum recursion depth $\tau \in \mathbb{N}$.

Parameters Setting.

- Set $n = \lambda^{1/c}$ for $c > 0$, $\mu = n^{-1/\log \log n}$, $d = \log^2 n$, $\tau = (\log \log \log \lambda)^3$, $k = \log \log \log \lambda$.

Shorthand Notation.

- $T := (k\ell)^\tau$ denotes the (maximum) total number of recursive calls throughout the evaluation of PRF.

Function Key. $\mathbf{k} = (\mathbf{S}, \mathbf{P})$, where

- (SparseLPN Secret Matrices) $\mathbf{S} = (\mathbf{S}_{(1)}, \dots, \mathbf{S}_{(k)})$, with $\mathbf{S}_{(k')} \in \mathbb{F}_2^{T \times n}$, for $k' \in [k]$.
- (Padding Matrices) $\mathbf{P} = (\mathbf{P}_{(1)}, \dots, \mathbf{P}_{(k)})$, with $\mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}$, for $k' \in [k]$.

Functionality. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.
2. Invoke the recursive function $\mathbf{F}$ (Figure 12)

$$\mathbf{y} \leftarrow \mathbf{F}_{(k)}(x, 0) \in \mathbb{F}_2^n.$$

3. Output $\mathbf{y} \in \mathbb{F}_2^n$.

Remark 10. If, at the base of the recursion, we return $R(x) \in \mathbb{F}_2^n$ (for a random function R) instead of $\mathbf{0}$, then the resulting function is a secure PRF under rsLPN via repeating GGM-style hybrid argument for τ times. Thus, to prove the security of our PRFs, it suffices to show that replacing $\mathbf{0}$ by $R(x)$ does not change the output of PRF except with negligible probability. This proof will crucially rely on the depth-efficient evaluation algorithm developed in the next section.

6.2 Walking on Expander Graphs

We first introduce a basic primitive that outputs the reachability set of a walk starting from a given node, over a layered graph induced by an unbalanced bipartite expander Γ ²⁷. Jumping ahead, this functionality enables us to

1. Compute all terms of the form $\mathbf{e} \cdot \prod \mathbf{A}$ in $o(\log n)$ depth via projecting both $\mathbf{e} \in \mathbb{F}_2^n$ and $\mathbf{A} \in \mathbb{F}_2^{n \times n}$ into a dimension (denoted by w) equal to the size of the reachability set (which is only $o(n)$).
2. Compute iterated matrix multiplication of $\prod_i \mathbf{A}_{x_i}$ in $O(\log n)$ depth.

Reachability over a layered graph induced by an unbalanced bipartite expander. Consider a $(k+1)$ -layer directed graph with n nodes per layer. Each node has an out-degree of d .²⁸ For each $i \in [k]$, the edges from layer i to layer $i+1$ are specified by a neighbor function $\phi_i: [n] \rightarrow [n]^d$, such that

$$\text{there is an edge } (i, u) \rightsquigarrow (i+1, v) \quad \text{iff} \quad v \in \phi_i(u).$$

²⁷We note that this graph should not be confused with the recursion tree induced by Figure 5, but they are related to some extent.

²⁸ k corresponds to PRF input length, n corresponds to LPN dimension, d corresponds to sparsity of $\mathbf{A}$.

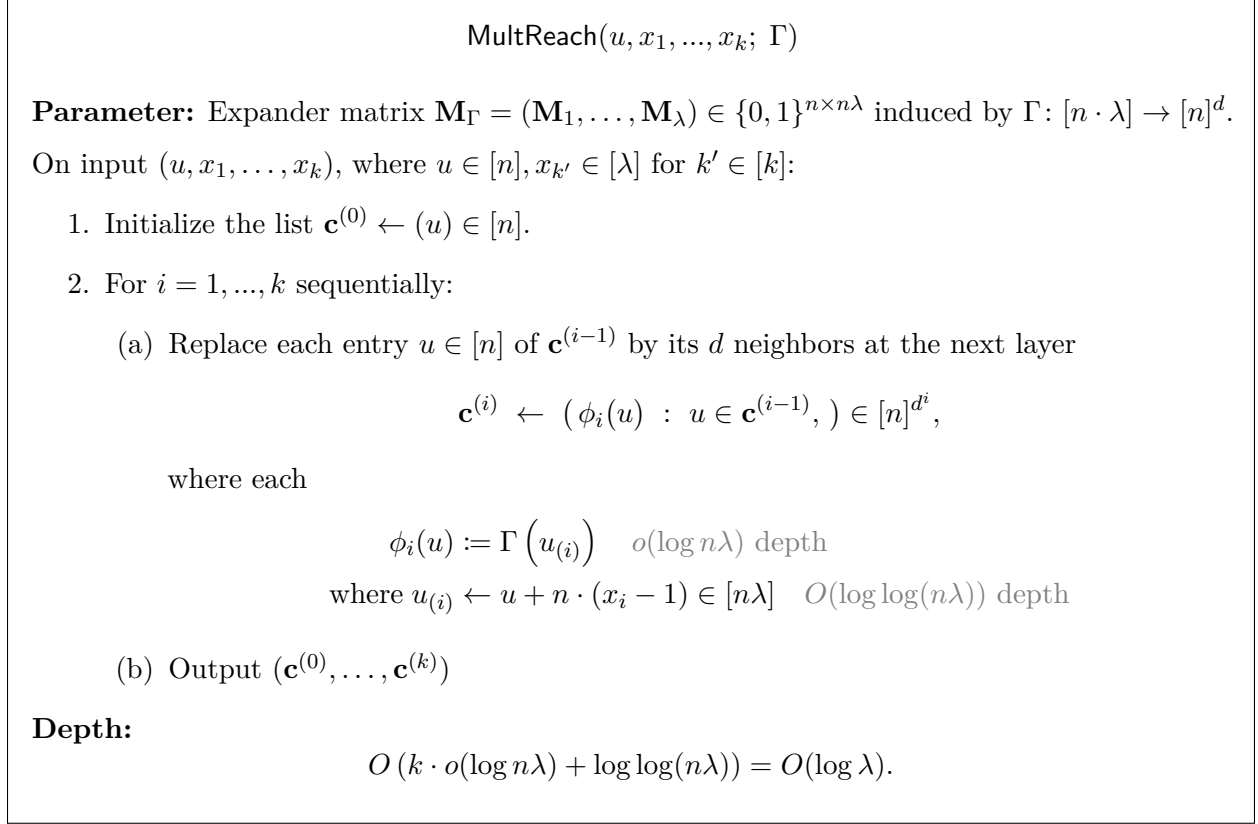


Figure 13: Reachability (with Multiplicity) over $(k + 1)$ -Layered Graph.

Let $\Gamma: [n\lambda] \times [d] \rightarrow [n]$ be an expander, which induces an adjacency matrix $\mathbf{M} = (\mathbf{M}_1, \dots, \mathbf{M}_\lambda) \in \mathbb{F}_2^{n \times n\lambda}$. In other words, the support of j -th column $\mathbf{M}_i$ is given by $\Gamma(j + (i - 1) \cdot n) \in [n]^d$. We define a function **Reach** that takes as input a starting node $u \in [n]$ and $x_1, \dots, x_k \in [\lambda]$, each will specify an adjacency matrix $\mathbf{M}_{x_i} \in \mathbb{F}_2^{n \times n}$ for layer i . In other words, $\phi_i(j) := \text{Supp}(\text{ColSelect}(\mathbf{M}_{x_i}, j))$. It then follows that

$$\phi_i(j) = \Gamma(j + n \cdot (x_i - 1)).$$

The output is a list of reachability sets $\{\mathbf{c}^{(i)}\}_{i=0}^k$, where each $\mathbf{c}^{(i)} \in [n]^{w_i}$ (with $w_i = d^i$ as an upper bound of the reachability set) contains all nodes at layer i that are reachable from u . Each $\mathbf{c}^{(i)}$ is sorted in increasing order and padded with zeros at the end; e.g., $\mathbf{c}^{(i)} = (1, 4, 5, 0, \dots, 0)$.

We rely on *explicit* expander constructions. In our setting, the input to the explicit neighbor function Γ has length $O(\log(n\lambda))$. Hence, if Γ is computable in sublinear depth, then our circuit depth is only $\log(n\lambda)^{o(1)}$. This stands in contrast to a random expander given by an adjacency table, where a neighbor lookup requires depth $\Theta(\log(n\lambda))$.

Fortunately, known (unbalanced) explicit construction of expanders [TUZ01, TUZ07, GUV09, KT22] admit sublinear-depth evaluation (though those works emphasize more on its poly-time evaluation); Indeed, these expanders are computable in $\text{poly}(\log \log(n\lambda))$. This low-depth computability lets us perform the required graph traversals in low depth.

To enable a modular treatment, we first present **MultReach** in Figure 13, which returns a multiset (unordered and possibly with duplicates) of reachable nodes. Procedure (2) can be viewed as a

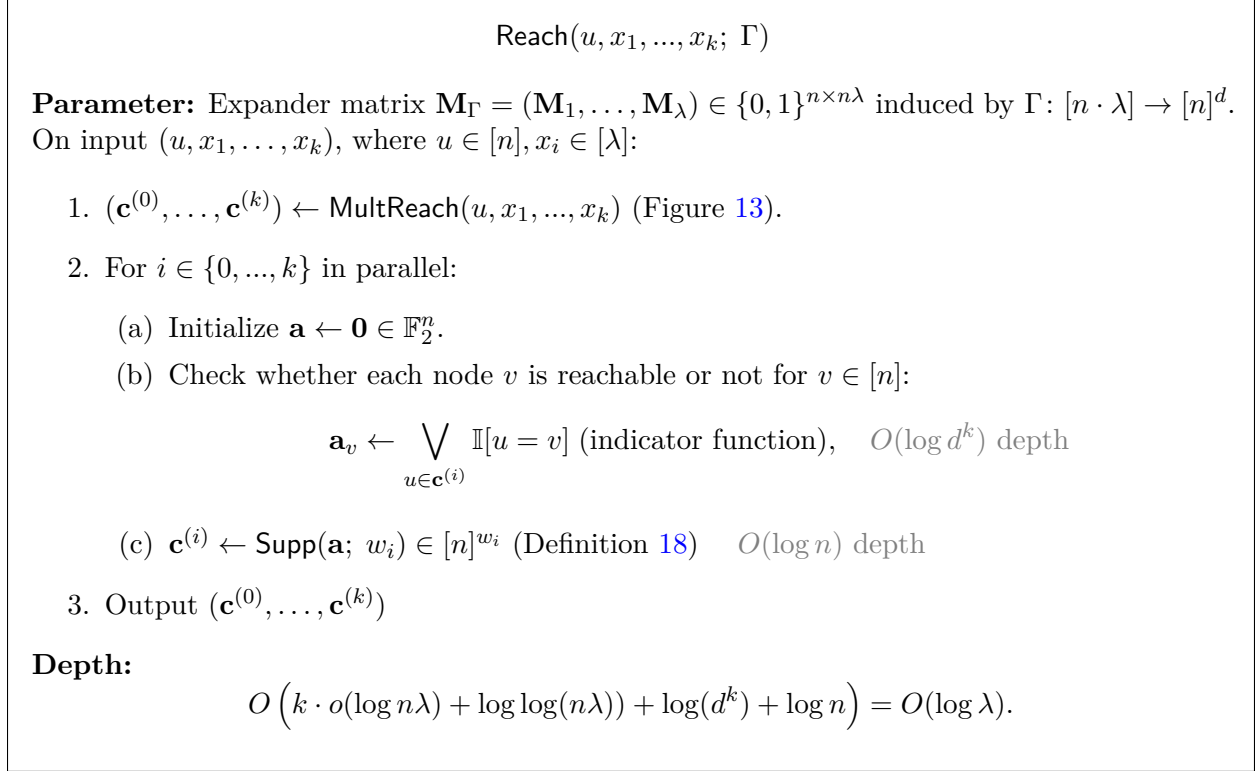


Figure 14: Reachability (without Multiplicity) over Layered Expander Graph.

breadth-first search (BFS) starting at u for k steps. The list $\mathbf{c}^{(i)}$ collects all nodes reachable by walks of exactly i steps²⁹.

We then define Reach (Figure 14), which basically sorts, deduplicates, and pads $\mathbf{c}^{(i)}$ to length w using a bucket/bin-style routine. In more detail, we exploit that all values lie in $[n]$, enabling parallel bucketing and compaction, which yields depth $O(\log n + \log w)$. Instantiating $w = d^k$, this becomes $O(\log n + \log d^k)$. Combining both phases, the overall circuit depth of Reach is

$$O\left(k \cdot (o(\log n\lambda) + \log \log(n\lambda)) + \log(d^k) + \log n\right).$$

Iterated matrix multiplication via path counting. We will rely on the observation that iterated matrix multiplication over $\mathbb{F}_2$ is equivalent to counting the number of paths (modulo 2) between each pair of nodes, which then reduces to the problem to reachability with multiplicity (i.e., MultReach we constructed before), and then use a bin-style approach to count (modulo 2) the occurrence of every node $i \in [n]$ in the reachability set.

Let $\Gamma: [n\lambda] \times [d] \rightarrow [n]$ be an expander, which induces an adjacency matrix $\mathbf{M} = (\mathbf{M}_1, \dots, \mathbf{M}_\lambda) \in \mathbb{F}_2^{n \times n\lambda}$. In other words, the support of j -th column $\mathbf{M}_i$ is given by $\Gamma(j + (i-1) \cdot n) \in [n]^d$. We define a function iMatMul that takes as input $x_1, \dots, x_k \in [\lambda]$, and outputs $\prod_{i=1}^k \mathbf{M}_{x_i}$, which is described in

²⁹Since it is a layered graph, all nodes reachable by t steps are at layer t

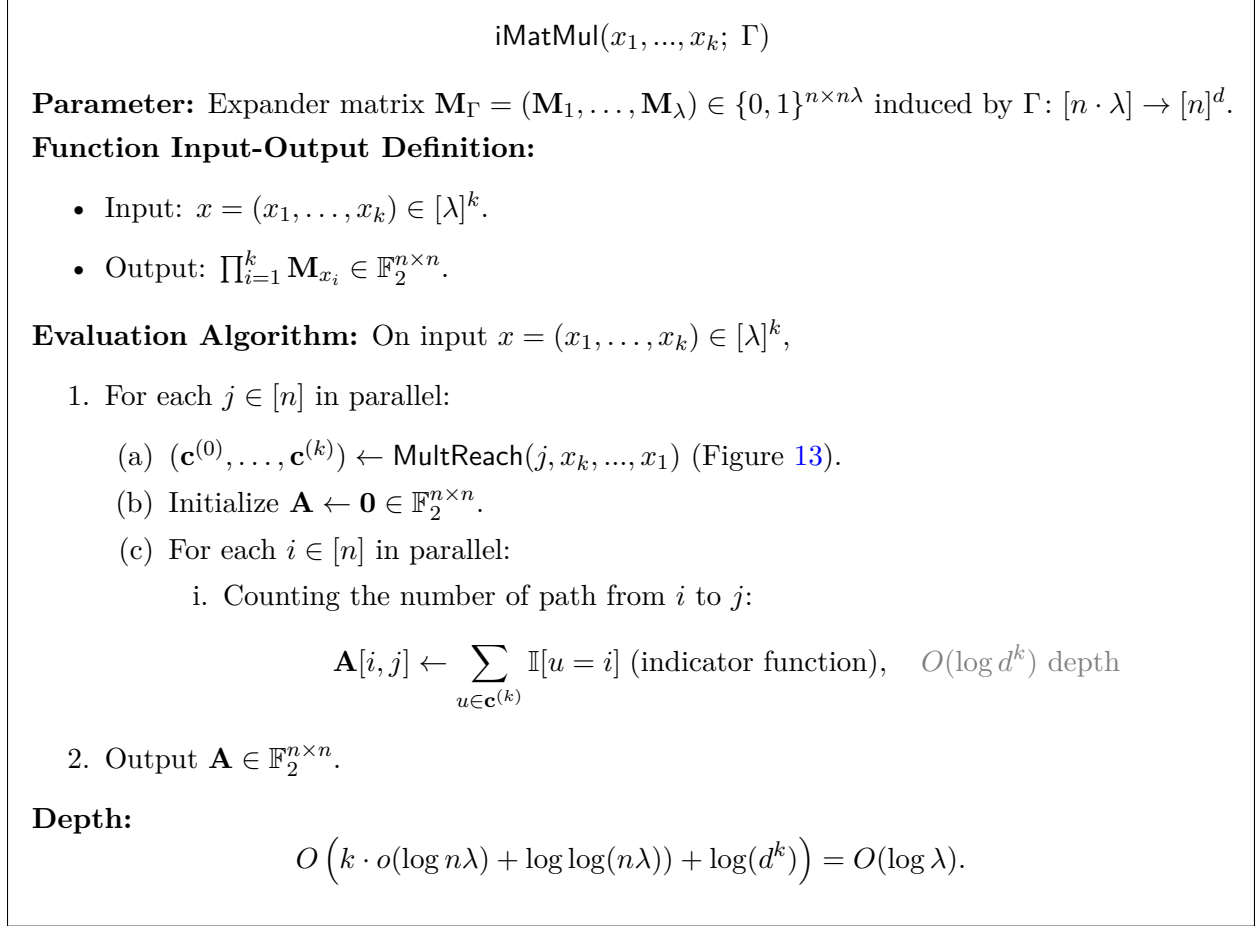


Figure 15: Iterated Matrix Multiplication.

Figure 15. The depth analysis is similar to `Reach`, and the overall circuit depth of `iMatMul` is

$$O\left(k \cdot (o(\log n\lambda) + \log \log(n\lambda)) + \log(d^k)\right)$$

6.3 Construction

In this section, we present a depth-efficient evaluation algorithm of the PRF functionality defined in Definition 23, which allows us to compute it in depth $O(\log \lambda)$. In addition, our security analysis (in particular, the resilience property) will crucially rely on this depth-efficient algorithm.

We are now ready to present our PRF construction, which is a depth-efficient algorithm for evaluating the functionality defined in Definition 23. The evaluation is given recursively, mirroring Definition 23, but with two crucial differences: (i) we precompute all products $\prod \mathbf{A}$ in advance via our `iMatMul`; and (ii) at each recursive call we compute $\bar{\mathbf{e}} \cdot \overline{\mathbf{A}}$ (rather than $\mathbf{e} \cdot \mathbf{A}$), leveraging the precomputed reachability sets so that the computation and output live in dimension w instead of n .

Construction 3 (PRFs from Polynomial Hardness of SparseLPN). Let all parameters as

Recursive Definition of Core G

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')} \in \mathbb{F}_2^{T \times w_0} \times \mathbb{F}_2^{T \times w_{k'}}, \text{ for } k' \in [k], \quad \overline{\mathbf{A}}_{(l,r)} \in \mathbb{F}_2^{w_{l-1} \times w_r}, \text{ for } l \in [k], r \in [l, k].$$

For every $k' = 1, \dots, k$, define $\mathsf{G}_{(k')}$ as follows.

Function $\mathsf{G}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If $\text{layer} = \tau$, return $\mathbf{0} \in \mathbb{F}_2^{w_{k'}}$.
2. If $\text{layer} = 0$, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')})$.
3. $(\overline{\mathbf{s}}, \overline{\mathbf{p}}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_2^{w_0} \times \mathbb{F}_2^{w_{k'}}$.
4. For each input prefix $i \in [k']$:

$$\begin{aligned} \overline{\mathbf{e}}_i &\leftarrow \text{S-Ber}_\mu(\overline{\mathbf{R}}_i) \in \mathbb{F}_2^{w_i}, \quad O(\log \ell) \text{ depth} \\ \text{where } \overline{\mathbf{R}}_i &\leftarrow \mathsf{G}_{(i)}^\ell(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell \times w_i}. \end{aligned}$$

5. Compute

$$\overline{\mathbf{y}} \leftarrow \overline{\mathbf{s}} \cdot \overline{\mathbf{A}}_{(1,k')} + \sum_{i=1}^{k'} \overline{\mathbf{e}}_i \cdot \overline{\mathbf{A}}_{(i+1,k')} \in \mathbb{F}_2^{w_{k'}}. \quad O(\log w) \text{ depth}$$

6. Output $\overline{\mathbf{y}} + \overline{\mathbf{p}} \in \mathbb{F}_2^{w_{k'}}$.

Remark. All overlined objects denote submatrices/subvectors of dimension w .

Figure 16: Evaluation of PRFs from SparseLPN, recursively defined.

defined in Definition 26.

$$\text{PRF}_{\mathbf{k}}: [\lambda]^k \rightarrow \mathbb{F}_2^n.$$

Shorthand Notation.

- $T := (k\ell)^\tau$ denotes an upper bound on the total number of recursive calls throughout one evaluation of PRF.
- $w_i := \prod_{j=i+1}^k d$ denotes the maximum column sparsity of the product $\prod_{j=i+1}^\tau \mathbf{A}_\star$. Define $w_k := 1$.
- $w := d^k$ denotes the maximum column sparsity of the product of the form $\prod_{i=l}^r \mathbf{A}_\star$.

Evaluation Algorithm. On input $x \in [\lambda]^k$:

1. Parse $x = (x_1, \dots, x_k)$ with $x_{k'} \in [\lambda]$ for $k' \in [k]$.

2. (In parallel) pre-compute the iterated products for $l \in [k]$, $r \in [l, k]$:

$$\mathbf{A}_{(l,r)} \leftarrow \text{iMatMul}(x_l, \dots, x_r) \in \mathbb{F}_2^{n \times n},^{30} \quad O(k \cdot (o(\log n \lambda) + \log \log(n \lambda)) + \log(d^k)) \text{ depth}$$

3. For each output index $j \in [n]$ in parallel:

(a) Compute reachability set from graph walking starting at node j :

$$(\mathcal{I}_k, \dots, \mathcal{I}_1) \leftarrow \text{Reach}(j, x_k, x_{k-1}, \dots, x_1)^{31} \quad (\text{Figure 6}) \quad O(k \cdot \log n + \log d^k) \text{ depth}$$

(b) Column selection and pruning for $l \in [k]$, $r \in [l, k]$ (all in parallel):

$$\overline{\mathbf{A}}_{(l,r)} \leftarrow \text{RowPrune} \left(\text{ColSelect} \left(\mathbf{A}_{(l,r)}, \mathcal{I}_r \right), w_{l-1} \right) \in \mathbb{F}_2^{w_{l-1} \times w_r}, \quad O(\log n) \text{ depth}$$

(c) Column selection (all in parallel):

$$\overline{\mathbf{S}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{S}_{(k')}, \mathcal{I}_{k'}; w_0 \right) \in \mathbb{F}_2^{T \times w_0}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

$$\overline{\mathbf{P}}_{(k')} \leftarrow \text{ColSelect} \left(\mathbf{P}_{(k')}, \mathcal{I}_{k'}; w_{k'} \right) \in \mathbb{F}_2^{T \times w_{k'}}, \quad \text{for } k' \in [k] \quad O(\log n) \text{ depth}$$

(d) Invoke the recursive core $\mathbf{G}$ (Figure 16) with parameter $\overline{\mathbf{S}}_{(k')}, \overline{\mathbf{P}}_{(k')}, \overline{\mathbf{A}}_{(l,r)}$ ³²:

$$\mathbf{y}_j \leftarrow \mathbf{G}_{(k)}(x, 0) \in \mathbb{F}_2. \quad O(\tau \cdot (\log \ell + \log w)) \text{ depth}$$

4. Output $\mathbf{y} = (\mathbf{y}_1, \dots, \mathbf{y}_n) \in \mathbb{F}_2^n$.

Depth analysis. The depth analysis is exactly the same as Construction 1, except that use iMatMul to perform iterated matrix multiplication in low depth. The overall circuit depth of PRF is

$$O(k \cdot (o(\log n \lambda) + \log \log(n \lambda)) + \log(d^k) + \log n + \tau \cdot (\log \ell + \log w)) = O(\log \lambda).$$

Correctness analysis. The correctness analysis is exactly the same as Theorem 4 for Construction 1.

Security analysis.

Theorem 8 (Reduction from SparseLPN). *Let all parameters be as in Construction 3. If $[n, m = n\lambda, \mu, t, \epsilon]_2$ -SparseLPN (with respect to Γ) (Definition 13) holds, then $F_{\mathbf{k}}: \{0, 1\}^{k \log \lambda} \rightarrow \{0, 1\}^n$ defined in Definition 26 is a $(q \leq \lambda^k, t - \text{poly}(\lambda), \epsilon + q\tau k^2 \cdot \lambda^k \cdot (\mu \cdot kd^k(k\ell)^\tau)^{\tau-1})$ -pseudorandom function (Definition 3).*

³⁰We will simply assume $\mathbf{A}_{(k+1,k)} := \mathbf{I}_n$ later.

³¹Note that the subscript is denoted in a reversed order to ease the later notation

³²Note that these matrices depend on j ; we omit the subscript j for simplicity.

Proof. The proof is exactly the same as in Theorem 5. $\square$

Corollary 5 (PRFs security). *Let λ be a security parameter. Assuming polynomial hardness of SparseLPN (with respect to Γ) (Definition 13), then there exists a secure pseudorandom function $\mathcal{K} \times \{0, 1\}^{\omega(\log \lambda)} \rightarrow \{0, 1\}^\lambda$ computable in $O(\log \lambda)$ depth.*

Key homomorphism. The constructed PRFs have KH structure, which then implies KH-PRF by Lemma 3.

Theorem 9 (KH structure). *Under the parameter setting in Corollary 5, Construction 3 has γ -KH structure (Definition 5) where $\gamma = n^{-c+\delta}$ for any $\delta > 0$.*

Proof. The proof is exactly the same as in Theorem 6. $\square$

Combining Corollary 5 (PRFs security), Theorem 9 (KH structure), Lemma 1 (domain extension), Lemma 2 (size reduction), we have the following corollary.

Corollary 6 (NC¹ PRF from Polynomial Hardness of SparseLPN). *Let λ be a security parameter. Assume the polynomial hardness of SparseLPN with respect to an expander family computable in sub-polynomial depth (Definition 14). There exists a secure KH-PRF family $\mathcal{F} = \{F_\lambda: \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^\lambda\}_{\lambda \in \mathbb{N}}$ in NC¹ for $n(\lambda) \in \omega(\log \lambda)$, and a secure PRF family $\mathcal{F} = \{F_\lambda: \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda\}_{\lambda \in \mathbb{N}}$ in linear-size NC¹.*

Acknowledgements

Ding and Komargodski were supported in part by a grant from the Israel Science Foundation (ISF Grant No. 1774/20), and by the European Union (ERC, SCALE,101162665). Jain was supported in part by a Google Faculty Research Scholarship, Amazon, Cylab and QCIT of CMU, 0xParc, a Stellar Foundation Research Grant, and the NSF career award. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

A Unrolled PRF Construction (with SPN structure)

The unrolled version of our PRFs is in Figure 17, which aligns with SPN paradigm.

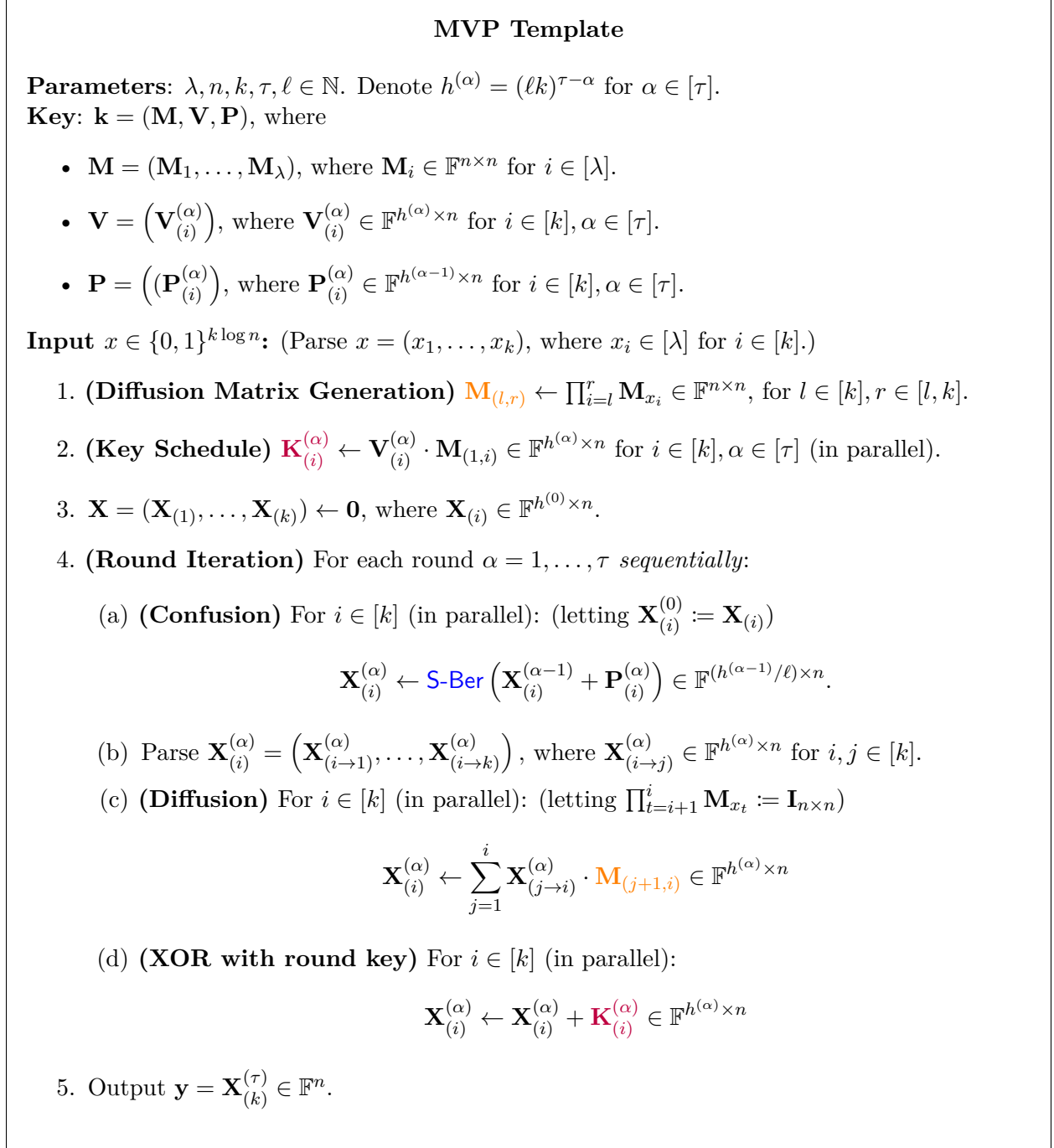


Figure 17: Unrolled version of our PRF construction.

B Omitted Proofs

B.1 Proof of Lemma 4

Lemma 19 (restating Lemma 4). *Let all parameters be as in Definitions 8–10. The $[n, m, \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN assumption implies $[n, m-1, \mu, t-m \cdot \text{poly}(n), \epsilon + 2^{-\Omega(n)}]_{\mathcal{R}_q}$ -RingLPN $^{s \sim \chi}$ assumption.*

Proof. Let $\mathcal{A}$ be a distinguisher for the RingLPN $^{s \sim \chi}$ problem. We construct an algorithm $\mathcal{A}'$ that uses $\mathcal{A}$ to break the RingLPN problem with the same advantage. Algorithm $\mathcal{A}'$ proceeds as follows:

Algorithm $\mathcal{A}'$ takes an RingLPN instance $(\mathbf{a}, \mathbf{b})$ as input and needs to decide whether this input is sampled from $(\mathbf{a}, \mathbf{a} \cdot s + \mathbf{e})$ for $s \leftarrow \mathcal{R}_q$ or from $(\mathbf{a}, \mathbf{u})$ where $\mathbf{a}$ is uniform in $\mathcal{R}_q^m$ and $\mathbf{u}$ is uniform in $\mathcal{R}_q^m$. Algorithm $\mathcal{A}'$ translates $(\mathbf{a}, \mathbf{b})$ into an RingLPN $^{s \sim \chi}$ instance $(\mathbf{a}', \mathbf{b}')$ and then runs $\mathcal{A}$ on $(\mathbf{a}', \mathbf{b}')$. Algorithm $\mathcal{A}'$ takes $(\mathbf{a}, \mathbf{b})$ as input and works as follows:

1. Parse $\mathbf{a} = (a_1, \dots, a_{m+1})$, $\mathbf{b} = (b_1, \dots, b_{m+1})$.
2. If $a_1 = 0$, abort.
3. Set $a'_i \leftarrow -a_1^{-1} \cdot a_i \in \mathcal{R}_q$, $b'_i \leftarrow b_i + b_1 \cdot a'_i \in \mathcal{R}_q$ for $i = 2, \dots, m$.
4. Set $\mathbf{a}' \leftarrow (a'_2, \dots, a'_m) \in \mathcal{R}_q^{m-1}$, $\mathbf{b}' \leftarrow (b'_2, \dots, b'_m) \in \mathcal{R}_q^{m-1}$.
5. Run $\mathcal{A}$ on input $(\mathbf{a}', \mathbf{b}')$ and output whatever $\mathcal{A}$ outputs.

Conditioned on $\mathcal{A}'$ does not abort, which happens except with probability $1/q^n$. It remains to show that $(\mathbf{a}', \mathbf{b}')$ is properly distributed as a RingLPN $^{s \sim \chi}$ problem instance. First, since a_i is uniform in $\mathcal{R}_q$, $\mathbf{a}'_i$ is also uniform in $\mathcal{R}_q$. Second, if the input $\mathbf{b}$ is uniform, then clearly $\mathbf{b}'$ is uniform. Third, If the input $\mathbf{b}$ satisfies $\mathbf{b} = \mathbf{a} \cdot s + \mathbf{e}$, we have

$$\begin{aligned} b'_i &= b_i + b_1 \cdot a'_i \\ &= a_i \cdot s + e_i + a_1 \cdot s + e_1 \cdot (-a_1^{-1} \cdot a_i) \\ &= a'_i \cdot e_1 + e_i \end{aligned}$$

Therefore $(\mathbf{a}', \mathbf{b}')$ is a proper RingLPN $^{s \sim \chi}$ sample where the secret is $s' = e_1$. It follows that $\mathcal{A}'$ decides RingLPN $^{s \sim \chi}$ with the same advantage as $\mathcal{A}$ decides RingLPN. $\square$

B.2 Proof of Lemma 8

Lemma 20 (restating Lemma 8). *Let all parameters be as in Definition 16-20. Every operation in Definition 16-20 can be done in $O(\log(n+m) + \log |\mathbb{F}|)$ depth.*

Proof. All selection operation is computable in depth $O(\log n)$.

Let w be a padding parameter. For support of a vector Supp , we construct the circuit as follows. On input $\mathbf{v} \in \mathbb{F}_q^n$:

1. $a_i \leftarrow \mathbb{I}[\mathbf{v}_i > 0] \in \{0, 1\}$ for $i \in [n]$ in parallel.

2. $b_i \leftarrow \sum_{j=1}^i a_i \in [n]$ (addition over integer) for $i \in [n]$ in parallel.

3. For $j \in [w]$ in parallel:

(a) For $i \in [n]$ in parallel:

$$c_i = \begin{cases} \mathbb{I}[b_i = j], & j = 1, \\ \mathbb{I}[b_i = j \wedge \mathbf{b}_{i-1} = j - 1] & j > 1. \end{cases}$$

(b) Compute $p \leftarrow \sum_{i=1}^n c_i \cdot i$.

(c) $p_j \leftarrow p \in [n]$

4. Output $p_1, \dots, p_w$.

The first step takes depth $\log q$, other steps takes depth $O(\log n)$. The overall depth is $O(\log q + \log n)$. RowSupp can be computed by computing Supp for each column in parallel. $\square$

B.3 Proof of Lemma 10

Lemma 21 (restating Lemma 10). *If LPN is quasi-polynomially hard (Definition 7), then rsLPN is also quasi-polynomially hard (Definition 22).*

Proof. Fix $i \in \{1, 2, \dots, \log \log \log n\}$. By the reduction in Lemma 9, the reduced LPN instance has dimension

$$n'_i = d_i \cdot \log(n/d_i).$$

Noise rate regime. We first check the noise regime. The reduction preserves the noise rate, so we require the LPN hardness to hold at $\mu(n'_i) = \Theta(n^{-1/(\log \log n)^{2i+3}})$. It suffices that

$$\begin{aligned} \mu(n'_i) &= \Theta(n^{-1/(\log \log n)^{2i+3}}) = \Omega((n'_i)^{-1/\log \log n'_i}) \\ &\Leftarrow \frac{\log n}{(\log \log n)^{2i+3}} = o\left(\frac{\log n'_i}{\log \log n'_i}\right) \\ &\Leftrightarrow \frac{\log n}{(\log \log n)^{2i+3}} = o\left(\frac{\log(d_i \log(n/d_i))}{\log \log(d_i \log(n/d_i))}\right) \\ &\Leftarrow \frac{\log n}{(\log \log n)^{2i+3}} = o\left(\frac{\log d_i}{\log \log d_i}\right), \end{aligned} \tag{14}$$

where the last implication uses that $x \mapsto \log x / \log \log x$ is increasing. Plugging $d_i = \Theta(n^{1/(\log \log n)^{2i+1}})$ yields

$$\begin{aligned} &\Leftrightarrow \frac{\log n}{(\log \log n)^{2i+3}} = o\left(\frac{\log n^{1/(\log \log n)^{2i+1}}}{\log \log n^{1/(\log \log n)^{2i+1}}}\right) \\ &\Leftrightarrow \frac{\log n}{(\log \log n)^{2i+3}} = o\left(\frac{\log n}{(\log \log n)^{2i+1}} \cdot \frac{1}{\log \log n - (2i+1) \log \log \log n}\right). \end{aligned} \tag{15}$$

Since $(2i+1) \leq O(\log \log \log n)$, we have $(2i+1) \log \log \log n = O((\log \log \log n)^2) = o(\log \log n)$. Hence

$$\begin{aligned} \iff (\log \log n)^2 &= \omega(\log \log n - (2i+1) \log \log \log n) \\ \iff (\log \log n)^2 &= \omega(\log \log n), \end{aligned} \tag{16}$$

as required, and the required noise regime for LPN is met.

Time/sample/advantage parameters. It remains to check that the running time, number of samples, and distinguishing advantage fall within the assumed LPN hardness window at dimension n'_i . It is enough to show that, for any constant $c > 0$,

$$\begin{aligned} 2^{c \log^{1.01} n} &= 2^{O(\log^{1.1} n'_i)} \\ \iff \log^{1.01} n &= o(\log^{1.1} n'_i) \\ \iff \log^{1.01} n &= o\left((\log(d_i \log(n/d_i)))^{1.1}\right) \\ \iff \log^{1.01} n &= o((\log d_i)^{1.1}). \end{aligned} \tag{17}$$

Substituting $d_i = \Theta(n^{1/(\log \log n)^{2i+1}})$ gives

$$\begin{aligned} \iff \log^{1.01} n &= o\left((\log n^{1/(\log \log n)^{2i+1}})^{1.1}\right) \\ \iff \log^{1.01} n &= o\left(\frac{\log^{1.1} n}{(\log \log n)^{1.1(2i+1)}}\right), \end{aligned} \tag{18}$$

which holds because $(\log \log n)^{1.1(2i+1)} = O(\log \log n)^{\log \log \log n} = o(\log^{0.01} n)$.

Combining the noise and parameter checks, the reduction from Lemma 9 transfers quasi-polynomial hardness from LPN to rsLPN, as claimed. $\square$

B.4 Proof of Lemma 16

Lemma 22 (Restating Lemma 16). *It holds that*

$$\text{SD}(\text{S-Ber}(\mathbf{u}), e) \leq 2/q, \tag{19}$$

where $\mathbf{u} \leftarrow \mathbb{F}_q^2$, $e \leftarrow \text{Ber}_\mu(\mathbb{F}_q)$. In particular, if we set $q = \lambda^{\omega(1)}$, then

$$\text{SD}(\text{S-Ber}(\mathbf{u}), \text{Ber}_\mu) \leq \text{negl}(\lambda) \tag{20}$$

Proof. Let D denote the distribution $\text{S-Ber}(\mathbf{u})$ where $\mathbf{u} \leftarrow \mathbb{Z}_q^2$ uniformly.

Define $A = \{x_1 \in \mathbb{Z}_q : x_1 \bmod 2^\ell = 0\}$. Then $|A| = \lfloor q/2^\ell \rfloor$, so

$$\Pr[x_1 \in A] = \mu' = \frac{\lfloor q/2^\ell \rfloor}{q}$$

Let us now compute the statistical distance $\text{SD}(D, e)$, where $e \leftarrow \text{Ber}_\mu$, defined as:

$$\text{Ber}_\mu = \begin{cases} 0 & \text{with probability } 1 - \mu, \\ a \leftarrow \mathbb{Z}_q \setminus \{0\} & \text{with probability } \mu. \end{cases}$$

We analyze the pointwise difference between the distributions:

- For $a = 0$:

$$|\Pr[D = 0] - \Pr[e = 0]| = |(1 - \mu') - (1 - \mu)| = |\mu - \mu'| \leq \frac{1}{q}.$$

- For $a \neq 0$:

$$\Pr[D = a] = \Pr[x_1 \in A] \cdot \Pr[x_2 = a] = \mu' \cdot \frac{1}{q},$$

$$\Pr[e = a] = \frac{\mu}{q-1},$$

$$|\Pr[D = a] - \Pr[e = a]| \leq \left| \frac{\mu'}{q} - \frac{\mu}{q} \right| + \left| \frac{\mu}{q} - \frac{\mu}{q-1} \right| \leq \frac{1}{q^2} + \frac{\mu}{q(q-1)} \leq \frac{2}{q^2}.$$

There are at most $q - 1$ such $a \neq 0$, so:

$$\sum_{a \neq 0} |\Pr[D = a] - \Pr[e = a]| \leq (q - 1) \cdot \frac{2}{q^2} \leq \frac{2}{q}.$$

Therefore, the total statistical distance is bounded by:

$$\text{SD}(D, e) = \frac{1}{2} \sum_{a \in \mathbb{Z}_q} |\Pr[D = a] - \Pr[e = a]| \leq \frac{1}{2} \left(\frac{1}{q} + \frac{2}{q} \right) \leq \frac{2}{q}.$$

□

B.5 Proof of Theorem 5

Theorem 10 (restating Theorem 5). *Let all parameters be as in Construction 1. If $[n, m = n\lambda, \mu, d, t, \epsilon]_2\text{-rsLPN}$ (Definition 21) assumption holds, then $F_{\mathbf{k}}: \{0, 1\}^{k \log \lambda} \rightarrow \{0, 1\}^n$ defined in Construction 1 is a $(q \leq \lambda^k, t - \text{poly}(\lambda), q\tau k^2 \cdot \epsilon + \lambda^k \cdot \epsilon')$ -pseudorandom function (Definition 3), where*

$$\epsilon' \leq \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^\tau \cdot \prod_{j=i+1}^k d_j \right).$$

Proof. We define hybrid games as follows.

Hybrid H_0 . This is the real PRF game: the attacker has oracle access to $\text{PRF}_{\mathbf{k}}(\cdot)$.

Hybrid H_1 . This is the same as Hybrid H_0 except that the attacker has oracle access to $\widetilde{\text{PRF}}_{\mathbf{k}}(\cdot)$, which is defined the same as PRF , except that instead of invoking F , it will invoke $\tilde{F}$ defined in Figure 18.

Hybrid H_1 : $\tilde{F}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{S}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad (\mathbf{A}_{1,1}, \dots, \mathbf{A}_{k,\lambda}), \text{ where } \mathbf{A}_{i,x} \in \mathbb{F}_2^{n \times n}.$$

For each input length $k' \in [k]$.

$$R_{(k')} = (R_1, \dots, R_T), \text{ where each } R_i \text{ is a random function } R_i: [\lambda]^{j'} \rightarrow \mathbb{F}_2^n$$

For each input length $k' \in [k]$, an iterator $\text{Iter}'_{(k')}$ for $R_{(k')}$

For every $k' = 1, \dots, k$, define $\tilde{F}_{(k')}$ as follows.

Function $\tilde{F}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If **layer** = τ , return $R(x) \in \mathbb{F}_2^n$, where $R \leftarrow \text{Iter}'_{(k')}.Next()$.
2. If **layer** = 0, initialize an iterator $\text{Iter}_{(k')}.Init(\mathbf{S}_{(k')}, \mathbf{P}_{(k')})$, and $\text{Iter}'_{(k')}.Init(R_{(k')})$.
3. $(\mathbf{s}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}.Next() \in \mathbb{F}_2^n \times \mathbb{F}_2^n$.
4. For each input prefix $i \in [k']$:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_{\mu_i}(\mathbf{R}_i) \in \mathbb{F}_2^n, \quad \text{where } \mathbf{R}_i \leftarrow \tilde{F}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times n}.$$

5. Compute:

$$\begin{aligned} \mathbf{y} &\leftarrow (\dots (\mathbf{s} \cdot \mathbf{A}_{1,x_1} + \mathbf{e}_1) \cdot \mathbf{A}_{2,x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{A}_{k',x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{s} \prod_{i=1}^{k'} \mathbf{A}_{i,x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{A}_{j,x_j} \in \mathbb{F}_2^n, \end{aligned}$$

6. Output $\mathbf{y} + \mathbf{p} \in \mathbb{F}_2^n$.

Figure 18: Hybrid H_1 .

Hybrid H_2 . This is the same as Hybrid H_1 except that instead of invoking $\tilde{G}$, PRF will invoke $\tilde{G}$ defined in Figure 19 (invoke for each output bit as we did in Construction 1).

Hybrid H_3 . This is the ideal PRF game.

It follows that $H_1 \equiv H_2$, since they are just two different algorithms to compute the same function.

Lemma 23. *For any efficient adversary $\mathcal{A}$ making at most Q queries, we have*

$$\text{Adv}_{\mathcal{A}}(H_1, H_3) \leq Q \cdot \tau \cdot k^2 \cdot \text{Adv}_{\text{rsLPN}}(n, n \cdot \lambda). \quad (21)$$

Hybrid H_2 : $\tilde{G}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\bar{\mathbf{S}}_{(k')}, \bar{\mathbf{P}}_{(k')} \in \mathbb{F}_2^{T \times n}, \text{ for } k' \in [k], \quad (\bar{\mathbf{A}}_{1,1}, \dots, \bar{\mathbf{A}}_{k,\lambda}), \text{ where } \bar{\mathbf{A}}_{i,x} \in \mathbb{F}_2^{n \times n}.$$

For each input length $k' \in [k]$.

$$\bar{R}_{(k')} = (\bar{R}_1, \dots, \bar{R}_T), \text{ where each } \bar{R}_i \text{ is a random function } : [\lambda]^{j'} \rightarrow \mathbb{F}_2^w$$

For each input length $k' \in [k]$, an iterator $\text{Iter}'_{(k')}$ for $R_{(k')}$

For every $k' = 1, \dots, k$, define $\tilde{G}_{(k')}$ as follows.

Function $\tilde{G}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If **layer** = τ , return $\bar{R}(x) \in \mathbb{F}_2^n$, where $\bar{R} \leftarrow \text{Iter}'_{(k')}. \text{Next}()$.
2. If **layer** = 0, initialize an iterator $\text{Iter}_{(k')}. \text{Init}(\bar{\mathbf{S}}_{(k')}, \bar{\mathbf{P}}_{(k')})$, and $\text{Iter}'_{(k')}. \text{Init}(\bar{R}_{(k')})$.
3. $(\bar{\mathbf{s}}, \bar{\mathbf{p}}) \leftarrow \text{Iter}_{(k')}. \text{Next}() \in \mathbb{F}_2^w \times \mathbb{F}_2^w$.
4. For each input prefix $i \in [k']$:

$$\bar{\mathbf{e}}_i \leftarrow \text{S-Ber}_{\mu_i}(\bar{\mathbf{R}}_i) \in \mathbb{F}_2^w, \quad \text{where } \bar{\mathbf{R}}_i \leftarrow \mathbf{G}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times w}.$$

5. Compute

$$\bar{\mathbf{y}} \leftarrow \bar{\mathbf{s}} \cdot \bar{\mathbf{A}}_{(1,k')} + \sum_{i=1}^{k'} \bar{\mathbf{e}}_i \cdot \bar{\mathbf{A}}_{(i+1,k')} \in \mathbb{F}_2^w.$$

6. Output $\bar{\mathbf{y}} + \bar{\mathbf{p}} \in \mathbb{F}_2^w$.

Remark. All overlined objects denote submatrices/subvectors of dimension w .

Figure 19: Hybrid H_2 .

Proof. The proof exactly mimics that in [DJK25, Lemma 27]. Below we sketch the main idea. Since we replace the base of the recursion by a truly random function. Under the rsLPN assumption, a standard GGM-style hybrid then implies that its the next layer is upgraded into a secure pseudorandom function. Iterating this layer-by-layer “upgrade” up to the output layer yields a sequence of τ hybrids.

□

Lemma 24. *For any adversary $\mathcal{A}$, we have*

$$\text{Adv}_{\mathcal{A}}(H_0, H_2) \leq n^\lambda \cdot \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^\tau \cdot \prod_{j=i+1}^k d_j \right). \quad (22)$$

Proof. We lower bound the probability that the $\text{PRF}_{\mathbf{k}}$ in H_0 and $\widetilde{\text{PRF}}_{\mathbf{k}}$ in H_2 have the same truth table over the randomness of secret key. Then, the failure probability upper bounds any attacker's distinguishing advantage. Therefore, it suffices to prove the following claim. The lemma then follows by applying a union bound over all $x \in [\lambda]^k$.

Claim 3. Fix any $x \in [\lambda]^k$, it holds that

$$\Pr_{\mathbf{k}} \left[\text{PRF}_{\mathbf{k}}(x) \neq \widetilde{\text{PRF}}_{\mathbf{k}}(x) \right] \leq \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^{\tau} \cdot \prod_{j=i+1}^k d_j \right),$$

where the probability is over the randomness of the secret key $\mathbf{k}$.

Proof. Fix any $x \in \{0, 1\}^{k \log n}$. We consider the transcript of the intermediate results during the evaluation of $F_{\mathbf{S}}(x)$ and $\tilde{F}_{\mathbf{S}}^{(3)}(x)$. We use X to denote the random variable during the evaluation of $F_{\mathbf{S}}(x)$ (e.g., $\mathbf{y}, \mathbf{z}$). We use $\tilde{X}$ to denote the random variable during the evaluation of $\tilde{F}_{\mathbf{S}}^{(3)}(x)$ (e.g., $\tilde{\mathbf{y}}, \tilde{\mathbf{z}}$).

Fix an input x . The evaluation of $\text{PRF}_{\mathbf{k}}(x)$ is a branch of recursive calls of $\mathbf{G}$. For a recursion depth $\alpha \in [\tau]$, we collect all the return values of $\mathbf{G}_{\star}(\star, \text{layer} = \alpha)$, and denote by a matrix $\mathbf{R}^{(\alpha)} \in \mathbb{F}_2^{T_{\alpha} \times w}$, where $T_{\alpha} \leq T$. Similarly, we collect all padding vectors that are used at recursion depth α , denoted by $\mathbf{P}^{(\alpha)} \in \mathbb{F}_2^{T_{\alpha} \times w}$. We collect all vectors $\bar{\mathbf{y}}$ that are padded, denoted by $\mathbf{Y}^{(\alpha)} \in \mathbb{F}_2^{T_{\alpha} \times w}$. The same notation applies for $\widetilde{\text{PRF}}_{\mathbf{k}}(x)$, analogously.

Observe that if $\text{PRF}_{\mathbf{k}}(x) \neq \widetilde{\text{PRF}}_{\mathbf{k}}(x)$, then it must hold that for all $\alpha \in [\tau]$, $\mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)}$. This is due to the fact that the execution of their callers at is deterministic. In other words, if there exists α , such that $\mathbf{R}^{(\alpha)} = \tilde{\mathbf{R}}^{(\alpha)}$, then it follows that $\mathbf{R}^{(\alpha-1)} = \tilde{\mathbf{R}}^{(\alpha-1)}$ since $\mathbf{F}_{(\star)}(\star, \text{layer} = \alpha - 1)$ is a deterministic function given the value $\mathbf{R}^{(\alpha)}$ returned by its invoked recursive calls. Repeating this argument yields $\text{PRF}_{\mathbf{k}}(x) = \widetilde{\text{PRF}}_{\mathbf{k}}(x)$.

For $\alpha = 1, \dots, \tau$, we consider the probability of $\mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)}$,

$$\begin{aligned} \Pr_{\mathbf{S}, \tilde{\mathbf{y}}^{(0)}} [\mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)}] &= \Pr[\text{S-Ber}(\mathbf{R}^{(\alpha)}) \neq \text{S-Ber}(\tilde{\mathbf{R}}^{(\alpha)})] \\ &= \Pr[\text{S-Ber}(\mathbf{P}^{(\alpha)} + \mathbf{Y}^{(\alpha)}) \neq \text{S-Ber}(\mathbf{P}^{(\alpha)} + \tilde{\mathbf{Y}}^{(\alpha)})] \\ &\leq 2\mu(k\ell)^{\tau} \cdot w_{\alpha}, \end{aligned} \tag{23}$$

where the last line is due to Lemma 7 and the fact that $\mathbf{P}^{(\alpha)}$ is uniformly random. Therefore, it holds that

$$\begin{aligned} \Pr_{x, \mathbf{S}, \mathbf{y}^{(0)}} \left[\text{PRF}_{\mathbf{k}}(x) \neq \widetilde{\text{PRF}}_{\mathbf{k}}(x) \right] &= \Pr \left[\bigcap_{\alpha=1}^{\tau} \mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)} \right] \\ &= \prod_{\alpha=0}^{\tau-1} \Pr \left[[\mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)}] \mid [\mathbf{R}^{(\alpha+1)} \neq \tilde{\mathbf{R}}^{(\alpha+1)}] \right] \\ &\leq \prod_{\alpha=0}^{\tau-1} \Pr \left[[\mathbf{R}^{(\alpha)} \neq \tilde{\mathbf{R}}^{(\alpha)}] \right] \\ &\leq \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^{\tau} \cdot \prod_{j=i+1}^k d_j \right). \end{aligned} \tag{24}$$

□

□

□

B.6 Proof of Theorem 7

Theorem 11 (Restating Theorem 7). *Let all parameters be as in Definition 25. If $[n, m = \text{poly}(n), \mu, t, \epsilon]_{\mathcal{R}_q}$ -RingLPN $^{s \sim \chi}$ (Definition 10) assumption holds, then $F_{\mathbf{k}}: \{0, 1\}^{k \log \lambda} \rightarrow \{0, 1\}^n$ defined in Construction 2 is a $(q = \lambda^k, t - \text{poly}(\lambda), \epsilon + \lambda^k \cdot (\mu \cdot kd^k(2k)^\tau)^{\tau-1})$ -pseudorandom function (Definition 3).*

Proof. We define hybrid games as follows.

Hybrid H_0 . This is the real PRF game: the attacker has oracle access to $\text{PRF}_{\mathbf{k}}(\cdot)$.

Hybrid H_1 . This is the same as Hybrid H_0 except that the attacker has oracle access to $\widetilde{\text{PRF}}_{\mathbf{k}}(\cdot)$, which is defined the same as PRF , except that instead of invoking F , it will invoke $\widetilde{F}$ defined in Figure 20.

Hybrid H_2 . This is the same as Hybrid H_1 except that instead of invoking $\widetilde{G}$, $\widetilde{\text{PRF}}$ will invoke $\widetilde{G}$ defined in Figure 21 (invoke for each output bit as we did in Construction 2).

Hybrid H_3 . This is the ideal PRF game.

It follows that $H_1 \equiv H_2$, since they are just two different algorithms to compute the same function.

Lemma 25. *For any efficient adversary $\mathcal{A}$ making at most Q queries, we have*

$$\text{Adv}_{\mathcal{A}}(H_1, H_3) \leq \tau \cdot k^2 \cdot \text{Adv}_{\text{RingLPN}^{s \sim \chi}}(n, n \cdot Q). \quad (25)$$

Proof. The proof exactly mimics that in [DJK25, Lemma 27]. Below we sketch the main idea. Since we replace the base of the recursion by a truly random function. Under the rsLPN assumption, a standard GGM-style hybrid then implies that its the next layer is upgraded into a secure pseudorandom function. Iterating this layer-by-layer “upgrade” up to the output layer yields a sequence of τ hybrids.

□

Lemma 26. *For any adversary $\mathcal{A}$, we have*

$$\text{Adv}_{\mathcal{A}}(H_0, H_2) \leq n^\lambda \cdot \prod_{i=1}^{\tau} \left(\mu_i \cdot (k\ell)^\tau \cdot \prod_{j=i+1}^k d_j \right). \quad (26)$$

Proof. The proof the exactly the same as in Lemma 24.

□

□

Hybrid H_1 : $\tilde{F}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\mathbf{A}_{(k')}, \mathbf{P}_{(k')} \in \mathbb{F}_q^{T \times n}, \text{ for } k' \in [k], \quad (\mathbf{S}_1, \dots, \mathbf{S}_\lambda), \text{ where } \mathbf{S}_x \in \mathbb{F}_q^{n \times n}.$$

For each input length $k' \in [k]$.

$$R_{(k')} = (R_1, \dots, R_T), \text{ where each } R_i \text{ is a random function } R_i: [\lambda]^{j'} \rightarrow \mathbb{F}_q^n$$

For each input length $k' \in [k]$, an iterator $\text{Iter}'_{(k')}$ for $R_{(k')}$

For every $k' = 1, \dots, k$, define $\tilde{F}_{(k')}$ as follows.

Function $\tilde{F}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. **If layer = τ , return $R(x) \in \mathbb{F}_q^n$, where $R \leftarrow \text{Iter}'_{(k')}.Next()$.**
2. If layer = 0, initialize an iterator $\text{Iter}_{(k')}.Init(\mathbf{A}_{(k')}, \mathbf{P}_{(k')})$, and $\text{Iter}'_{(k')}.Init(R_{(k')})$.
3. $(\mathbf{a}, \mathbf{p}) \leftarrow \text{Iter}_{(k')}.Next() \in \mathbb{F}_q^n \times \mathbb{F}_q^n$.
4. For each input prefix $i \in [k']$:

$$\mathbf{e}_i \leftarrow \text{S-Ber}_{\mu_i}(\mathbf{R}_i) \in \mathbb{F}_q^n, \quad \text{where } \mathbf{R}_i \leftarrow \tilde{F}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_q^{\ell_i \times n}.$$

5. Compute:

$$\begin{aligned} \mathbf{y} &\leftarrow (\dots (\mathbf{a} \cdot \mathbf{S}_{x_1} + \mathbf{e}_1) \cdot \mathbf{S}_{x_2} + \mathbf{e}_2 \dots) \cdot \mathbf{S}_{x_{k'}} + \mathbf{e}_{k'} \\ &= \mathbf{a} \prod_{i=1}^{k'} \mathbf{S}_{x_i} + \sum_{i=1}^{k'} \mathbf{e}_i \cdot \prod_{j=i+1}^{k'} \mathbf{S}_{x_j} \in \mathbb{F}_q^n, \end{aligned}$$

6. Output $\mathbf{y} + \mathbf{p} \in \mathbb{F}_q^n$.

Figure 20: Hybrid H_1 .

References

- [ABW10] Benny Applebaum, Boaz Barak, and Avi Wigderson. Public-key cryptography from different assumptions. In *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC*, pages 171–180, 2010.
- [ADI⁺17] Benny Applebaum, Ivan Damgård, Yuval Ishai, Michael Nielsen, and Lior Zichron. Secure arithmetic computation with constant computational overhead. In *Advances in Cryptology - CRYPTO*, volume 10401, pages 223–254, 2017.

Hybrid H_2 : $\tilde{G}$

Global Parameters: Max recursion depth τ , iterators, denoted by $\text{Iter}_{(k')}$ for $k' \in [k]$, and

$$\overline{\mathbf{A}}_{(k')}, \overline{\mathbf{P}}_{(k')} \in \mathbb{F}_q^{T \times n}, \text{ for } k' \in [k], \quad (\overline{\mathbf{S}}_1, \dots, \overline{\mathbf{S}}_\lambda), \text{ where } \overline{\mathbf{S}}_x \in \mathbb{F}_q^{n \times n}.$$

For each input length $k' \in [k]$.

$$\overline{R}_{(k')} = (\overline{R}_1, \dots, \overline{R}_T), \text{ where each } \overline{R}_i \text{ is a random function : } [\lambda]^{j'} \rightarrow \mathbb{F}_q^w$$

For each input length $k' \in [k]$, an iterator $\text{Iter}'_{(k')}$ for $R_{(k')}$

For every $k' = 1, \dots, k$, define $\tilde{G}_{(k')}$ as follows.

Function $\tilde{G}_{(k')}(x \in [\lambda]^{k'}, \text{layer})$:

1. If **layer** = τ , return $\overline{R}(x) \in \mathbb{F}_2^n$, where $\overline{R} \leftarrow \text{Iter}'_{(k')}.Next()$.
2. If **layer** = 0, initialize an iterator $\text{Iter}_{(k')}.Init(\overline{\mathbf{A}}_{(k')}, \overline{\mathbf{P}}_{(k')})$, and $\text{Iter}'_{(k')}.Init(\overline{R}_{(k')})$.
3. $(\overline{\mathbf{a}}, \overline{\mathbf{p}}) \leftarrow \text{Iter}_{(k')}.Next() \in \mathbb{F}_2^w \times \mathbb{F}_2^w$.
4. For each input prefix $i \in [k']$:

$$\overline{\mathbf{e}}_i \leftarrow \text{S-Ber}_{\mu_i}(\overline{\mathbf{R}}_i) \in \mathbb{F}_2^w, \quad \text{where } \overline{\mathbf{R}}_i \leftarrow \mathbf{G}_{(i)}^{\ell_i}(x_{1,\dots,i}, \text{layer}+1) \in \mathbb{F}_2^{\ell_i \times w}.$$

5. Compute

$$\overline{\mathbf{y}} \leftarrow \overline{\mathbf{a}} \cdot \overline{\mathbf{S}}_{(1,k')} + \sum_{i=1}^{k'} \overline{\mathbf{e}}_i \cdot \overline{\mathbf{S}}_{(i+1,k')} \in \mathbb{F}_2^w.$$

6. Output $\overline{\mathbf{y}} + \overline{\mathbf{p}} \in \mathbb{F}_2^w$.

Remark. All overlined objects denote submatrices/subvectors of dimension w .

Figure 21: Hybrid H_2 .

- [AIK08] Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. On pseudorandom generators with linear stretch in NC^0 . *Comput. Complex.*, 17(1):38–69, 2008.
- [Ale03] Michael Alekhnovich. More on average case vs approximation complexity. In *44th Symposium on Foundations of Computer Science, FOCS*, pages 298–307, 2003.
- [Ale11] Michael Alekhnovich. More on average case vs approximation complexity. *Comput. Complex.*, 20(4):755–786, 2011.
- [AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Slightly sublinear trapdoor hash functions and PIR from low-noise LPN. In *Theory of Cryptography - 23rd International Conference, TCC (to appear)*, 2025.

- [AN25] Benny Applebaum and Oded Nir. The meta-complexity of secret sharing. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC*, pages 965–976. ACM, 2025.
- [App16] Benny Applebaum. Cryptographic hardness of random local functions - survey. *Comput. Complex.*, 25(3):667–722, 2016.
- [AR16] Benny Applebaum and Pavel Raykov. Fast pseudorandom functions based on expander graphs. In *Theory of Cryptography - 14th International Conference, TCC*, volume 9985, pages 27–56, 2016.
- [BBTV25] Kiril Bangachev, Guy Bresler, Stefan Tiegel, and Vinod Vaikuntanathan. Near-optimal time-sparsity trade-offs for solving noisy linear equations. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC*, pages 1910–1920, 2025.
- [BCG⁺20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from Ring-LPN. In *Advances in Cryptology - CRYPTO*, volume 12171, pages 387–416, 2020.
- [BCH86] Paul Beame, Stephen A. Cook, and H. James Hoover. Log depth circuits for division and related problems. *SIAM J. Comput.*, 15(4):994–1003, 1986.
- [BDGJ20] Colin Boyd, Gareth T. Davies, Kristian Gjøsteen, and Yao Jiang. Fast and secure updatable encryption. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 464–493, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- [BEKS20] Dan Boneh, Saba Eskandarian, Sam Kim, and Maurice Shih. Improving speed and security in updatable encryption schemes. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 559–589, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland.
- [BFKL93] Avrim Blum, Merrick L. Furst, Michael J. Kearns, and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO*, volume 773, pages 278–291, 1993.
- [BHK⁺13] Itay Berman, Iftach Haitner, Ilan Komargodski, and Moni Naor. Hardness preserving reductions via cuckoo hashing. In *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC*, volume 7785, pages 40–59, 2013.
- [BHK⁺19] Itay Berman, Iftach Haitner, Ilan Komargodski, and Moni Naor. Hardness-preserving reductions via Cuckoo hashing. *J. Cryptol.*, 32(2):361–392, 2019.
- [BIP⁺18] Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu. Exploring crypto dark matter: New simple PRF candidates and their applications. In *Theory of Cryptography - 16th International Conference, TCC*, volume 11240, pages 699–729, 2018.
- [BJKL21] Fabrice Benhamouda, Aayush Jain, Ilan Komargodski, and Huijia Lin. Multiparty reusable non-interactive secure computation from LWE. In *Advances in Cryptology - EUROCRYPT*, volume 12697, pages 724–753, 2021.

- [BJMM12] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In *Advances in Cryptology - EUROCRYPT*, volume 7237, pages 520–536, 2012.
- [BKW03] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *J. ACM*, 50(4):506–519, 2003.
- [BL12] Daniel J. Bernstein and Tanja Lange. Never trust a bunny. In *Radio Frequency Identification. Security and Privacy Issues - 8th International Workshop, RFIDSec*, volume 7739, pages 137–148, 2012.
- [BLMR13] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *Advances in Cryptology - CRYPTO*, volume 8042, pages 410–428, 2013.
- [BLP11] Daniel J. Bernstein, Tanja Lange, and Christiane Peters. Smaller decoding exponents: Ball-collision decoding. In *Advances in Cryptology - CRYPTO*, volume 6841, pages 743–760, 2011.
- [BLVW19] Zvika Brakerski, Vadim Lyubashevsky, Vinod Vaikuntanathan, and Daniel Wichs. Worst-case hardness for LPN and cryptographic hashing via code smoothing. In *Advances in Cryptology - EUROCRYPT*, volume 11478, pages 619–635, 2019.
- [BMM24] Amos Beimel, Tal Malkin, and Noam Mazon. Structural lower bounds on black-box constructions of pseudorandom functions. In *Advances in Cryptology - CRYPTO*, volume 14924, pages 459–488, 2024.
- [BP14] Abhishek Banerjee and Chris Peikert. New and improved key-homomorphic pseudorandom functions. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 353–370, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.
- [BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *Advances in Cryptology - EUROCRYPT*, volume 7237, pages 719–737, 2012.
- [BQ12] Andrej Bogdanov and Youming Qiao. On the security of goldreich’s one-way function. *Comput. Complex.*, 21(1):83–127, 2012.
- [BR17] Andrej Bogdanov and Alon Rosen. Pseudorandom functions: Three decades later. In *Tutorials on the Foundations of Cryptography*, pages 79–158. Springer International Publishing, 2017.
- [CC98] Anne Canteaut and Florent Chabaud. A new algorithm for finding minimum-weight words in a linear code: Application to McEliece’s cryptosystem and to narrow-sense BCH codes of length 511. *IEEE Trans. Inf. Theory*, 44(1):367–378, 1998.
- [CHKV25] Henry Corrigan-Gibbs, Alexandra Henzinger, Yael Tauman Kalai, and Vinod Vaikuntanathan. Somewhat homomorphic encryption from linear homomorphism and sparse LPN. In *Advances in Cryptology - EUROCRYPT*, volume 15602, pages 3–33, 2025.

- [CIKK16] Marco L. Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Learning algorithms from natural proofs. In *31st Conference on Computational Complexity, CCC 2016*, volume 50 of *LIPICs*, pages 10:1–10:24, 2016.
- [CM21] Geoffroy Couteau and Pierre Meyer. Breaking the circuit size barrier for secure computation under quasi-polynomial LPN. In *Advances in Cryptology - EUROCRYPT*, volume 12697, pages 842–870, 2021.
- [CSZ25] Xue Chen, Wenxuan Shu, and Zhaienhe Zhou. Algorithms for sparse LPN and LSPN against low-noise (extended abstract). In *The Thirty Eighth Annual Conference on Learning Theory*, volume 291, pages 1091–1093, 2025.
- [DI06] Ivan Damgård and Yuval Ishai. Scalable secure multiparty computation. In Cynthia Dwork, editor, *Advances in Cryptology - CRYPTO*, volume 4117 of *Lecture Notes in Computer Science*, pages 501–520, 2006.
- [DIJL23] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear MPC from sparse LPN. In *Advances in Cryptology - CRYPTO*, volume 14082, pages 315–348, 2023.
- [DJK25] Youlong Ding, Aayush Jain, and Ilan Komargodski. A new approach for LPN-based pseudorandom functions: Low-depth and key-homomorphic. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC*, pages 1898–1909, 2025.
- [DP12] Ivan Damgård and Sunoo Park. How practical is public-key encryption based on LPN and ring-LPN? Cryptology ePrint Archive, Paper 2012/699, 2012.
- [DS15] Nico Döttling and Dominique Schröder. Efficient pseudorandom functions via on-the-fly adaptation. In *Advances in Cryptology - CRYPTO*, volume 9215, pages 329–350, 2015.
- [EKM17] Andre Esser, Robert Kübler, and Alexander May. LPN decoded. In *Advances in Cryptology - CRYPTO*, volume 10402, pages 486–514, 2017.
- [EPRS17] Adam Everspaugh, Kenneth G. Paterson, Thomas Ristenpart, and Samuel Scott. Key rotation for authenticated encryption. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology - CRYPTO 2017, Part III*, volume 10403 of *Lecture Notes in Computer Science*, pages 98–129, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland.
- [FLY22] Zhiyuan Fan, Jiayu Li, and Tianqi Yang. The exact complexity of pseudorandom functions and the black-box natural proof barrier for bootstrapping results in computational complexity. In *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 962–975, 2022.
- [GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *J. ACM*, 33(4):792–807, 1986.
- [GHJS25] Riddhi Ghosal, Isaac M. Hair, Aayush Jain, and Amit Sahai. Using the planted clique conjecture for cryptography: Public-key encryption from planted clique and noisy k -LIN over expanders. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC*, pages 1921–1932, 2025.

- [GJL15] Qian Guo, Thomas Johansson, and Carl L ndahl. A new algorithm for solving ring-lpn with a reducible polynomial. *IEEE Trans. Inf. Theory*, 61(11):6204–6212, 2015.
- [Gol00] Oded Goldreich. Candidate one-way functions based on expander graphs. Cryptology ePrint Archive, Report 2000/063, 2000. <https://eprint.iacr.org/2000/063>.
- [GUV09] Venkatesan Guruswami, Christopher Umans, and Salil P. Vadhan. Unbalanced expanders and randomness extractors from Parvaresh-Vardy codes. *J. ACM*, 56(4):20:1–20:34, 2009.
- [GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In *Advances in Cryptology - CRYPTO*, volume 7417, pages 162–179. Springer, 2012.
- [HAB02] William Hesse, Eric Allender, and David A. Mix Barrington. Uniform constant-depth threshold circuits for division and iterated multiplication. *J. Comput. Syst. Sci.*, 65(4):695–716, 2002.
- [HILL99] Johan H stad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM J. Comput.*, 28(4):1364–1396, 1999.
- [HKL⁺12] Stefan Heyse, Eike Kiltz, Vadim Lyubashevsky, Christof Paar, and Krzysztof Pietrzak. Lapin: An efficient authentication protocol based on ring-LPN. In *Fast Software Encryption - 19th International Workshop, FSE*, volume 7549, pages 346–365, 2012.
- [HRV13] Iftach Haitner, Omer Reingold, and Salil P. Vadhan. Efficiency improvements in constructing pseudorandom generators from one-way functions. *SIAM J. Comput.*, 42(3):1405–1430, 2013.
- [IKOS08] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Cryptography with constant computational overhead. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pages 433–442. ACM, 2008.
- [IL90] Russell Impagliazzo and Leonid A. Levin. No better ways to generate hard NP instances than picking uniformly at random. In *31st Annual Symposium on Foundations of Computer Science*, pages 812–821, St. Louis, MO, USA, October 22–24, 1990. IEEE Computer Society Press.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *STOC ’21: 53rd Annual ACM SIGACT Symposium on Theory of Computing*, pages 60–73, 2021.
- [JLS22] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from LPN over $\mathbb{F}_p$, DLIN, and PRGs in NC^0 . In *Advances in Cryptology - EUROCRYPT*, volume 13275, pages 670–699, 2022.
- [JLS24] Aayush Jain, Huijia Lin, and Sagnik Saha. A systematic study of sparse LWE. In *Advances in Cryptology - CRYPTO*, volume 14922, pages 210–245, 2024.
- [Kim20] Sam Kim. Key-homomorphic pseudorandom functions from LWE with small modulus. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 576–607, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.

- [Kir11] Paul Kirchner. Improved generalized birthday attack. *IACR Cryptol. ePrint Arch.*, page 377, 2011.
- [KLR19] Michael Klooß, Anja Lehmann, and Andy Rupp. (R)CCA secure updatable encryption with integrity protection. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part I*, volume 11476 of *Lecture Notes in Computer Science*, pages 68–99, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [KT22] Itay Kalev and Amnon Ta-Shma. Unbalanced expanders from multiplicity codes. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM*, volume 245, pages 12:1–12:14, 2022.
- [KV89] Michael J. Kearns and Leslie G. Valiant. Cryptographic limitations on learning Boolean formulae and finite automata. In *21st Annual ACM Symposium on Theory of Computing*, pages 433–444, Seattle, WA, USA, May 15–17, 1989. ACM Press.
- [Lev85] Leonid A. Levin. One-way functions and pseudorandom generators. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing*, pages 363–365, 1985.
- [Lev87] Leonid A. Levin. One-way functions and pseudorandom generators. *Comb.*, 7(4):357–363, 1987.
- [LF06] Éric Levieil and Pierre-Alain Fouque. An improved LPN algorithm. In *Security and Cryptography for Networks, 5th International Conference, SCN*, volume 4116 of *Lecture Notes in Computer Science*, pages 348–359, 2006.
- [LLPT23] Hanjun Li, Huijia Lin, Antigoni Polychroniadou, and Stefano Tessaro. LERNA: Secure single-server aggregation via key-homomorphic masking. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part I*, volume 14438 of *Lecture Notes in Computer Science*, pages 302–334, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- [LP15] Helger Lipmaa and Kateryna Pavlyk. Analysis and implementation of an efficient ring-lpn based commitment scheme. In *Cryptology and Network Security - 14th International Conference, CANS*, volume 9476, pages 160–175, 2015.
- [LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology - EUROCRYPT*, volume 6110, pages 1–23, 2010.
- [LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *J. ACM*, 60(6):43:1–43:35, 2013.
- [LT18] Anja Lehmann and Björn Tackmann. Updatable encryption with post-compromise security. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 685–716, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
- [LW09] Allison B. Lewko and Brent Waters. Efficient pseudorandom functions from the decisional linear assumption and weaker variants. In *Proceedings of the 2009 ACM Conference on Computer and Communications Security, CCS*, pages 112–120, 2009.

- [LWYY24] Hanlin Liu, Xiao Wang, Kang Yang, and Yu Yu. The hardness of LPN over any integer ring and field for PCG applications. In *Advances in Cryptology - EUROCRYPT*, volume 14656, pages 149–179, 2024.
- [Lyu05] Vadim Lyubashevsky. The parity problem in the presence of noise, decoding random linear codes, and the subset sum problem. In *Approximation, Randomization and Combinatorial Optimization. Algorithms and Techniques*, pages 378–389, 2005.
- [McE78] R. J. McEliece. A Public-Key Cryptosystem Based On Algebraic Coding Theory. *Deep Space Network Progress Report*, 44:114–116, January 1978.
- [MMT11] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In *Advances in Cryptology - ASIACRYPT*, volume 7073, pages 107–124, 2011.
- [MV12] Eric Miles and Emanuele Viola. Substitution-permutation networks, pseudorandom functions, and natural proofs. In *Advances in Cryptology - CRYPTO*, volume 7417, pages 68–85, 2012.
- [MV15] Eric Miles and Emanuele Viola. Substitution-permutation networks, pseudorandom functions, and natural proofs. *J. ACM*, 62(6):46:1–46:29, 2015.
- [NPR99] Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and KDCs. In Jacques Stern, editor, *Advances in Cryptology – EUROCRYPT’99*, volume 1592 of *Lecture Notes in Computer Science*, pages 327–346, Prague, Czech Republic, May 2–6, 1999. Springer Berlin Heidelberg, Germany.
- [NR95] Moni Naor and Omer Reingold. Synthesizers and their application to the parallel construction of pseudo-random functions. In *36th Annual Symposium on Foundations of Computer Science, FOCS*, pages 170–181. IEEE Computer Society, 1995.
- [NR97] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. In *38th Annual Symposium on Foundations of Computer Science, FOCS*, pages 458–467, 1997.
- [NR99] Moni Naor and Omer Reingold. Synthesizers and their application to the parallel construction of pseudo-random functions. *J. Comput. Syst. Sci.*, 58(2):336–375, 1999.
- [NR04] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. *J. ACM*, 51(2):231–262, 2004.
- [NRR02] Moni Naor, Omer Reingold, and Alon Rosen. Pseudorandom functions and factoring. *SIAM J. Comput.*, 31(5):1383–1404, 2002.
- [OST19] Igor Carboni Oliveira, Rahul Santhanam, and Roei Tell. Expander-based cryptography meets natural proofs. In *10th Innovations in Theoretical Computer Science Conference, ITCS*, volume 124, pages 18:1–18:14, 2019.
- [OST22] Igor C. Oliveira, Rahul Santhanam, and Roei Tell. Expander-based cryptography meets natural proofs. *Comput. Complex.*, 31(1):4, 2022.
- [OW14] Ryan O’Donnell and David Witmer. Goldreich’s PRG: evidence for near-optimal polynomial stretch. In *IEEE 29th Conference on Computational Complexity, CCC*, pages 1–12. IEEE Computer Society, 2014.

- [Pra62] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Trans. Inf. Theory*, 8(5):5–9, 1962.
- [RR97] Alexander A. Razborov and Steven Rudich. Natural proofs. *J. Comput. Syst. Sci.*, 55(1):24–35, 1997.
- [RVV24] Seyoon Ragavan, Neekon Vafa, and Vinod Vaikuntanathan. Indistinguishability obfuscation from bilinear maps and LPN variants. In *Theory of Cryptography - 22nd International Conference, TCC*, volume 15367, pages 3–36, 2024.
- [Sch18] Peter Scholl. Extending oblivious transfer with low communication via key-homomorphic PRFs. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018: 21st International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 10769 of *Lecture Notes in Computer Science*, pages 554–583, Rio de Janeiro, Brazil, March 25–29, 2018. Springer, Cham, Switzerland.
- [SS94] Michael Sipser and Daniel A. Spielman. Expander codes. In *35th Annual Symposium on Foundations of Computer Science*, pages 566–576, 1994.
- [SS96] Michael Sipser and Daniel A. Spielman. Expander codes. *IEEE Trans. Inf. Theory*, 42(6):1710–1722, 1996.
- [Ste89] Jacques Stern. A method for finding codewords of small weight. In *Coding Theory and Applications*, pages 106–113, 1989.
- [TUZ01] Amnon Ta-Shma, Christopher Umans, and David Zuckerman. Loss-less condensers, unbalanced expanders, and extractors. In *Proceedings on 33rd Annual ACM Symposium on Theory of Computing*, pages 143–152, 2001.
- [TUZ07] Amnon Ta-Shma, Christopher Umans, and David Zuckerman. Lossless condensers, unbalanced expanders, and extractors. *Comb.*, 27(2):213–240, 2007.
- [Val84] Leslie G. Valiant. A theory of the learnable. *Commun. ACM*, 27(11):1134–1142, 1984.
- [Vio15] Emanuele Viola. The communication complexity of addition. *Comb.*, 35(6):703–747, 2015.
- [VZ12] Salil P. Vadhan and Colin Jia Zheng. Characterizing pseudoentropy and simplifying pseudorandom generator constructions. In *Proceedings of the 44th Symposium on Theory of Computing Conference, STOC*, pages 817–836. ACM, 2012.
- [YZ16] Yu Yu and Jiang Zhang. Cryptography with auxiliary input and trapdoor from constant-noise LPN. In *Advances in Cryptology - CRYPTO*, volume 9814, pages 214–243, 2016.
- [YZW⁺19] Yu Yu, Jiang Zhang, Jian Weng, Chun Guo, and Xiangxue Li. Collision resistant hashing from sub-exponential learning parity with noise. In *Advances in Cryptology - ASIACRYPT*, volume 11922, pages 3–24, 2019.
- [Zha25] Mark Zhandry. Optimal traitor tracing from pairings. In *Advances in Cryptology - EUROCRYPT*, volume 15603, pages 305–335, 2025.